**ALL DATA ANALYTICS PROJECTS**
***(by Om Satyawan Pathak)***

**Name:** Om Satyawan Pathak
**Github Profile:** https://github.com/omsatyawanpathakgit
**Linkedin:** www.linkedin.com/in/om-satyawan-pathak-029b02368
**Email:** omsatyawanpathakwebdevelopment@gmail.com

**NETFLIX EDA PROJECT**
**(in Python, MS PowerBI, MS Excel, SQL)**

***By Om Satyawan Pathak***
***Github:***
*https://github.com/omsatyawanpathakgit/DataAnalysisProjects/tree/main/NETFLIX%20EDA/NETFLIX%20EDA*

# NETFLIX EDA PROJECT GUIDE


**Project by:**
***Name:*** *Om Satyawan Pathak*
***Email:*** *omsatyawanpathakwebdevelopment@gmail.com*
***LinkedIn Profile Link:*** *www.linkedin.com/in/om-satyawan-pathak-029b02368*
***Github Profile Link:*** *https://github.com/omsatyawanpathakgit*

Performed exploratory data analysis (EDA) for Netflix using the following tools: MS-Excel, MS Power BI and Python Programming.

**In Python:**
1) Contribution by Content-Type (means; Number of Movies v/s Number of TV-Shows)
2) Yearly Contribution (including both the content-types).
3) Top-10 Genres having the most contribution.
4) Top-10 Countries having the most contribution.
5) Occurrence of all ratings
6) Average duration of Movies

**In MS-Excel:**
***(Data cleaning was done using Python)***

***(Name of the Jupyter Notebook: cleaning_netflix_for_Excel.ipynb)***

1) Content-wise Contribution
2) Occurrence of Ratings
3) Genre-wise Contribution
4) Monthly Contribution
5) Yearly Trends
6) Director-wise Contribution
7) Country-wise Contribution

**In MS PowerBI:**

1) Yearly Trends
2) Content-Type contribution
3) Genre-wise Contribution
4) Ratings Occurrence
5) Country-wise Contribution

**In SQL:**

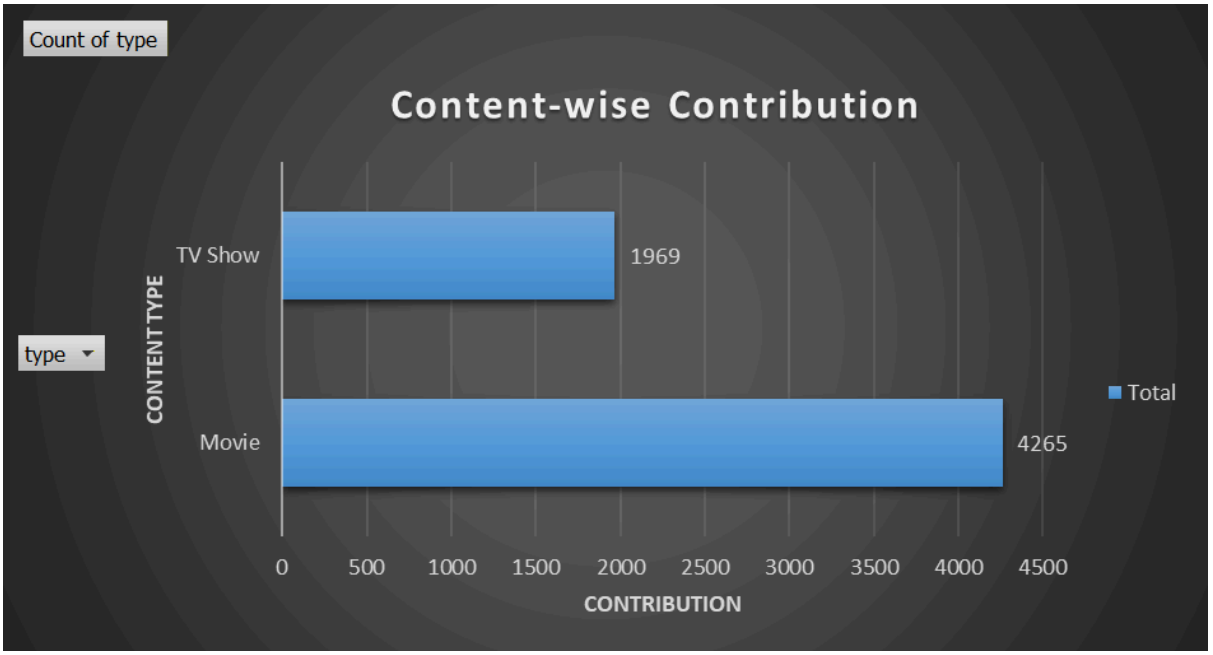
1) Total no. of Movies and TV-Shows (Both separately)
2) Countries with most contribution
3) Years with most contribution
4) Ratings with most contribution
5) Top 10 Genres
5) Top 5 Movies with the Longest duration
6) Directors who contributed most content to Netflix
7) Yearly Trends

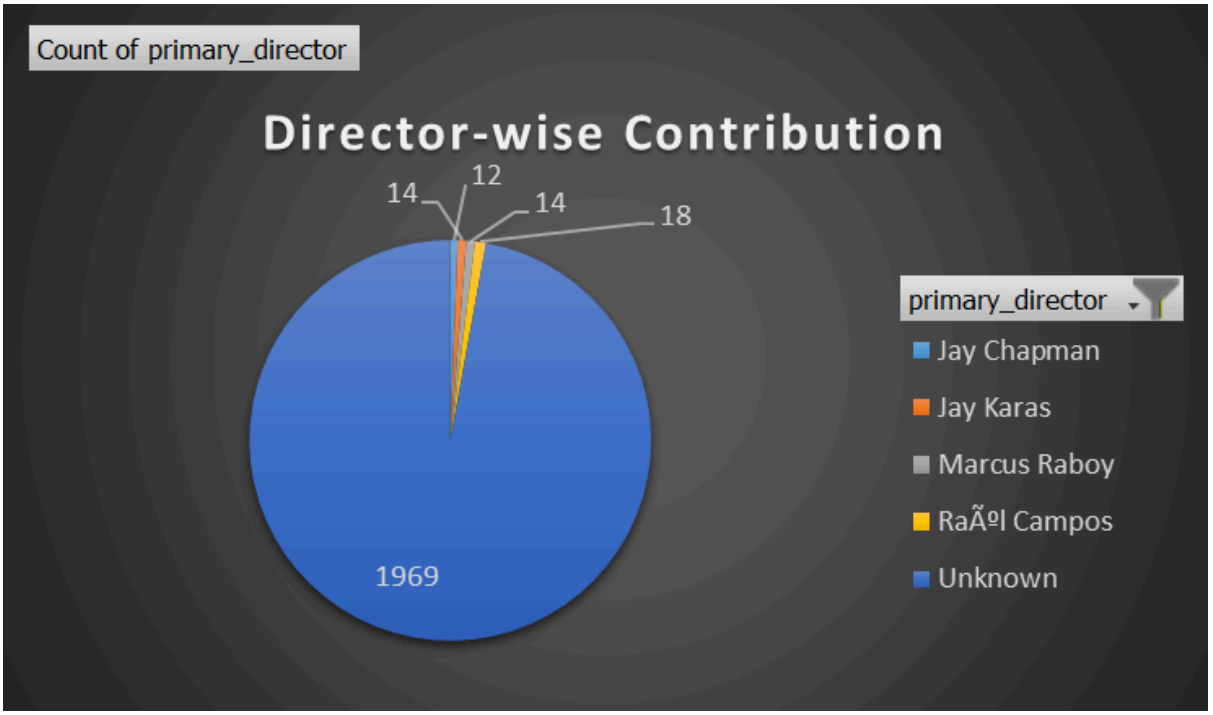
# FINAL EXCEL EDA FOR NETFLIX:

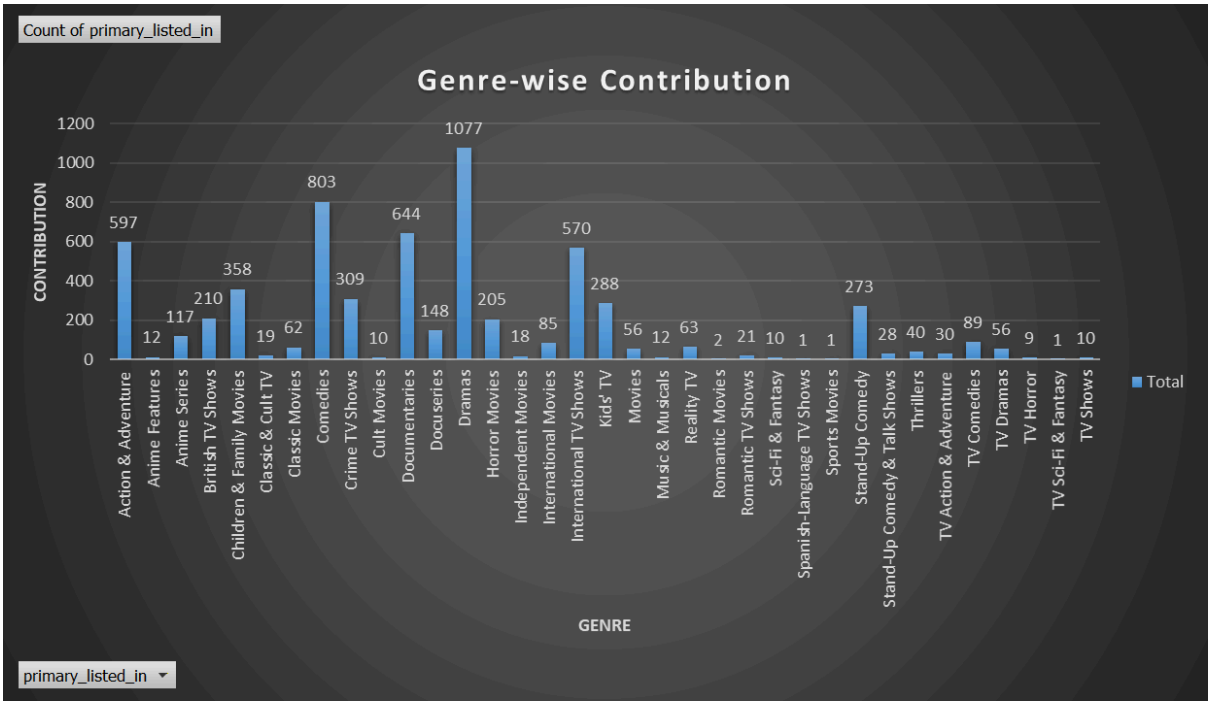
**Insights drawn:**

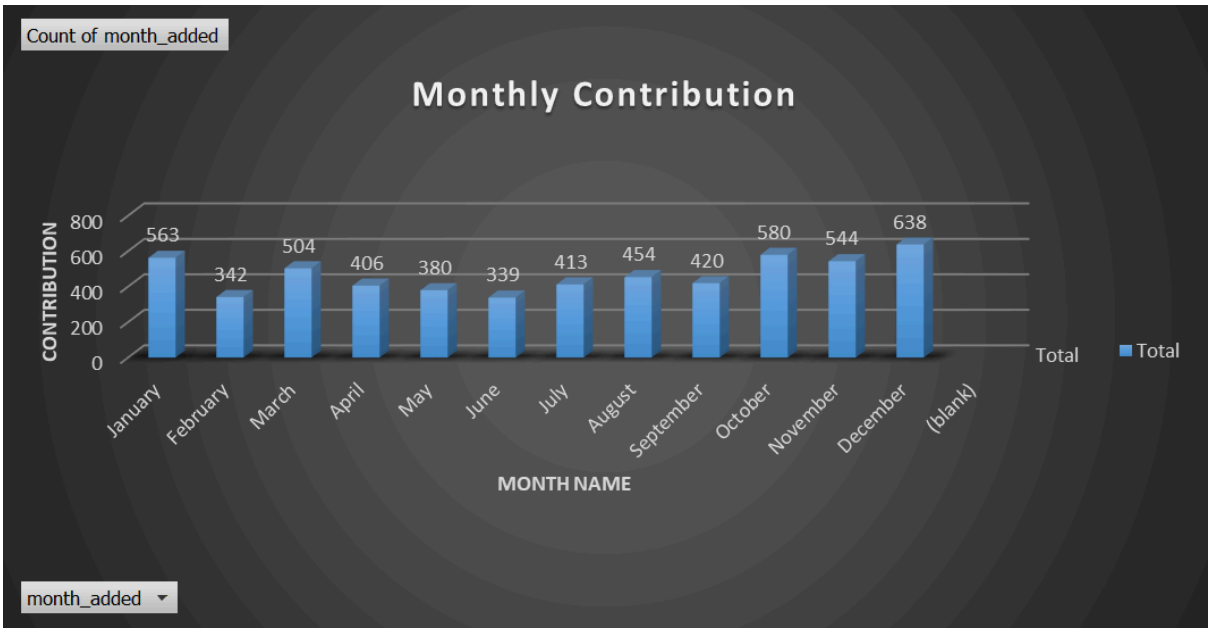
1) Movies dominate over TV Shows.
2) **Dominating ratings are:**
   i) TV-MA
   ii) TV-14
   iii) TV-PG.
3) Most of the content was produced in *December*.
4) **Dominating Genres are:**
   i) Dramas
   ii) Comedies
   iii) Action & Adventure.
5) **Top 4 Directors are:**
   i) Jay Chapman
   ii) Jay Karas
   iii) Marcus Raboy
   iv) Raúl Campos
6) Most of the contents were produced in the period of *2015- 2020.*
7) Most of the content was contributed by the *United States.*

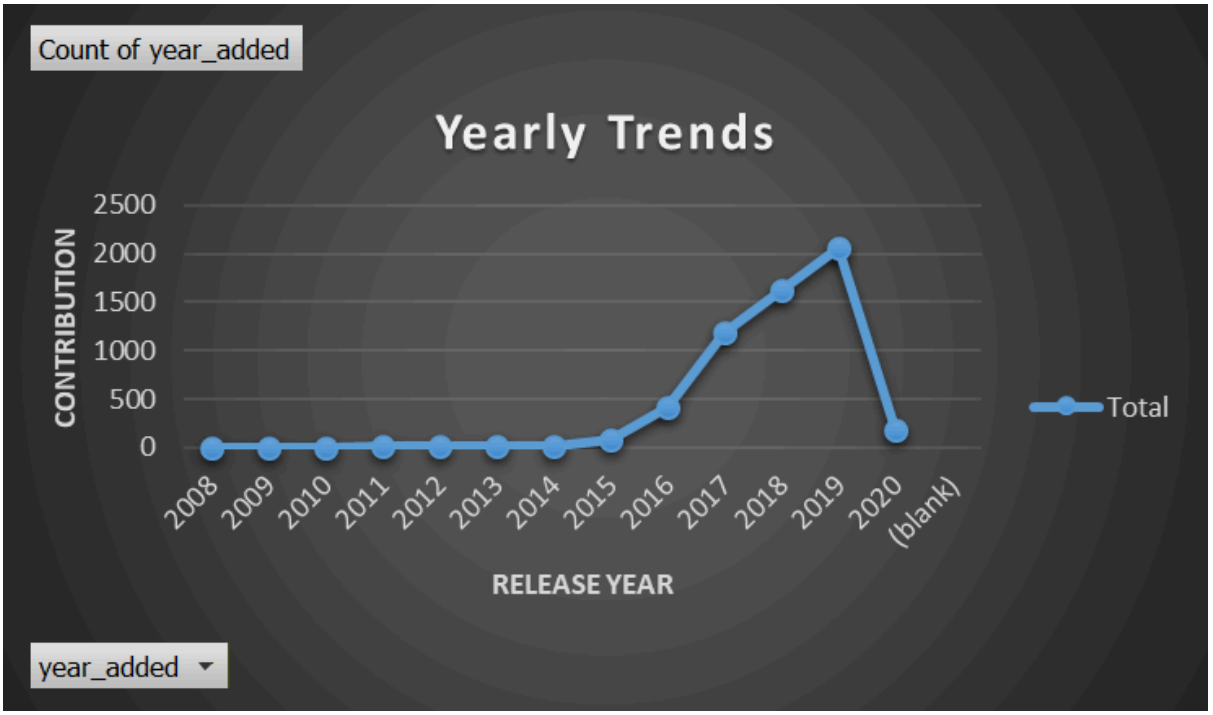
**Visualizations drawn:**

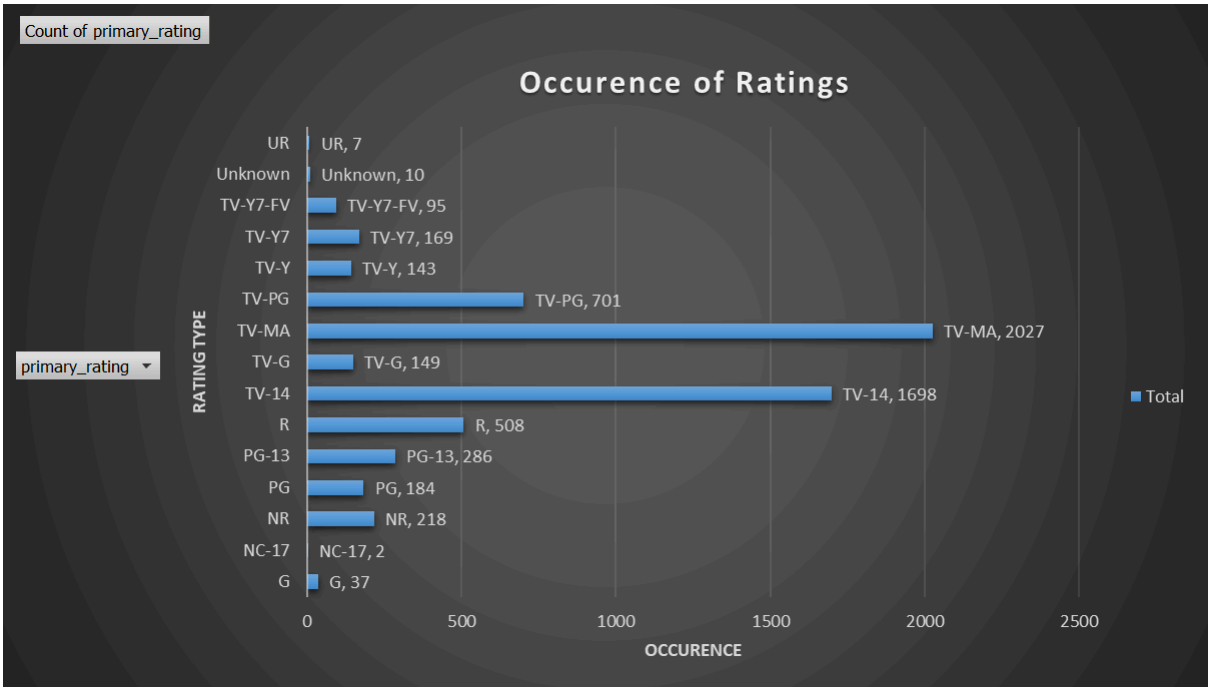

Count of type
Content-wise Contribution
type
CONTENT TYPE
TV Show
Movie
1969
4265
Total
0
500
1000
1500
2000
2500
3000
3500
4000
4500
CONTRIBUTION

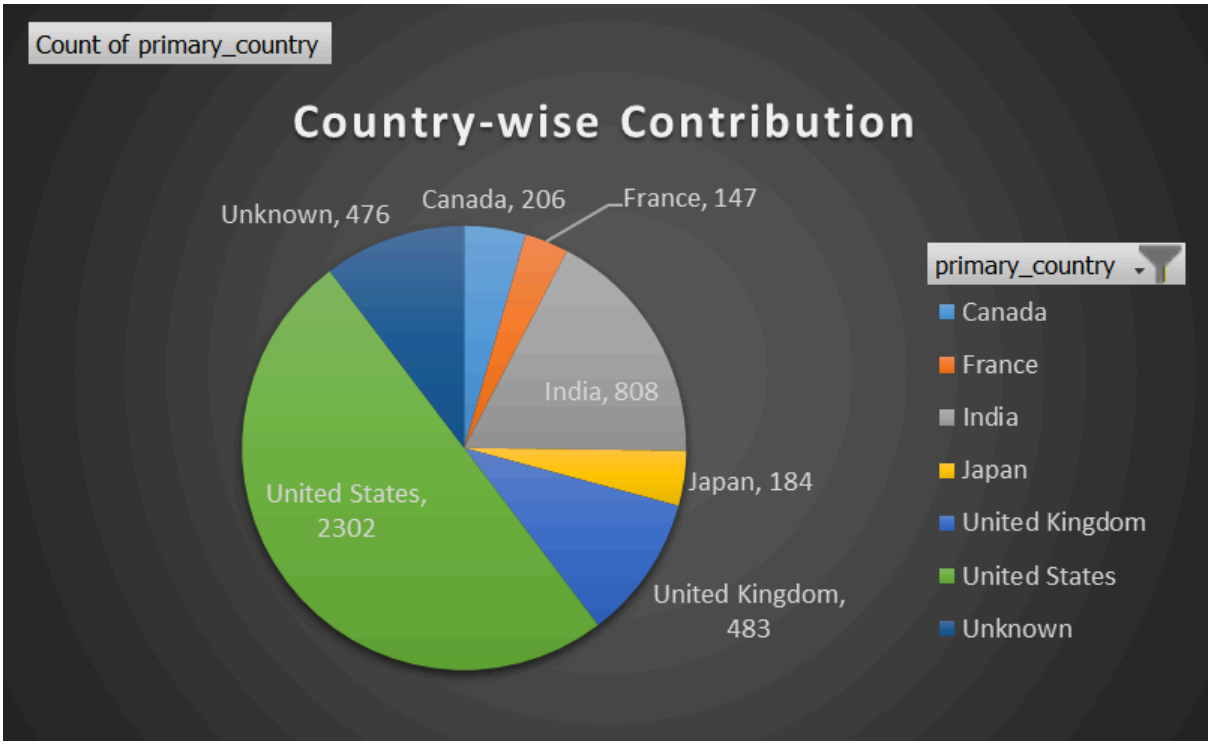

Count of primary_director
Director-wise Contribution
14
12
14
18
1969
primary_director
Jay Chapman
Jay Karas
Marcus Raboy
RaÃºl Campos
Unknown


Count of primary_listed_in
Genre-wise Contribution
CONTRIBUTION
1200
1000
800
600
400
200
0
597
12
117
210
358
19
62
803
309
10
644
148
1077
205
18
85
570
288
56
12
63
2
21
10
1
1
273
28
40
30
89
56
9
1
10
Action & Adventure
Anime Features
Anime Series
British TV Shows
Children & Family Movies
Classic & Cult TV
Classic Movies
Comedies
Crime TV Shows
Cult Movies
Documentaries
Docuseries
Dramas
Horror Movies
Independent Movies
International Movies
International TV Shows
Kids' TV
Movies
Music & Musicals
Reality TV
Romantic Movies
Romantic TV Shows
Sci-Fi & Fantasy
Spanish-Language TV Shows
Sports Movies
Stand-Up Comedy
Stand-Up Comedy & Talk Shows
Thrillers
TV Action & Adventure
TV Comedies
TV Dramas
TV Horror
TV Sci-Fi & Fantasy
TV Shows
GENRE
Total
primary_listed_in

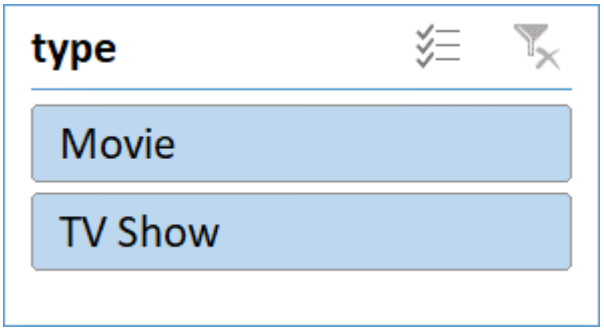

Count of month_added
Monthly Contribution
CONTRIBUTION
800
600
400
200
0
563
342
504
406
380
339
413
454
420
580
544
638
Total
January
February
March
April
May
June
July
August
September
October
November
December
(blank)
MONTH NAME
Total
month_added

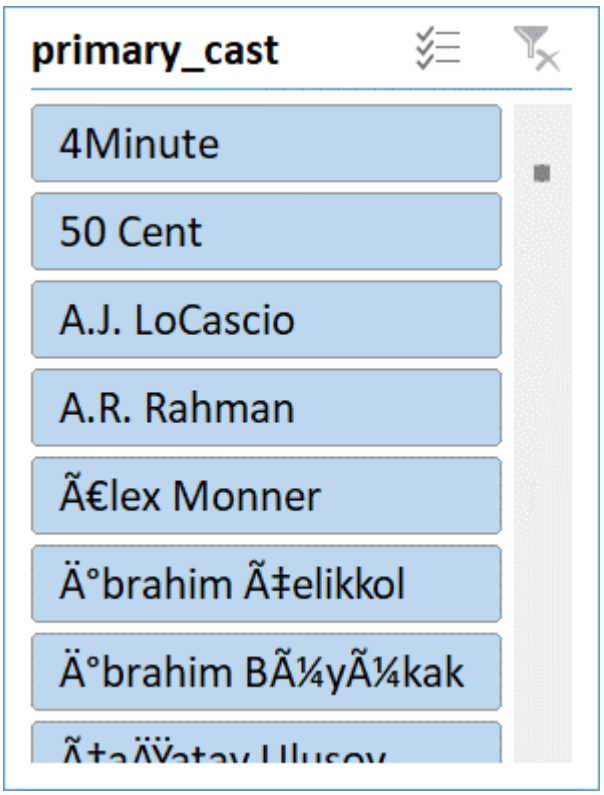

Count of year_added
Yearly Trends
CONTRIBUTION
2500
2000
1500
1000
500
0
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
(blank)
RELEASE YEAR
Total
year_added


Count of primary_rating
Occurence of Ratings
primary_rating
RATING TYPE
UR
Unknown
TV-Y7-FV
TV-Y7
TV-Y
TV-PG
TV-MA
TV-G
TV-14
R
PG-13
PG
NR
NC-17
G
UR, 7
Unknown, 10
TV-Y7-FV, 95
TV-Y7, 169
TV-Y, 143
TV-PG, 701
TV-MA, 2027
TV-G, 149
TV-14, 1698
R, 508
PG-13, 286
PG, 184
NR, 218
NC-17, 2
G, 37
0
500
1000
1500
2000
2500
OCCURENCE
Total

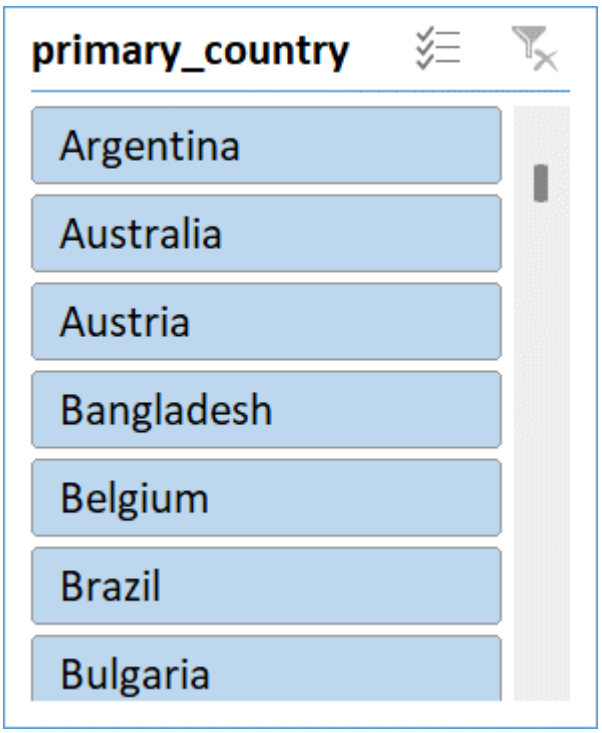

Count of primary_country
Country-wise Contribution
Unknown, 476
Canada, 206
France, 147
India, 808
Japan, 184
United States, 2302
United Kingdom, 483
primary_country
Canada
France
India
Japan
United Kingdom
United States
Unknown

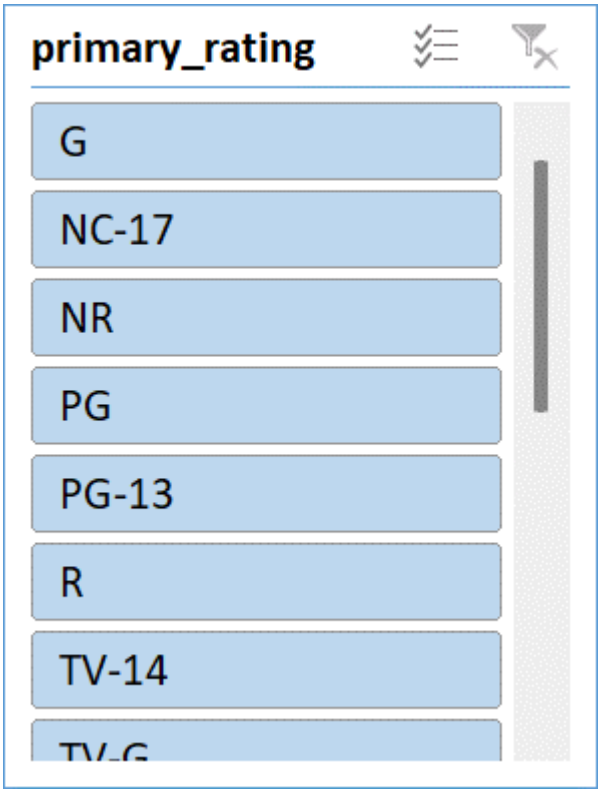
**Slicers:**

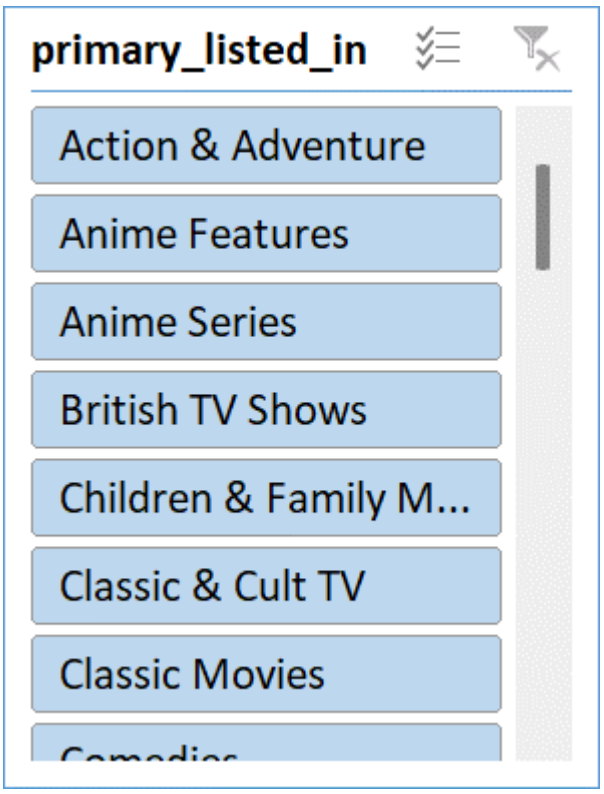

type
Movie
TV Show

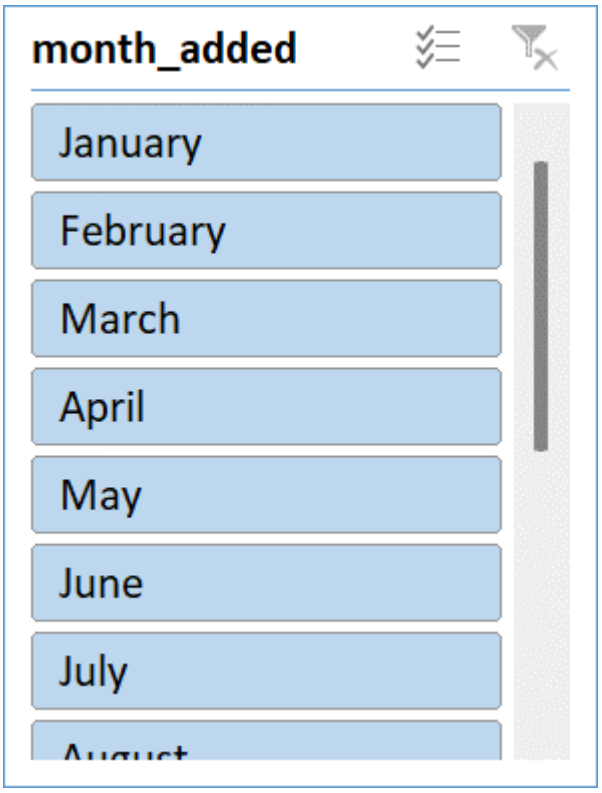

primary_cast
4Minute
50 Cent
A.J. LoCascio
A.R. Rahman
Ã€lex Monner
Ä°brahim Ã‡elikkol
Ä°brahim BÃ¼yÃ¼kak


year_added
2008
2009
2010
2011
2012
2013
2014


primary_country
Argentina
Australia
Austria
Bangladesh
Belgium
Brazil
Bulgaria


primary_rating
G
NC-17
NR
PG
PG-13
R
TV-14


primary_listed_in
Action & Adventure
Anime Features
Anime Series
British TV Shows
Children & Family M...
Classic & Cult TV
Classic Movies


month_added
January
February
March
April
May
June
July

# PYTHON Exploratory-Data-Analysis:

In [ ]:

# IMPORT REQUIRED LIBRARIES:

In [14]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from wordcloud import WordCloud


import warnings
warnings.filterwarnings('ignore')
```

In [15]:
```
df = pd.read_csv("netflix_titles.csv")
print("Initial shape:", df.shape)
df.head()
```

```
Initial shape: (6234, 12)
```

Out[15]:

| | show_id | type | title | director | cast | country | date_added | rele |
|---|---|---|---|---|---|---|---|---|
| **0** | 81145628 | Movie | Norm of the North: King Sized Adventure | Richard Finn, Tim Maltby | Alan Marriott, Andrew Toth, Brian Dobson, Cole... | United States, India, South Korea, China | September 9, 2019 | |
| **1** | 80117401 | Movie | Jandino: Whatever it Takes | NaN | Jandino Asporaat | United Kingdom | September 9, 2016 | |
| **2** | 70234439 | TV Show | Transformers Prime | NaN | Peter Cullen, Sumalee Montano, Frank Welker, J... | United States | September 8, 2018 | |
| **3** | 80058654 | TV Show | Transformers: Robots in Disguise | NaN | Will Friedle, Darren Criss, Constance Zimmer, ... | United States | September 8, 2018 | |
| **4** | 80125979 | Movie | #realityhigh | Fernando Lebrija | Nesta Cooper, Kate Walsh, John Michael Higgins... | United States | September 8, 2017 | |

# DATA CLEANING:

In [16]:

```
print("\nMissing values per column:\n")
print(df.isnull().sum())

df['director'].fillna('Unknown', inplace=True)
df['cast'].fillna('Unknown', inplace=True)
df['country'].fillna('Unknown', inplace=True)
df['rating'].fillna('Unknown', inplace=True)
df['duration'].fillna('Unknown', inplace=True)
df['date_added'].fillna('Unknown', inplace=True)

df.drop_duplicates(inplace=True)

df['date_added'] = pd.to_datetime(df['date_added'], errors='coerce')
```

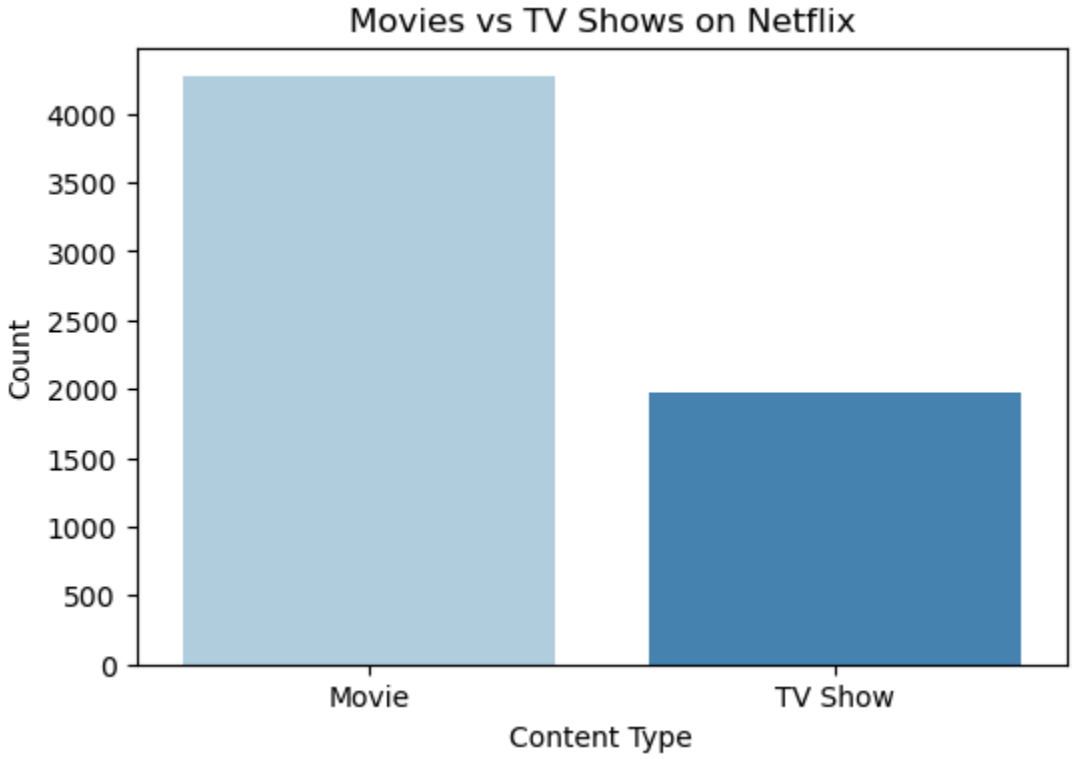
```
# Extract Year and Month from date_added
df['year_added'] = df['date_added'].dt.year
df['month_added'] = df['date_added'].dt.month_name()

# Extract numeric duration (for Movies only)
def extract_duration(x):
    try:
        if 'min' in x:
            return int(x.split()[0])
        else:
            return np.nan
    except:
        return np.nan

df['duration_num'] = df['duration'].apply(extract_duration)

df['primary_country'] = df['country'].apply(lambda x: x.split(',')[0] if x !=
```

```
Missing values per column:

show_id            0
type               0
title              0
director        1969
cast             570
country          476
date_added        11
release_year       0
rating            10
duration           0
listed_in          0
description        0
dtype: int64
```

# EDA:

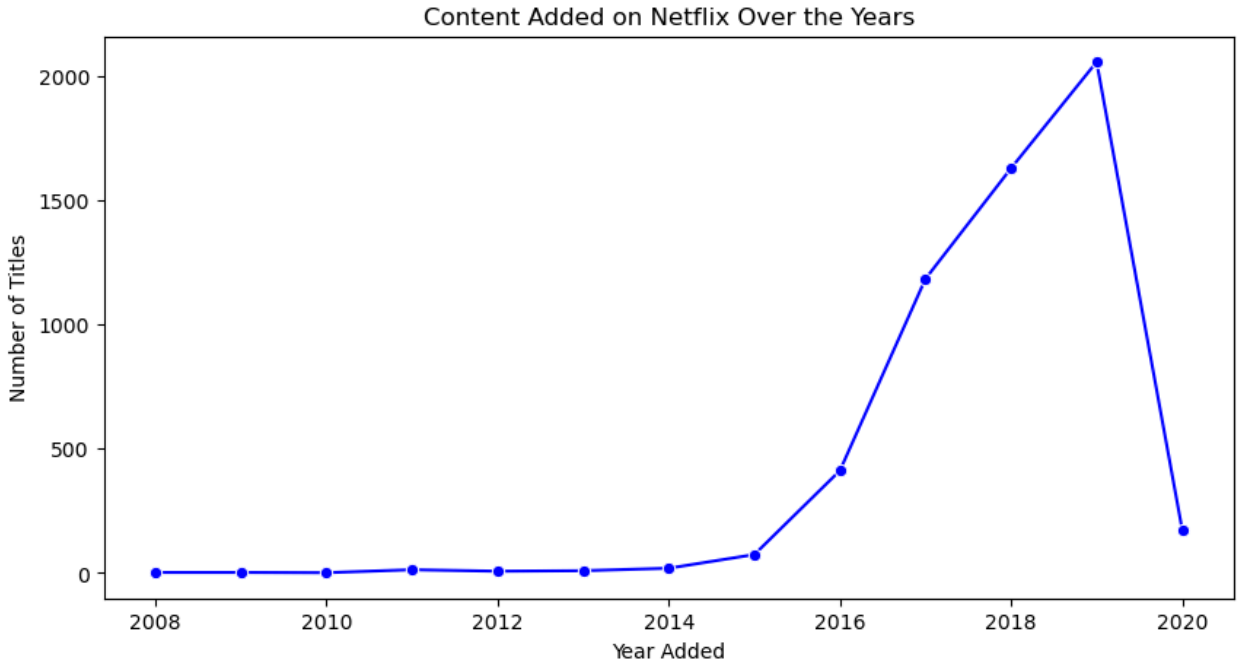
In [17]:
```
print("\nDataset after cleaning and feature creation:\n")
print(df.info())


# 1. NUMBER OF MOVIES v/s TV Shows:
plt.figure(figsize=(6,4))
sns.countplot(x='type', data=df, palette='Blues')
plt.title('Movies vs TV Shows on Netflix')
plt.xlabel('Content Type')
plt.ylabel('Count')
plt.show()
```

```
Dataset after cleaning and feature creation:

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6234 entries, 0 to 6233
Data columns (total 16 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   show_id          6234 non-null   int64
 1   type             6234 non-null   object
 2   title            6234 non-null   object
 3   director         6234 non-null   object
 4   cast             6234 non-null   object
 5   country          6234 non-null   object
 6   date_added       5583 non-null   datetime64[ns]
 7   release_year     6234 non-null   int64
 8   rating           6234 non-null   object
 9   duration         6234 non-null   object
 10  listed_in        6234 non-null   object
 11  description      6234 non-null   object
 12  year_added       5583 non-null   float64
 13  month_added      5583 non-null   object
 14  duration_num     4265 non-null   float64
 15  primary_country  6234 non-null   object
dtypes: datetime64[ns](1), float64(2), int64(2), object(11)
memory usage: 779.4+ KB
None
```

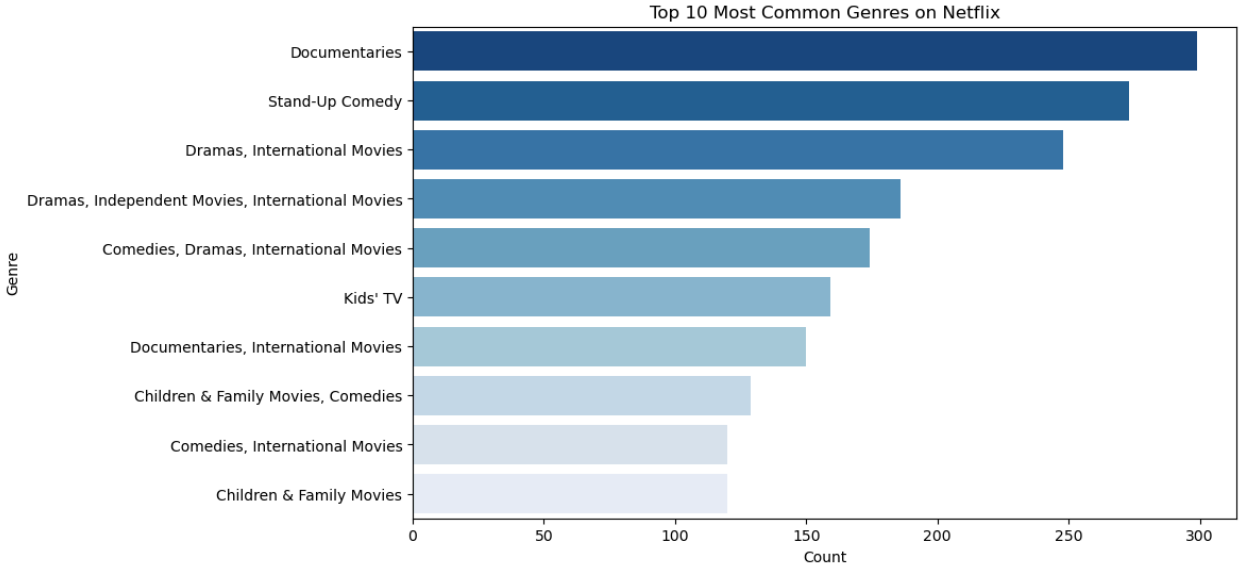



In [18]:
```
# 2. YEARLY TRENDS:
yearly_count = df['year_added'].value_counts().sort_index()
plt.figure(figsize=(10,5))
```

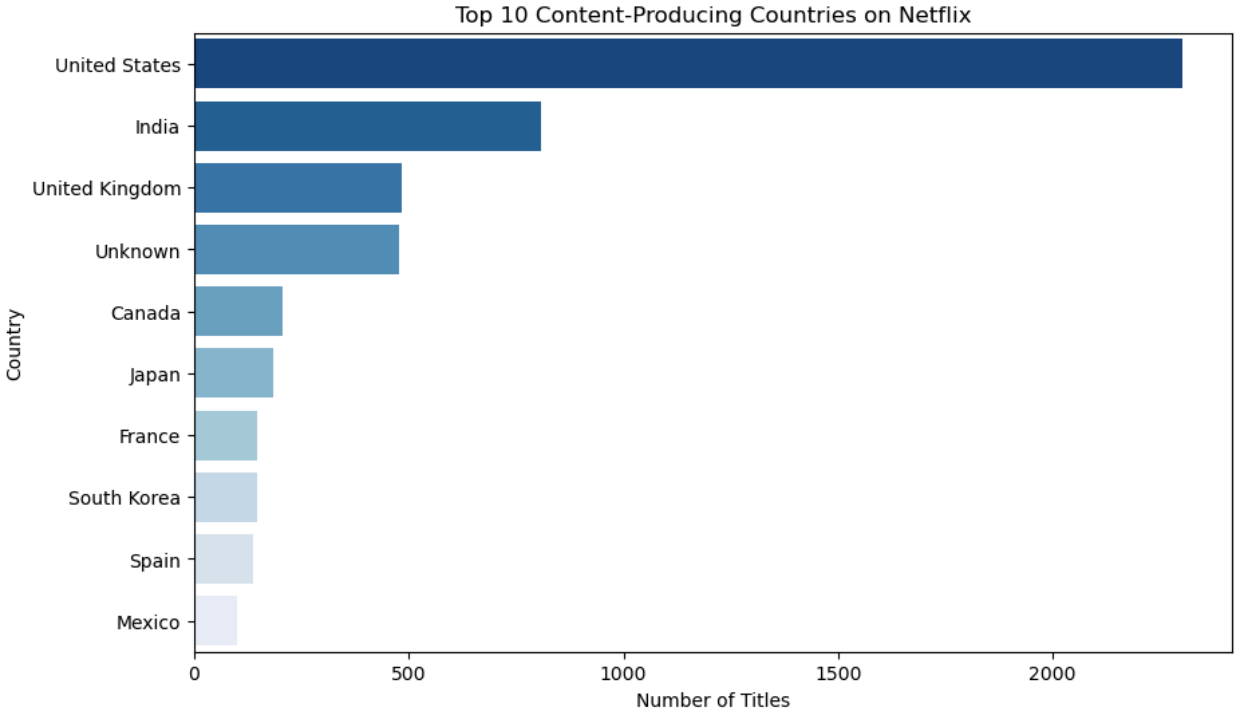
```
sns.lineplot(x=yearly_count.index, y=yearly_count.values, marker='o', color='b
plt.title('Content Added on Netflix Over the Years')
plt.xlabel('Year Added')
plt.ylabel('Number of Titles')
plt.show()
```



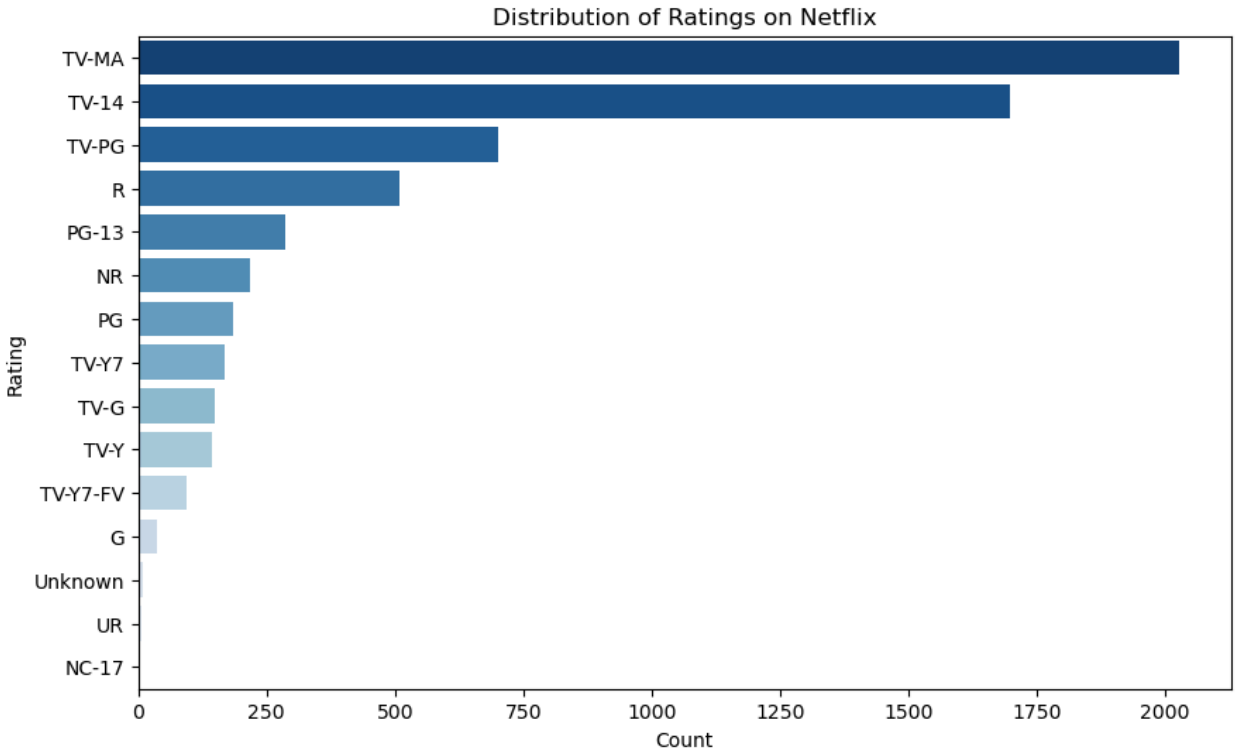

In [19]:
```
# 3. HIGH PERFORMANCE GENRES:
plt.figure(figsize=(10,6))
genre_data = df['listed_in'].value_counts().head(10)
sns.barplot(y=genre_data.index, x=genre_data.values, palette='Blues_r')
plt.title('Top 10 Most Common Genres on Netflix')
plt.xlabel('Count')
plt.ylabel('Genre')
plt.show()
```



In [20]:
```
# 4. TOP-10 CONTRIBUTING COUNTRIES:
country_data = df['primary_country'].value_counts().head(10)
plt.figure(figsize=(10,6))
sns.barplot(y=country_data.index, x=country_data.values, palette='Blues_r')
plt.title('Top 10 Content-Producing Countries on Netflix')
plt.xlabel('Number of Titles')
plt.ylabel('Country')
plt.show()
```




In [21]:
```
# 5.RATINGS DISTRIBUTION:
plt.figure(figsize=(10,6))
sns.countplot(y='rating', data=df, order=df['rating'].value_counts().index, pa
plt.title('Distribution of Ratings on Netflix')
plt.xlabel('Count')
plt.ylabel('Rating')
plt.show()
```




In [22]:
```
# 6. AVERAGE DURATION OF MOVIES:
avg_duration = df[df['type']=='Movie']['duration_num'].mean()
print(f"\nAverage Movie Duration: {avg_duration:.2f} minutes")
```

```
Average Movie Duration: 99.10 minutes
```

In [23]:
```
# 7. GENRES WORDCLOUD:
plt.figure(figsize=(10,7))
text = ' '.join(df['listed_in'].dropna().astype(str))
wordcloud = WordCloud(width=1000, height=600, background_color='black', colorm
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title('Most Common Genres on Netflix', fontsize=15)
plt.show()
```

Most Common Genres on Netflix


In [24]:
```
df_directors = df.copy()
df_directors["director"] = df_directors["director"].str.split(',')
df_directors=df_directors.explode("director").reset_index(drop=True)
df_directors.head()
```

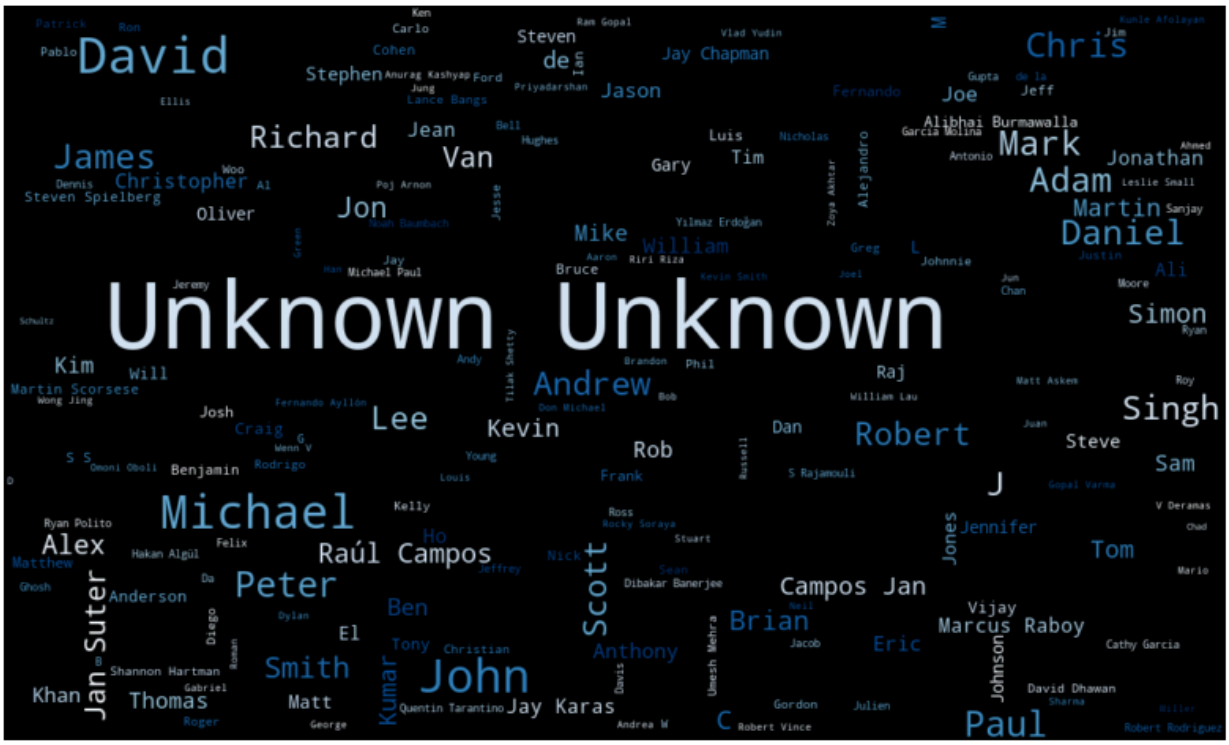
Out[24]:

| | show_id | type | title | director | cast | country | date_added | rele |
|---|---|---|---|---|---|---|---|---|
| **0** | 81145628 | Movie | Norm of the North: King Sized Adventure | Richard Finn | Alan Marriott, Andrew Toth, Brian Dobson, Cole... | United States, India, South Korea, China | 2019-09-09 | |
| **1** | 81145628 | Movie | Norm of the North: King Sized Adventure | Tim Maltby | Alan Marriott, Andrew Toth, Brian Dobson, Cole... | United States, India, South Korea, China | 2019-09-09 | |
| **2** | 80117401 | Movie | Jandino: Whatever it Takes | Unknown | Jandino Asporaat | United Kingdom | 2016-09-09 | |
| **3** | 70234439 | TV Show | Transformers Prime | Unknown | Peter Cullen, Sumalee Montano, Frank Welker, J... | United States | 2018-09-08 | |
| **4** | 80058654 | TV Show | Transformers: Robots in Disguise | Unknown | Will Friedle, Darren Criss, Constance Zimmer, ... | United States | 2018-09-08 | |

In [25]:

```
# 8. DIRECTORS WORDCLOUD:
plt.figure(figsize=(10,7))
text = ' '.join(df_directors['director'].dropna().astype(str))
wordcloud = WordCloud(width=1000, height=600, background_color='black', colorm
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title('Most Common Directors on Netflix', fontsize=15)
plt.show()
```

Most Common Directors on Netflix




In [26]:
```
# 9. COUNTRIES-WISE MAP:
try:
    import plotly.express as px
    country_counts = df['primary_country'].value_counts().reset_index()
    country_counts.columns = ['country', 'count']
    fig = px.choropleth(country_counts, locations='country', locationmode='cou
                        color='count', title='Netflix Content by Country',
                        color_continuous_scale='Blues')
    fig.show()
except:
    print("Plotly not installed. Install via: pip install plotly")

# SAVE CLEANED DATA:
df.to_csv("netflix_cleaned.csv", index=False)
print("\n Data cleaning complete. File saved as netflix_cleaned.csv")
```

```
Data cleaning complete. File saved as netflix_cleaned.csv
```

In [ ]:

# Netflix Titles Dataset — Storytelling & Insights (EDA Report)

## ## 1. Introduction – What This Analysis Is About

Netflix hosts thousands of movies and TV shows across genres and countries. In this analysis, we explore:

- What types of content Netflix hosts
- Which countries contribute the most
- Which genres dominate

- How the content library has grown
- Audience rating patterns
- Duration trends
- Popular directors and themes

This storytelling section explains the insights discovered during the Exploratory Data Analysis (EDA) of the **Netflix Titles Dataset (6,234 records)**.

---

# ## 2. Data Overview

The dataset ( `netflix_titles.csv` ) includes:

- Title, type (Movie/TV Show)
- Director & cast
- Country of origin
- Date added to Netflix
- Release year
- Rating
- Duration
- Listed genres
- Description

Before analysis, the data required cleaning to ensure accuracy.

---

# ## 3. Data Cleaning Summary

Key cleaning steps performed:

## ** Handling Missing Values**

- Replaced missing values in `director` , `cast` , `country` , `rating` , `duration` , `date_added` with `"Unknown"` .

## ** Removing Duplicates**

- Duplicate records were removed.

## ** Converting Columns**

- Converted `date_added` to a proper datetime format.

- Extracted:
    - `year_added`
    - `month_added`
- Extracted numeric duration (for movies only).

** Structuring Country Info**

- Created `primary_country` containing only the first country listed.

After cleaning, the dataset was consistent and ready for exploration.

---

# ## 4. Insights & Storytelling

---

## ### 4.1 Movies vs TV Shows

Netflix hosts more **Movies** than **TV Shows**.

**Insight:** Movies form the majority of Netflix's library, but TV shows remain vital for keeping users engaged through long-form content.

---

## ### 4.2 Netflix Library Growth Over the Years

Plotting content by `year_added` shows a major rise between **2015 to 2020**, after which additions stabilize.

**Insight:** This period aligns with Netflix's global expansion and shift toward producing original content.

---

## ### 4.3 Top Content Genres

The most common genres include:

- International Movies
- Dramas
- Comedies

- Documentaries
- Action & Adventure

**Insight:** Drama and International content dominate, indicating Netflix’s focus on global audiences and diverse storytelling.

---

## ### 4.4 Top Contributing Countries

The top content-producing countries:

1. United States
2. India
3. United Kingdom
4. Canada
5. Japan

**Insight:** The US is the largest contributor, but India’s rising position reflects Netflix’s investment in regional markets.

---

## ### 4.5 Ratings Distribution

Most common ratings:

- TV-MA
- TV-14
- TV-PG

**Insight:** Netflix leans heavily toward **mature content**, which attracts adult audiences globally.

---

## ### 4.6 Average Movie Duration

Average movie duration on Netflix: **~99 minutes**

**Insight:** Most Netflix movies follow the standard film length of 90–110 minutes.

---

## ### 4.7 Genre Word Cloud

A word cloud generated from `listed_in` highlights:

- Drama
- Comedy
- International
- Documentary
- Action

**Insight:** These categories reflect Netflix’s core content pillars.

---

## ### 4.8 Director Frequency

After splitting director names, a director word cloud reveals recurring creators.

**Insight:** Certain directors appear frequently, especially in documentaries, comedy specials, and international titles.

---

## ### 4.9 Country-Wise Global View

A choropleth map shows wide global participation, with strong contributions from North America, Europe, and Asia.

**Insight:** Netflix’s catalog has global diversity, aligning with its international user base.

---

# ## 5. Conclusion – What the Data Tells Us

This EDA reveals that Netflix:

- Primarily hosts **Movies**, with steady growth in TV content.
- Expanded rapidly between **2015–2020**.
- Focuses heavily on **Drama**, **Comedy**, and **International** genres.
- Receives major content contributions from the **USA**, but countries like **India** and **UK** are also significant.
- Favors **mature content (TV-MA)**, targeted at adult audiences.
- Offers movies with a typical duration of around **99 minutes**.

- Showcases a diverse range of directors and countries.

Overall, Netflix’s strategy is built around **global reach**, **diverse genres**, and **strong movie dominance**, supported by increasing international content offerings.

In [ ]:

In [ ]:

# NETFLIX POWERBI EDA (aka Exploratory Data-Analysis)

**INSIGHTS DRAWN:**

1. In terms of contribution; movies have more occurrence over tv-shows.
2. Most content was produced in the period of 2015 to 2020.
3. Most of the contents were having these ratings:
    a. TV-MA (Mature Audience Only)
    b. TV-14 (Parents Strongly Cautioned)
    c. TV-PG (Parental Guidance Suggested)
    d. R (Restricted)
    e. PG-13 (Parents Strongly Cautioned)
4. Dramas is the dominant genre.
5. Most of the movies (and/or) tv-shows are from United-States.

**DAX QUERIES USED:**

```
Total Titles = COUNTROWS(netflix_titles)
```

```
Total Movies = CALCULATE(COUNTROWS(netflix_titles), netflix_titles[type] = "Movie")
```

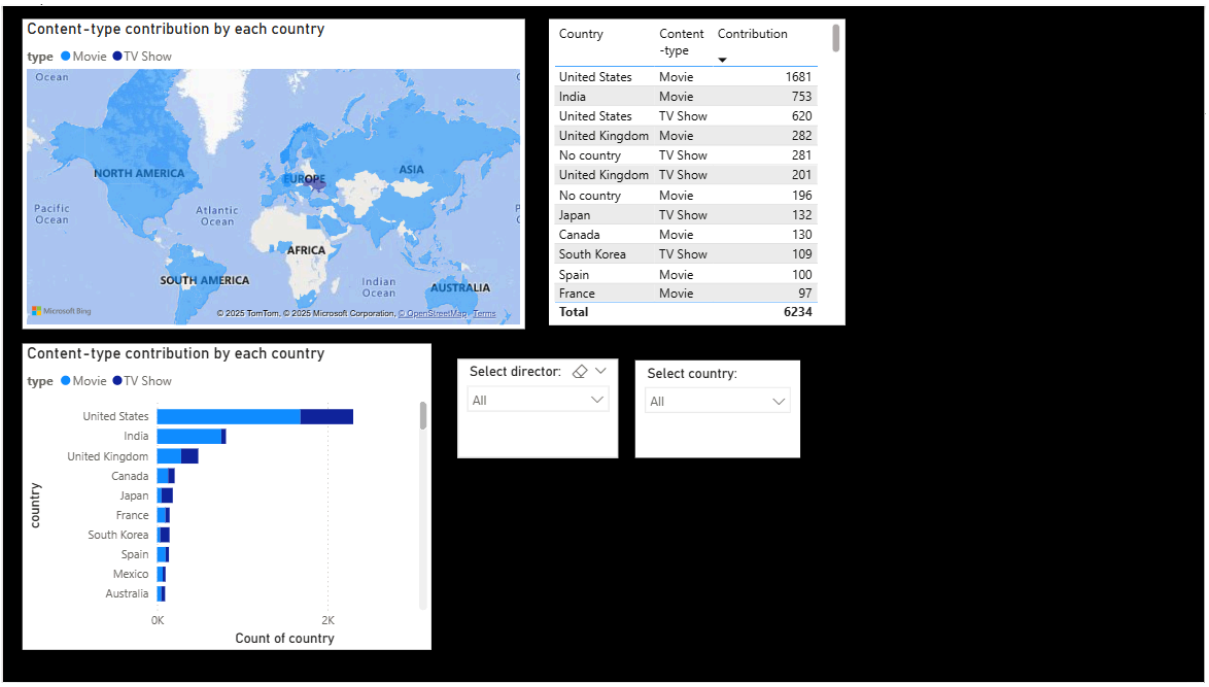
```
Total TV Shows = CALCULATE(COUNTROWS(netflix_titles), netflix_titles[type] = "TV Show")
```

```
Movies Contribution in % = DIVIDE([Total Movies],[Total Titles],0)*100
```

```
TV Shows Contribution in % = DIVIDE([Total TV Shows],[Total Titles],0)*100
```

**DASHBOARD IMAGES:**


4265
Total Movies
1969
Total TV Shows
31.57
TV Shows Contribution in %
68.39
Movies Contribution in %
Select content-type:
Movie
TV Show
Yearly Trend of Content Added
type Movie TV Show
Contribution
Year
Yearly contribution
type Movie TV Show
Content Release Year
Contribution
Select director:
All
Select country:
All
release_year
All
Content-Type contribution
1.97K (31....)
4.27K (68.42%)
type
Movie
TV Sh...
Ratings
Contribution
Rating Type
Contribution by Genre
Contribution
Genre
Contribution by Genre
Genre Contribution
Dramas 1077
Comedies 803
Documentaries 644
Action & Adventure 597
International TV Shows 570
Children & Family Movies 358
Crime TV Shows 309
Total 6233


Content-type contribution by each country
type Movie TV Show
NORTH AMERICA
EUROPE
ASIA
AFRICA
SOUTH AMERICA
AUSTRALIA
Pacific Ocean
Atlantic Ocean
Indian Ocean
Country Content-type Contribution
United States Movie 1681
India Movie 753
United States TV Show 620
United Kingdom Movie 282
No country TV Show 281
United Kingdom TV Show 201
No country Movie 196
Japan TV Show 132
Canada Movie 130
South Korea TV Show 109
Spain Movie 100
France Movie 97
Total 6234
Content-type contribution by each country
type Movie TV Show
country
Count of country
Select director:
All
Select country:
All

In [14]:

```
# for loading Netflix's image logo in this jupyter notebook:
from IPython.display import Image
Image("netflix_Eda_project_logo.jpg")
```

Out[14]:


In [ ]:

In [ ]:

In [ ]:

# EXPLORATORY DATA ANALYSIS (EDA) USING SQL:

In [15]:

```
# Preview of the dataset:
import pandas as pd
df=pd.read_csv("netflix_titles.csv")
df.head(3)
```

Out[15]:

| | show_id | type | title | director | cast | country | date_added | release_year | rating | duration | listed_in | desc |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 81145628 | Movie | Norm of the North: King Sized Adventure | Richard Finn, Tim Maltby | Alan Marriott, Andrew Toth, Brian Dobson, Cole... | United States, India, South Korea, China | September 9, 2019 | 2019 | TV-PG | 90 min | Children & Family Movies, Comedies | pla awe we |

| | show_id | type | title | director | cast | country | date_added | release_year | rating | duration | listed_in | desc |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 80117401 | Movie | Jandino: Whatever it Takes | NaN | Jandino Asporaat | United Kingdom | September 9, 2016 | 2016 | TV-MA | 94 min | Stand-Up Comedy | As riffs chall |
| 2 | 70234439 | TV Show | Transformers Prime | NaN | Peter Cullen, Sumalee Montano, Frank Welker, J... | United States | September 8, 2018 | 2013 | TV-Y7-FV | 1 Season | Kids' TV | W h h allie A |

In [16]:

```
df.isnull().sum()
```

Out[16]:

```
show_id            0
type               0
title              0
director        1969
cast             570
country          476
date_added        11
release_year       0
rating            10
duration           0
listed_in          0
description        0
dtype: int64
```

In [17]:

```
import pymysql

connection = pymysql.connect(
    host="localhost",
    user="root",
    password="",
    database="all_netflix_titles"
)

cursor = connection.cursor()
```

In [ ]:

In [ ]:

# Q1) WHAT ARE THE TOTAL NO. OF RECORDS:

In [18]:

```
cursor.execute("SELECT COUNT(*) AS total_records FROM netflix_titles")
data = cursor.fetchall()

print("Total no. of records in the dataset = ",data[0][0])
```

```
Total no. of records in the dataset =  6235
```

In [ ]:

# Q2) TOTAL NO. OF MOVIES AND TV SHOWS:

In [19]:

```
cursor.execute("SELECT type, COUNT(*) FROM netflix_titles WHERE type!='type' GROUP BY typ
e")
data = cursor.fetchall()

print("These many movies and tv-shows (respectively) exist in the dataset : ")
for i in data:
    if(i[0]=="Movie"):
        print(i[1],"movies")
    else:
        print(i[1],"tv shows")
```

```
These many movies and tv-shows (respectively) exist in the dataset :
4265 movies
1969 tv shows
```

In [ ]:

```

```

# Q3) NUMBER OF RECORDS WITH MISSING VALUES:

In [20]:

```
cursor.execute("""SELECT COUNT(*) FROM netflix_titles
WHERE director IS NULL
OR director = '' """)

missing_directors = cursor.fetchone()[0]
print(f" Missing Directors: {missing_directors}")
```

```
 Missing Directors: 1969
```

In [21]:

```
cursor.execute("""SELECT COUNT(*) FROM netflix_titles
WHERE cast IS NULL
OR cast = '' """)

missing_cast = cursor.fetchone()[0]
print(f" Missing Cast: {missing_cast}")
```

```
 Missing Cast: 570
```

In [22]:

```
cursor.execute("""SELECT COUNT(*) FROM netflix_titles
WHERE date_added IS NULL
OR date_added = '' """)

missing_dates = cursor.fetchone()[0]
print(f" Missing Dates: {missing_dates}")
```

```
 Missing Dates: 11
```

In [23]:

```
cursor.execute("""SELECT COUNT(*) FROM netflix_titles
WHERE country IS NULL
OR country = '' """)

missing_countries = cursor.fetchone()[0]
print(f" Missing Countries: {missing_countries}")
```

```
 Missing Countries: 476
```

Missing Countries: 476

In [24]:

```
cursor.execute("""SELECT COUNT(*) FROM netflix_titles
WHERE rating IS NULL
OR rating = '' """)

missing_ratings = cursor.fetchone()[0]
print(f" Missing Ratings: {missing_ratings}")
```

```
 Missing Ratings: 10
```

In [ ]:

```

```

# Q4) WHICH COUNTRIES PRODUCE THE MOST CONTENT:

In [25]:

```
cursor.execute("""
    SELECT country, COUNT(*) AS total
    FROM netflix_titles
    WHERE country IS NOT NULL AND country <> ''
    GROUP BY country
    ORDER BY total DESC
    LIMIT 10
""")
top_countries = cursor.fetchall()

print(" Top 10 Countries:")
for country, total in top_countries:
    print(f"{country} → {total}")
```

```
 Top 10 Countries:
United States → 2032
India → 777
United Kingdom → 348
Japan → 176
Canada → 141
South Korea → 136
Spain → 117
France → 90
Mexico → 83
Turkey → 79
```

In [ ]:

```

```

# Q5) IN WHICH YEARS MOST CONTENT WAS ADDED:

In [26]:

```
cursor.execute("""
    SELECT release_year, COUNT(*) AS total
    FROM netflix_titles
    GROUP BY release_year
    ORDER BY total DESC
    LIMIT 10
""")
years = cursor.fetchall()

print(" Top 10 Years:")
for year, total in years:
    print(f"{year} → {total}")
```

Top 10 Years:

```
 Top 10 Years:
2018 → 1063
2017 → 959
2019 → 843
2016 → 830
2015 → 517
2014 → 288
2013 → 237
2012 → 183
2010 → 149
2011 → 136
```

In [ ]:

```
```

# Q6) WHICH RATINGS ARE GIVEN MOST OFTENLY?

In [27]:

```
cursor.execute("""
    SELECT rating, COUNT(*) AS total
    FROM netflix_titles
    WHERE rating IS NOT NULL
    GROUP BY rating
    ORDER BY total DESC
""")
ratings = cursor.fetchall()

print(" Most Common Ratings:")
for rating, total in ratings:
    print(f"{rating} → {total}")
```

```
 Most Common Ratings:
TV-MA → 2027
TV-14 → 1698
TV-PG → 701
R → 508
PG-13 → 286
NR → 218
PG → 184
TV-Y7 → 169
TV-G → 149
TV-Y → 143
TV-Y7-FV → 95
G → 37
 → 10
UR → 7
NC-17 → 2
rating → 1
```

In [ ]:

```
```

# Q7) WHAT ARE THE TOP 10 GENRES:

In [28]:

```
cursor.execute("""
    SELECT listed_in, COUNT(*) AS total
    FROM netflix_titles
    GROUP BY listed_in
    ORDER BY total DESC
    LIMIT 10
""")
genres = cursor.fetchall()
```

```
print(" Top 10 Genres:")
for genre, total in genres:
    print(f"{genre} → {total}")
```

```
 Top 10 Genres:
Documentaries → 299
Stand-Up Comedy → 273
Dramas, International Movies → 248
Dramas, Independent Movies, International Movies → 186
Comedies, Dramas, International Movies → 174
Kids' TV → 159
Documentaries, International Movies → 150
Children & Family Movies, Comedies → 129
Children & Family Movies → 120
Comedies, International Movies → 120
```

In [ ]:

```
```

# Q8) WHAT ARE THE TOP 5 MOVIES WITH THE LONGEST DURATION:

In [29]:

```
cursor.execute("""
    SELECT title, duration
    FROM netflix_titles
    WHERE type = 'Movie'
    ORDER BY CAST(REPLACE(duration, ' min', '') AS UNSIGNED) DESC
    LIMIT 5
""")
longest_movies = cursor.fetchall()

print(" Top 5 Longest Movies:")
for title, duration in longest_movies:
    print(f"{title} → {duration}")
```

```
 Top 5 Longest Movies:
Black Mirror: Bandersnatch → 312 min
Sangam → 228 min
Lagaan → 224 min
Jodhaa Akbar → 214 min
The Irishman → 209 min
```

In [ ]:

```
```

# Q9) WHO ARE THE TOP-10 DIRECTORS:

In [30]:

```
cursor.execute("""
    SELECT director, COUNT(*) AS total
    FROM netflix_titles
    WHERE director IS NOT NULL AND director <> ''
    GROUP BY director
    ORDER BY total DESC
    LIMIT 10
""")
top_directors = cursor.fetchall()

print(" Top 10 Directors:")
for director, total in top_directors:
    print(f"{director} → {total}")
```

```
 Top 10 Directors:
```

```
Raúl Campos, Jan Suter → 18
Marcus Raboy → 14
Jay Karas → 13
Jay Chapman → 12
Martin Scorsese → 9
Steven Spielberg → 9
Johnnie To → 8
David Dhawan → 8
Lance Bangs → 8
Quentin Tarantino → 7
```

In [ ]:

```
```

# Q10) WHAT IS THE YEARLY TREND?

In [31]:

```
cursor.execute("""
    SELECT YEAR(STR_TO_DATE(date_added, '%M %e, %Y')) AS added_year,
           COUNT(*) AS total
    FROM netflix_titles
    WHERE date_added IS NOT NULL
    GROUP BY added_year
    ORDER BY added_year
""")
added_years = cursor.fetchall()

print(" Content Added Per Year:")
for year, total in added_years:
    if(year!="None"):
        print(f"{year} → {total}")
```

```
 Content Added Per Year:
None → 12
2008 → 2
2009 → 2
2010 → 1
2011 → 13
2012 → 7
2013 → 12
2014 → 25
2015 → 90
2016 → 456
2017 → 1300
2018 → 1782
2019 → 2349
2020 → 184
```

In [ ]:

```
```

In [ ]:

```
```

---

## Business Storytelling: Netflix Content Strategy Insights

> ***Dataset Size: 6,235 records (Movies + TV Shows)***

### 1. Content Type Overview

From the total content:

- **Movies:** 4,265
- **TV Shows:** 1,969

**Insight:** Netflix is primarily a **Movie-dominated platform**, with **~68%** of its catalog being movies. However, the increasing popularity of TV shows (series, miniseries, etc.) reflects a **growing trend in long-form storytelling** and binge-worthy content.

**Business Action:** Netflix could balance its catalog by investing more in **TV series and episodic content**, especially in regions where binge-watching behavior is high.

---

## 2. Geographic Distribution

Top producing countries:

- 🇺🇸 **USA (2032)** titles
- 🇮🇳 **India (777)** titles
- 🇬🇧 **UK (348)** titles
- 🇯🇵 Japan, 🇨🇦 Canada, 🇰🇷 South Korea also rank in top 10.

**Insight:** The US dominates content production on Netflix. But there's a **strong international presence** led by India, UK, and Asia-Pacific regions.

**Business Action:** To increase **global market share**, Netflix should **localize content more aggressively in emerging markets** like India and Korea where content growth is already strong.

---

## 3. Yearly Trends

- Most content was released in:
  - 2018 → 1,063 titles
  - 2017 → 959 titles
  - 2019 → 843 titles
- Steady growth from 2014 onwards, peaking around 2018–2019.

**Insight:** Netflix experienced a **content boom between 2017–2019**, which correlates with its aggressive expansion strategy in global markets.

**Business Action:** This historical trend suggests that **periods of content expansion match subscription growth**. Investing again in high-volume content production could reignite growth in stagnant markets.

---

## 4. Ratings Distribution

- **Top Ratings:**
  - TV-MA (2027 titles)
  - TV-14 (1698 titles)
  - TV-PG (701 titles)

**Insight:** Most Netflix content targets **adult and mature audiences**. Only a small portion is kids/family content.

**Business Action:** To **capture family segments**, Netflix could strategically **increase PG and Kids content** while continuing its strong TV-MA pipeline for adult subscribers.

---

## 5. Genre Distribution

- Top genres:
  - Documentaries (299)
  - Stand-Up Comedy (273)
  - Dramas & International Movies (248+)

**Insight:** Netflix's content library is heavily invested in **Documentaries, Stand-Up, and Dramas** — genres that perform well with engaged, repeat viewers.

**Business Action:** Expand **documentary and international drama production** , and explore **under-represented genres** to diversify audience reach.

---

### 6. Top 5 Longest Movies

- *Black Mirror: Bandersnatch* – 312 min
- *Sangam* – 228 min
- *Lagaan* – 224 min
- *Jodhaa Akbar* – 214 min
- *The Irishman* – 209 min

**Insight:** Long-duration movies are largely **Indian and International historical epics**, indicating **a niche but loyal audience base** for extended storytelling.

**Business Action:** Netflix could leverage long-duration content as **event experiences** — marketing them like cinematic releases to attract premium subscribers.

---

### 7. Top 10 Directors

- Leading creators: Raúl Campos & Jan Suter (18), Marcus Raboy (14), Jay Karas (13), Martin Scorsese (9) etc.

**Insight:** Netflix has built **strong partnerships with specific creators**. These directors contribute a significant share of total titles.

**Business Action:** Continue nurturing **exclusive creator partnerships** — it's an effective way to secure unique, consistent content for subscribers.

---

### 8. Content Addition Over Years (date_added trend)

- Sharp increase after 2016, peaking in **2018 (1782)** and **2019 (2349)** additions.
- Before 2014, content addition was negligible.

**Insight:** Netflix shifted from a **US-based streaming service** to a **global content powerhouse** post-2016. This was a **strategic inflection point**.

**Business Action:** Understanding this historical growth can help Netflix **plan its next major expansion wave**, potentially focusing on new content hubs (e.g., Southeast Asia, Africa).

---

## Executive Summary — Business Impact

| Area | Key Insight | Recommended Action |
|---|---|---|
| Content Type | Movies dominate (68%) | Increase TV show investment |
| Geography | US leads, India is strong | Localize content in emerging markets |
| Time Trend | Peak content growth: 2017–2019 | Repeat expansion strategy |
| Ratings | TV-MA dominates | Diversify toward family & kids |
| Genres | Docs & Dramas dominate | Expand under-represented genres |
| Duration | Long movies = niche audience | Market as "event content" |
| Directors | Key creator partnerships | Deepen exclusive deals |
| Yearly Addition Trend | Explosive post-2016 growth | Plan next expansion strategically |

---

## Final Storytelling Question to Stakeholders:

> **“Given that Netflix’s content catalog is heavily movie-focused, US-dominated, and adult-rated, with explosive growth between 2017–2019 — how can we strategically rebalance our content investments across geographies, genres, and audience segments to maximize global reach and engagement in the next expansion phase?”**

---

In [ ]:

In [32]:

```
# CLOSE THE CONNECTION:

cursor.close()
connection.close()

print("Connection to MySQL successfully closed!")
```

```
Connection to MySQL successfully closed!
```

In [ ]:

In [ ]:

In [ ]:

# END OF THE NOTEBOOK

## Please feel free to contact me.

**Here are my contact details:**


- **Name:** ***Om Satyawan Pathak***
- **Email:** ***omsatyawanpathakwebdevelopment@gmail.com***


In [ ]:

In [ ]:

In [ ]:

In [ ]:

**Employees Performance Analysis EDA PROJECT**
**(in MS PowerBI)**

***By Om Satyawan Pathak***
***Github:***
*https://github.com/omsatyawanpathakgit/DataAnalysisProjects/tree/main/codebasics%20HR%20PROJECT/codebasics%20HR%20PROJECT*

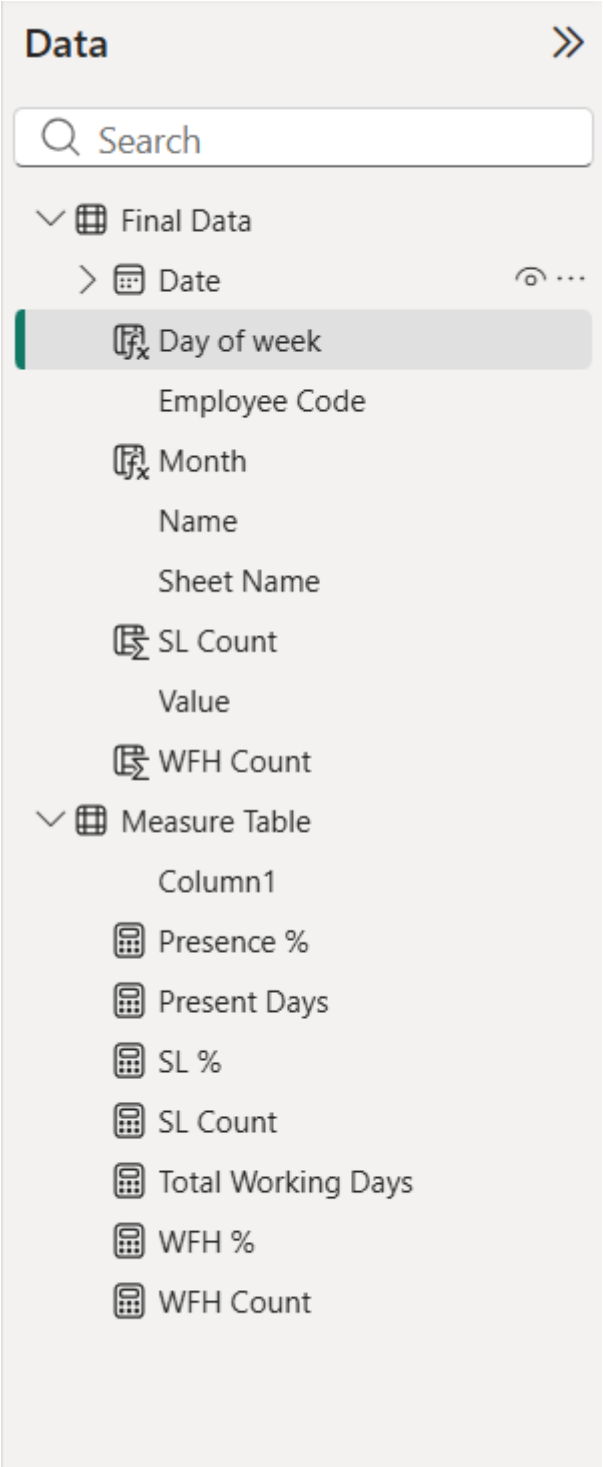
# Atliq Employees Performance Data Analysis Project

**About Project and what we have done:**

This project gives us Atliq's all employees presence for the months April , May and June (in the year 2022).

Insights drawn in this project:

1) How many employees were present (in each month separately and across all months).
2) For every month how many people preferred working from home.
3) How many people were present , opted for Sick-leave and Work from home across all the 3 months (both by Day of week and Employee-code).
4) Daily Attendance of all employees.
5) KPIs for these data:
    a) Total Working Days
    b) Present Days
    c) Presence %
    d) Work from Home (abbreviated as WFH) %
    e) Sick Leave (abbreviated as SL) %
6) Trends observed in these data:
    a) Highest attendance is observed to be in June
    b) Most people preferred to work from Home in month of May.




**Measures used:**

➔ `WFH % = DIVIDE([WFH Count],[Present Days],0)*100`

➔ `Total Working Days =`

```
VAR totaldays = COUNT('Final Data'[Value])
```

```
VAR nonworkdays = CALCULATE( COUNT('Final Data'[Value]),'Final Data'[Value]
in {"WO","HO"})

RETURN totaldays - nonworkdays
```

➔ WFH Count = SUM('Final Data'[WFH Count])

➔ SL Count = SUM('Final Data'[SL Count])

➔ SL % = DIVIDE([SL Count],[Total Working Days],0)*100

➔ Present Days =

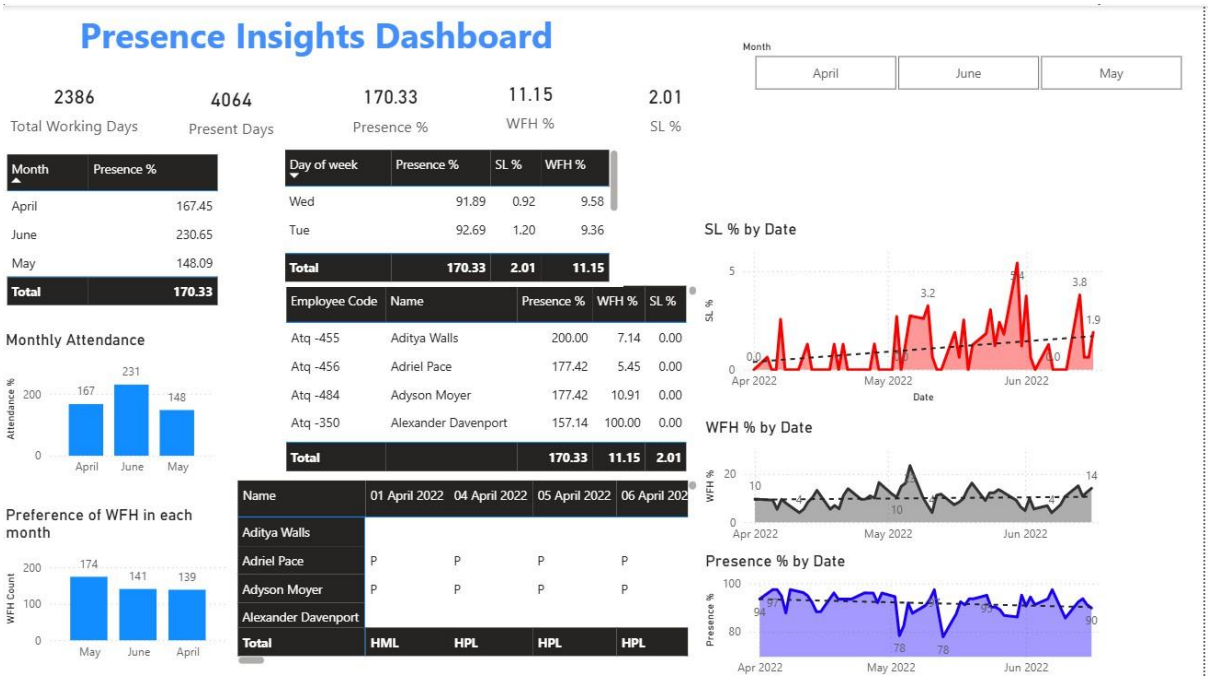
```
VAR Presentdays =
    CALCULATE(
        COUNT('Final Data'[Value]),
        'Final Data'[Value] = "P"
    )

RETURN Presentdays + [WFH Count]
```

➔ Presence % = DIVIDE([Present Days],'Measure Table'[Total Working Days],0) * 100

## New columns created:

- **WFH Count = SWITCH(TRUE(),**
  **'Final Data'[Value] = "WFH",1,**
  **'Final Data'[Value] = "HWFH",0.5,**
  **0)**
- **SL Count = SWITCH(TRUE(),**
  **'Final Data'[Value] = "SL",1,**
  **'Final Data'[Value] = "HSL",0.5,**
  **0)**
- **Month = FORMAT('Final Data'[Date], "MMMM")**
- **Day of week = FORMAT('Final Data'[Date],"ddd")**

## Final Dashboard:




## Conclusion drawn:

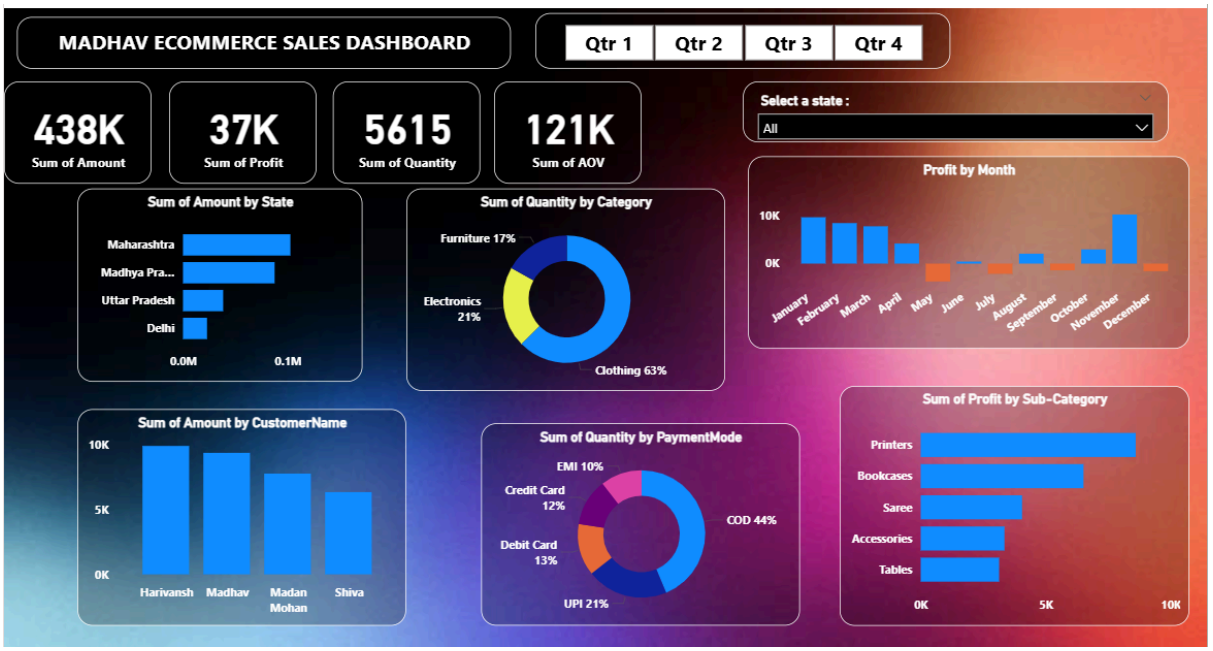
1. Sick-leaves are increasing with time
2. Employees are not preferring to work from office with time (Since Presence % is decreasing)

**MADHAV Ecommerce EDA PROJECT**
**(in MS PowerBI)**

***By Om Satyawan Pathak***
***Github:***
*https://github.com/omsatyawanpathakgit/DataAnalysisProjects/tree/main/MADHAV%20ECOMMERCE%20STORE%20EDA*

**MADHAV ECOMMERCE SALES PROJECT:**

***Dashboard:***




# Madhav E-Commerce Sales Dashboard (Power BI)

---

Overview:

This Power BI dashboard provides information of an e-commerce business's sales performance, helping stakeholders make useful decisions by visualizing trends, profit patterns and customer behaviour.

## Key Metrics at a Glance

- 438K — Total Revenue (`Sum of Amount`)
- 37K — Total Profit
- 5615 — Units Sold
- 121K — Average Order Value (AOV)

## Dashboard Features

- Quarter Selector: View KPIs by Q1–Q4
- State-wise Sales: Maharashtra leads, followed by UP & Delhi
- Customer-wise Spend: Visualizes top-spending customers
- Category Split: Clothing dominates at 63% of quantity sold
- Payment Mode Insights: COD is king (44%), followed by UPI (21%)
- Monthly Profit Trends: Track performance month-over-month
- Sub-category Breakdown: Profits by product segment like "Bottomwear", "Accessories", etc.

## Use Cases

- E-commerce business performance reviews
- Quarterly or monthly sales meetings
- Inventory & marketing strategy planning
- Visual storytelling for stakeholders

## Author

Om Satyawan Pathak
Data Analyst

**Telco Customer Churn Analysis EDA PROJECT**
**(in Python)**

***By Om Satyawan Pathak***
***Github:***
*https://github.com/omsatyawanpathakgit/DataAnalysisProjects/tree/main/TELCO%20CUSTOMER%20CHURN%20EDA/TELCO%20CUSTOMER%20CHURN%20ANALYSIS*

In [12]:

```
import pandas as pd
import seaborn as sns
import numpy as np
import matplotlib.pyplot as m
```

In [13]:

```
df = pd.read_csv("Customer Churn.csv")
```

In [14]:

```
df
```

Out[14]:

| | customerID | gender | SeniorCitizen | Partner | Dependents | tenure | PhoneService | MultipleLines | InternetService | OnlineSe |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 7590-VHVEG | Female | 0 | Yes | No | 1 | No | No phone service | DSL | |
| 1 | 5575-GNVDE | Male | 0 | No | No | 34 | Yes | No | DSL | |
| 2 | 3668-QPYBK | Male | 0 | No | No | 2 | Yes | No | DSL | |
| 3 | 7795-CFOCW | Male | 0 | No | No | 45 | No | No phone service | DSL | |
| 4 | 9237-HQITU | Female | 0 | No | No | 2 | Yes | No | Fiber optic | |
| ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... |
| 7038 | 6840-RESVB | Male | 0 | Yes | Yes | 24 | Yes | Yes | DSL | |
| 7039 | 2234-XADUH | Female | 0 | Yes | Yes | 72 | Yes | Yes | Fiber optic | |
| 7040 | 4801-JZAZL | Female | 0 | Yes | Yes | 11 | No | No phone service | DSL | |
| 7041 | 8361-LTMKD | Male | 1 | Yes | No | 4 | Yes | Yes | Fiber optic | |
| 7042 | 3186-AJIEK | Male | 0 | No | No | 66 | Yes | No | Fiber optic | |

7043 rows × 21 columns

In [15]:

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   object
 1   gender            7043 non-null   object
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   object
 4   Dependents        7043 non-null   object
```

```
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   object
 7   MultipleLines     7043 non-null   object
 8   InternetService   7043 non-null   object
 9   OnlineSecurity    7043 non-null   object
 10  OnlineBackup      7043 non-null   object
 11  DeviceProtection  7043 non-null   object
 12  TechSupport       7043 non-null   object
 13  StreamingTV       7043 non-null   object
 14  StreamingMovies   7043 non-null   object
 15  Contract          7043 non-null   object
 16  PaperlessBilling  7043 non-null   object
 17  PaymentMethod     7043 non-null   object
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   object
 20  Churn             7043 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB
```

# replacing blanks with 0 as tenure is 0 and no total charges are recorded

In [16]:

```
df["TotalCharges"] = df["TotalCharges"].replace(" ","0")
df["TotalCharges"]=df["TotalCharges"].astype('float')
```

In [17]:

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   object
 1   gender            7043 non-null   object
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   object
 4   Dependents        7043 non-null   object
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   object
 7   MultipleLines     7043 non-null   object
 8   InternetService   7043 non-null   object
 9   OnlineSecurity    7043 non-null   object
 10  OnlineBackup      7043 non-null   object
 11  DeviceProtection  7043 non-null   object
 12  TechSupport       7043 non-null   object
 13  StreamingTV       7043 non-null   object
 14  StreamingMovies   7043 non-null   object
 15  Contract          7043 non-null   object
 16  PaperlessBilling  7043 non-null   object
 17  PaymentMethod     7043 non-null   object
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   float64
 20  Churn             7043 non-null   object
dtypes: float64(2), int64(2), object(17)
memory usage: 1.1+ MB
```

In [18]:

```
df.isnull()
```

Out[18]:

| | customerID | gender | SeniorCitizen | Partner | Dependents | tenure | PhoneService | MultipleLines | InternetService | OnlineSe |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | False | False | False | False | False | False | False | False | False | |

| | customerID | gender | SeniorCitizen | Partner | Dependents | tenure | PhoneService | MultipleLines | InternetService | OnlineSe |
|---|---|---|---|---|---|---|---|---|---|---|
| 1 | False | False | False | False | False | False | False | False | False | |
| 2 | False | False | False | False | False | False | False | False | False | |
| 3 | False | False | False | False | False | False | False | False | False | |
| 4 | False | False | False | False | False | False | False | False | False | |
| ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | |
| 7038 | False | False | False | False | False | False | False | False | False | |
| 7039 | False | False | False | False | False | False | False | False | False | |
| 7040 | False | False | False | False | False | False | False | False | False | |
| 7041 | False | False | False | False | False | False | False | False | False | |
| 7042 | False | False | False | False | False | False | False | False | False | |

**7043 rows × 21 columns**

In [19]:

```
df.isnull().sum()
```

Out[19]:

```
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64
```

In [20]:

```
df.isnull().sum().sum()
```

Out[20]:

```
np.int64(0)
```

In [21]:

```
df.describe()
```

Out[21]:

| | SeniorCitizen | tenure | MonthlyCharges | TotalCharges |
|---|---|---|---|---|
| count | 7043.000000 | 7043.000000 | 7043.000000 | 7043.000000 |
| mean | 0.162147 | 32.371149 | 64.761692 | 2279.734304 |
| std | 0.368612 | 24.559481 | 30.090047 | 2266.794470 |
| min | 0.000000 | 0.000000 | 18.250000 | 0.000000 |
| 25% | 0.000000 | 9.000000 | 35.500000 | 398.550000 |

| | | | | |
|---|---|---|---|---|
| 50% | SeniorCitizen | 29.000000 | MonthlyCharges | TotalCharges |
| 75% | 0.000000 | 55.000000 | 89.850000 | 3786.600000 |
| max | 1.000000 | 72.000000 | 118.750000 | 8684.800000 |

In [22]:

```
df.duplicated()
```

Out[22]:

```
0       False
1       False
2       False
3       False
4       False
        ...
7038    False
7039    False
7040    False
7041    False
7042    False
Length: 7043, dtype: bool
```

In [23]:

```
df.duplicated().sum()
```

Out[23]:

```
np.int64(0)
```

# We will ensure that no duplicate customer-id is there:

In [24]:

```
df['customerID'].duplicated().sum()
```

Out[24]:

```
np.int64(0)
```

In [ ]:

```

```

In [ ]:

```

```

# now we will apply a function to make senior-citizen COLUMN value as yes or no instead of any numerical value:

In [25]:

```
def conv(value):
    if value == 1:
        return "yes"
    else:
        return "no"

df["SeniorCitizen"] = df["SeniorCitizen"].apply(conv)
```

In [26]:

```
df.head(5) #check for first 5 values
```

Out[26]:

Out[26]:

| | customerID | gender | SeniorCitizen | Partner | Dependents | tenure | PhoneService | MultipleLines | InternetService | OnlineSecuri |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 7590-VHVEG | Female | no | Yes | No | 1 | No | No phone service | DSL | N |
| 1 | 5575-GNVDE | Male | no | No | No | 34 | Yes | No | DSL | Y |
| 2 | 3668-QPYBK | Male | no | No | No | 2 | Yes | No | DSL | Y |
| 3 | 7795-CFOCW | Male | no | No | No | 45 | No | No phone service | DSL | Y |
| 4 | 9237-HQITU | Female | no | No | No | 2 | Yes | No | Fiber optic | N |

5 rows × 21 columns

In [ ]:

```
```

In [ ]:

```
```

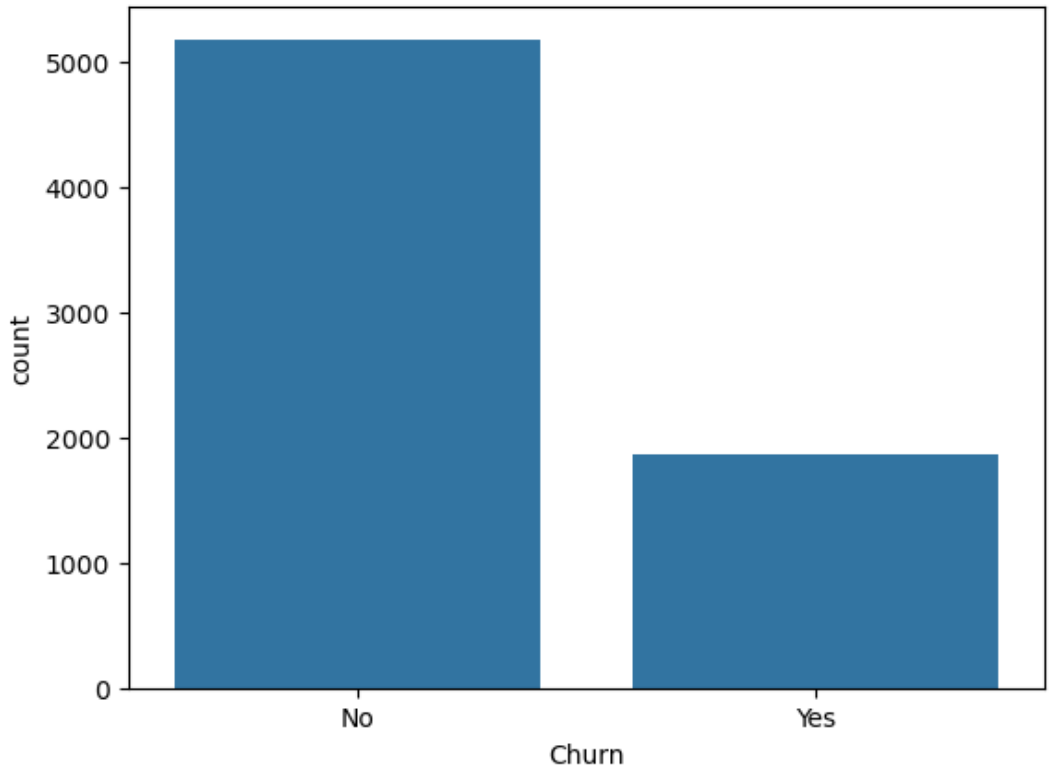
# check how many customers have churned-out or not:

In [27]:

```
sns.countplot(x=df["Churn"],data=df)
```

Out[27]:

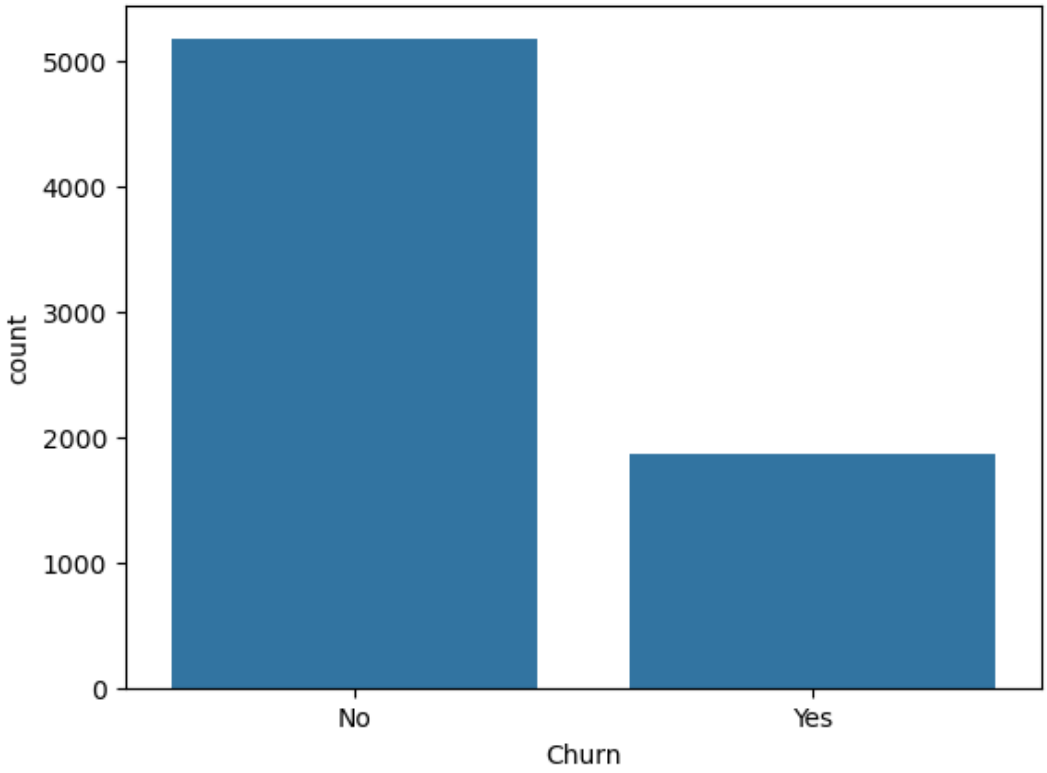
```
<Axes: xlabel='Churn', ylabel='count'>
```




In [28]:

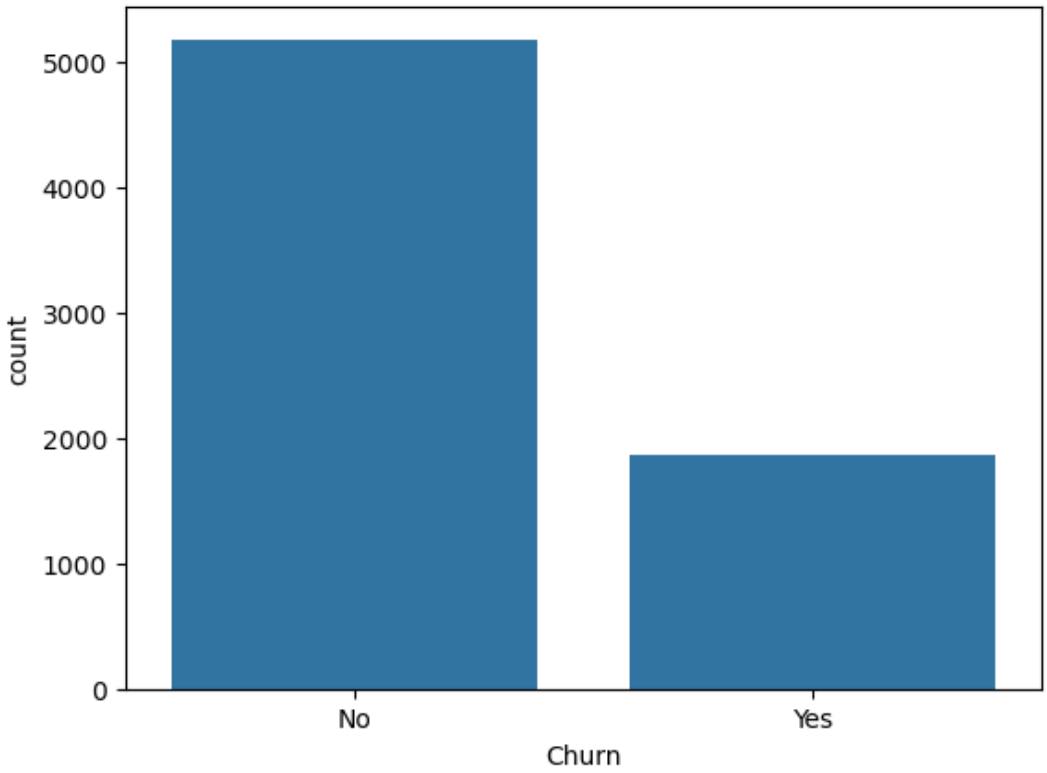
```
sns.countplot(x="Churn",data=df)
```

```
sns.countplot(x="Churn",data=df)
m.show()
```



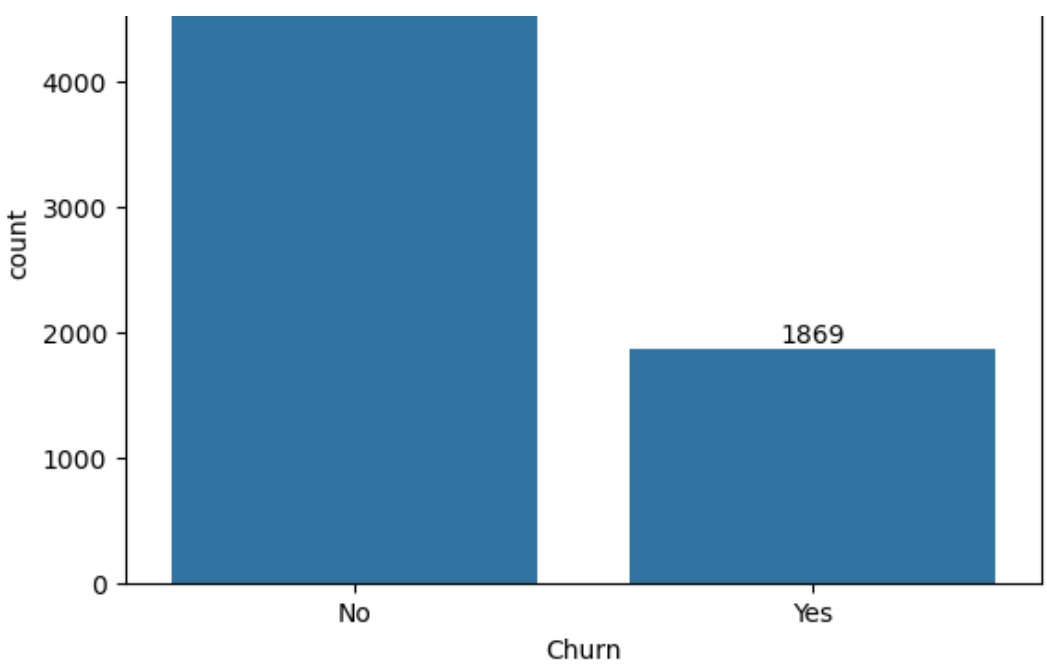

In [29]:

```
ax = sns.countplot(x="Churn",data=df)
m.show()
```



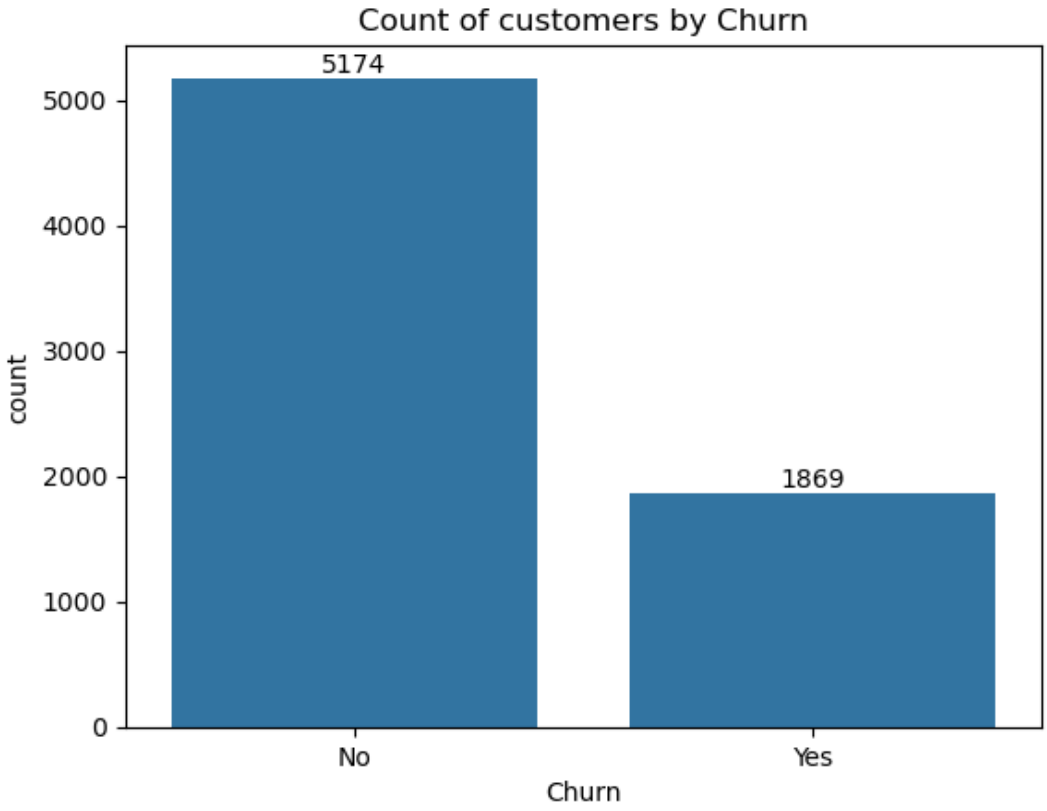

In [30]:

```
ax = sns.countplot(x="Churn",data=df)
ax.bar_label(ax.containers[0]) #check how many customers have churned out or not on top o
f the bars.
m.show()
```




In [31]:

```
ax = sns.countplot(x=df["Churn"],data=df)
ax.bar_label(ax.containers[0]) #check how many customers have churned out or not on top o
f the bars.
m.title("Count of customers by Churn")
m.show()
```




In [32]:

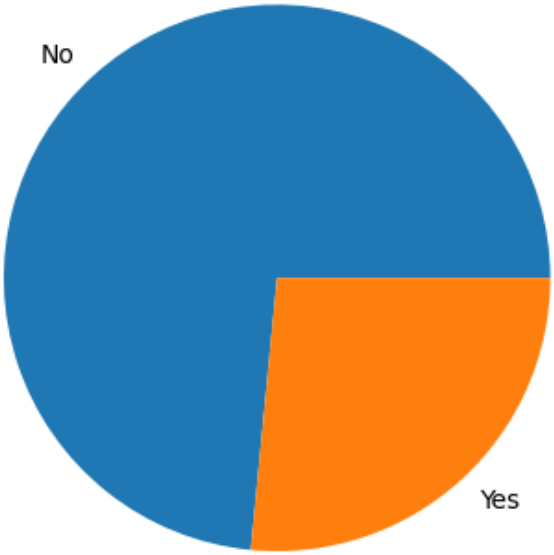
```
gb = df.groupby("Churn").agg({"Churn":"count"})
gb
```

Out[32]:

| | Churn |
|---|---|
| **Churn** | |
| **No** | **5174** |
| **Yes** | **1869** |

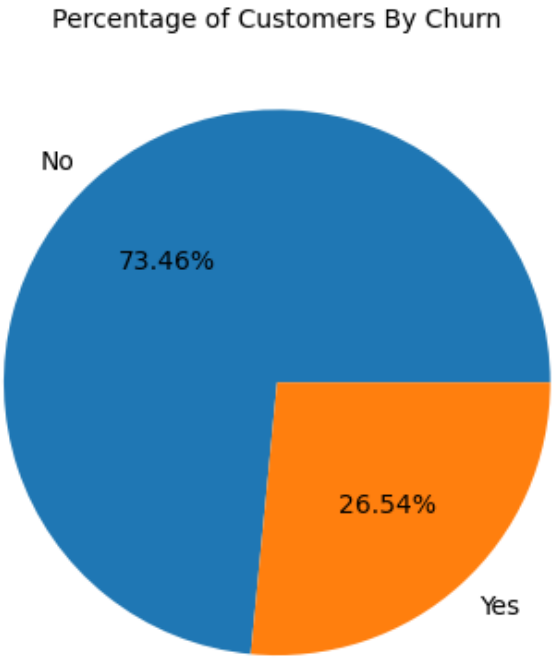
In [33]:

```
gb = df.groupby("Churn").agg({"Churn":"count"})
m.pie(gb["Churn"] , labels = gb.index)
m.show()
```



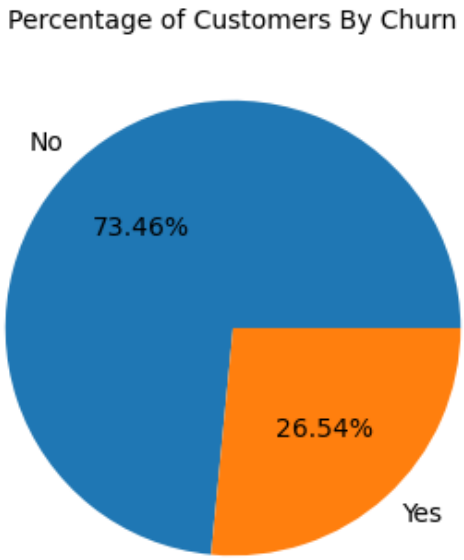

In [34]:

```
gb = df.groupby("Churn").agg({"Churn":"count"})
m.pie(gb["Churn"] , labels = gb.index, autopct = "%1.2f%%")
m.title("Percentage of Customers By Churn",fontsize=10)
m.show()
```




In [35]:

```
m.figure(figsize=(4,4))
gb = df.groupby("Churn").agg({"Churn":"count"})
m.pie(gb["Churn"] , labels = gb.index, autopct = "%1.2f%%")
m.title("Percentage of Customers By Churn",fontsize=10)
m.show()
```

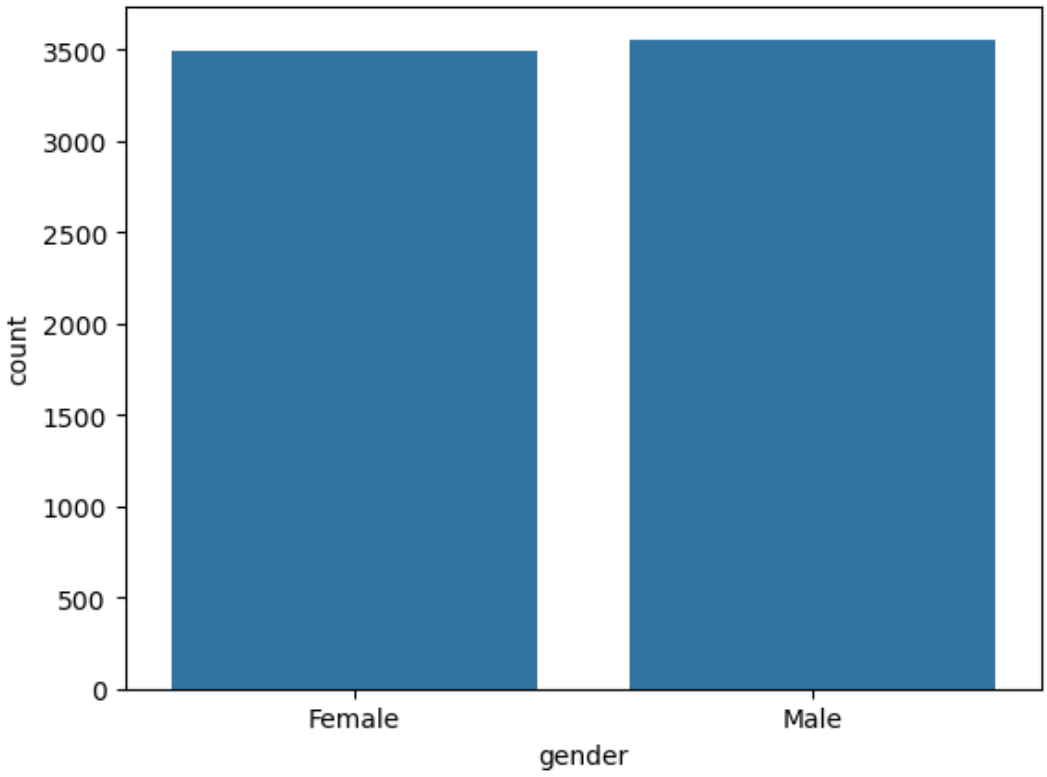



## From the given pie chart we can conclude that 26.54% of our customers have churned out.

## Now lets explore the reason behind it.

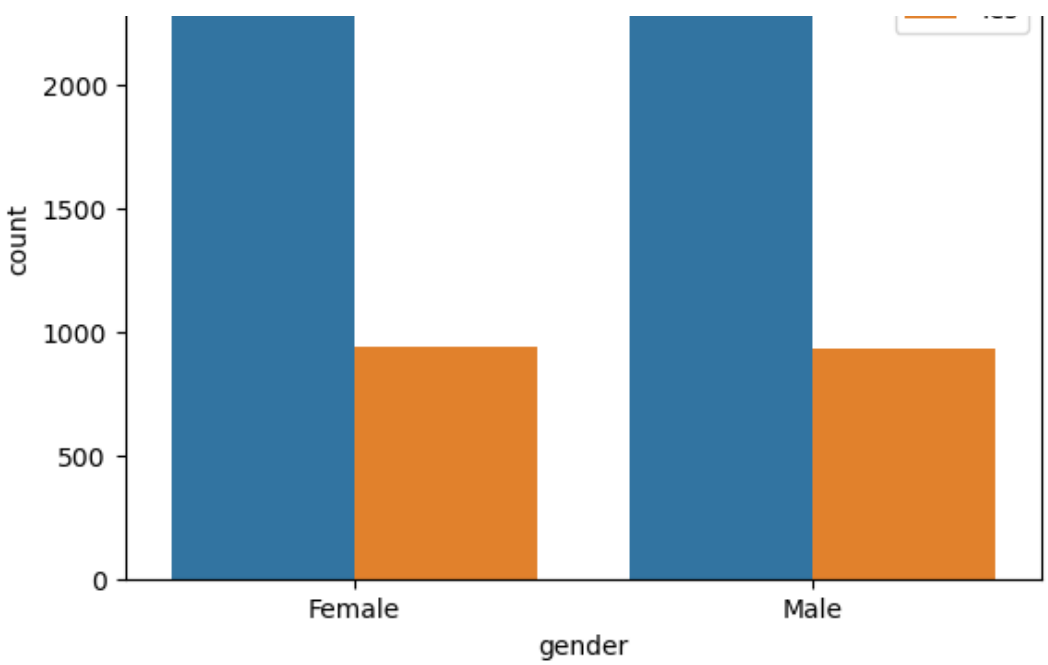
In [36]:

```
sns.countplot(x="gender",data=df)
m.show()
```



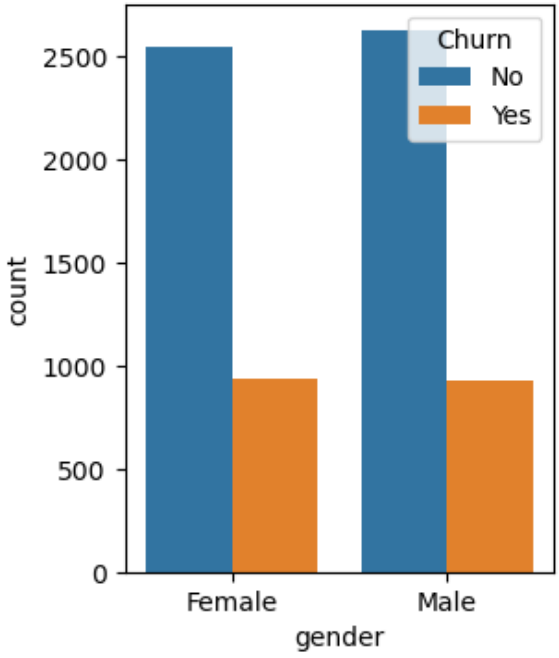

In [37]:

```
sns.countplot(x="gender",data=df,hue="Churn") #check by gender that how many people from each gender have churned out
m.show()
```



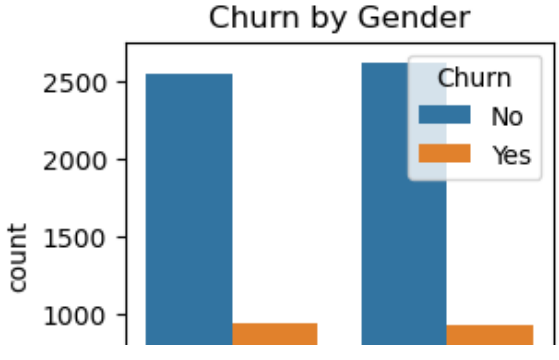

In [38]:

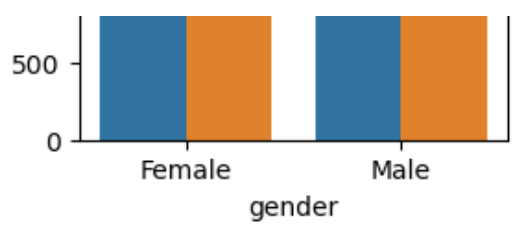
```
m.figure(figsize=(3,4))
sns.countplot(x="gender",data=df,hue="Churn") #check by gender that how many people from each gender have churned out
m.show()
```




In [39]:

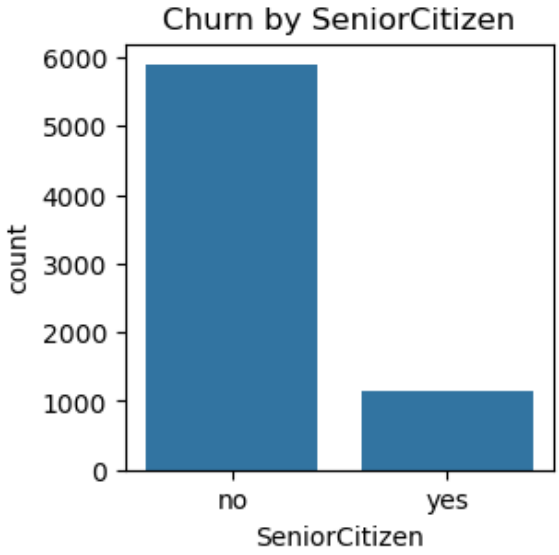
```
m.figure(figsize=(3,3))
sns.countplot(x="gender",data=df,hue="Churn") #check by gender that how many people from each gender have churned out
m.title("Churn by Gender")
m.show()
```





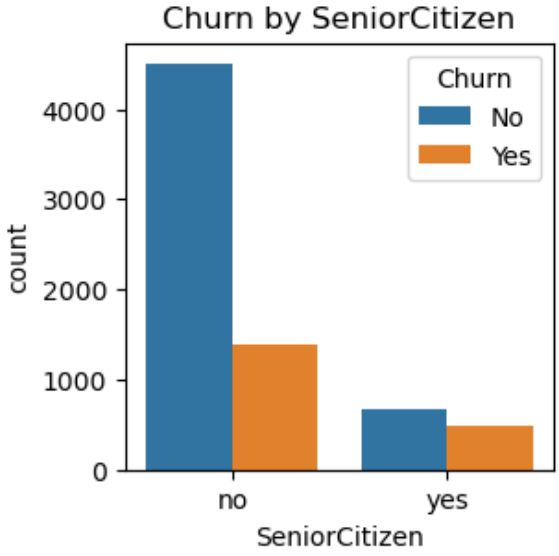

In [40]:

```
m.figure(figsize=(3,3))
sns.countplot(x="SeniorCitizen",data=df) #check by SeniorCitizen that how many people fro
m have churned out and how many have not
m.title("Churn by SeniorCitizen")
m.show()
```




In [41]:

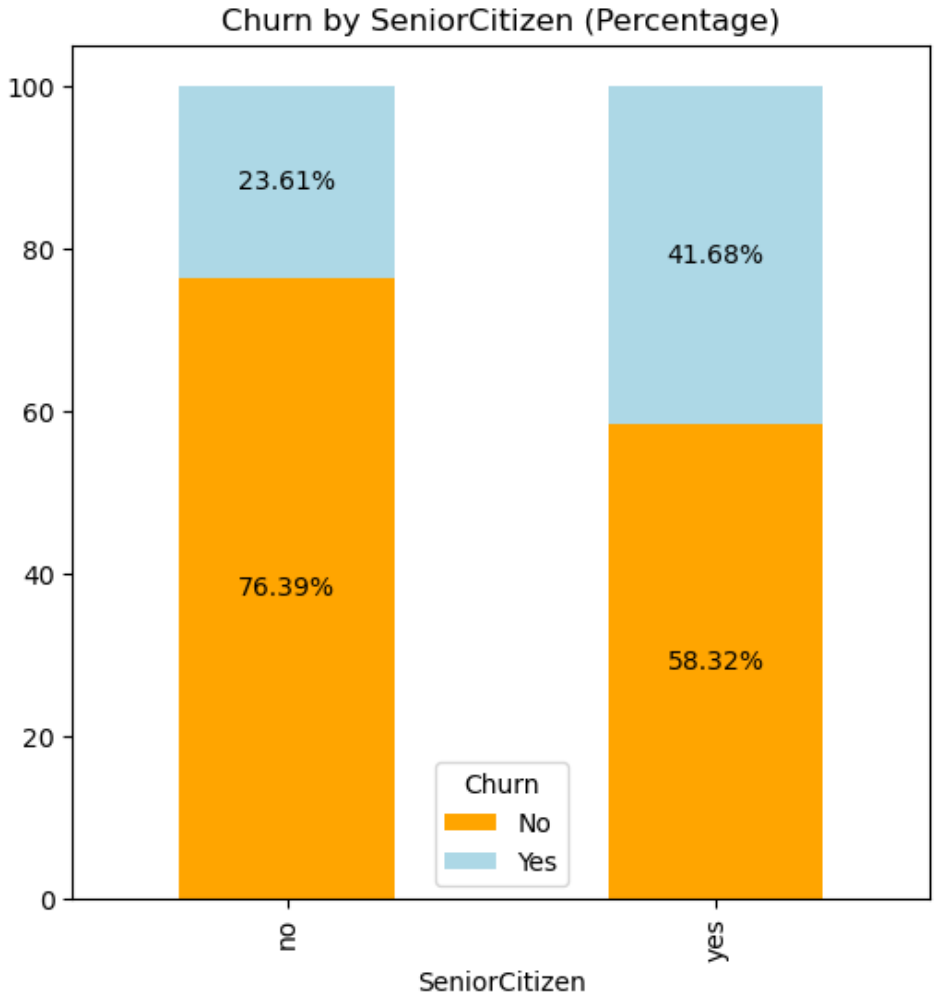
```
#how many senior citizens and non-senior citizens have churned (left the service) and how
many stayed:
m.figure(figsize=(3,3))
sns.countplot(x="SeniorCitizen",data=df,hue="Churn")
m.title("Churn by SeniorCitizen")
m.show()
```




In [42]:

```
#THE ABOVE GRAPH IN STACK BAR-CHART FORM:
count_data = df.groupby(['SeniorCitizen', 'Churn']).size().unstack(fill_value=0)
percentage_data = count_data.div(count_data.sum(axis=1), axis=0) * 100

# Plotting the stacked bar chart
```

```
m.figure(figsize=(6,6))
percentage_data.plot(kind='bar', stacked=True, color=['orange', 'lightblue'], ax=m.gca()
)

# Adding the percentage labels
for p in m.gca().patches:
    height = p.get_height()
    width = p.get_width()
    x, y = p.get_xy()  # Get the x and y position of the rectangle
    percentage = round(height, 2)  # Get the height (which is the percentage here)
    m.gca().text(x + width / 2, y + height / 2, f'{percentage}%', ha='center', va='cente
r', color='black')

m.title("Churn by SeniorCitizen (Percentage)")
m.show()
```

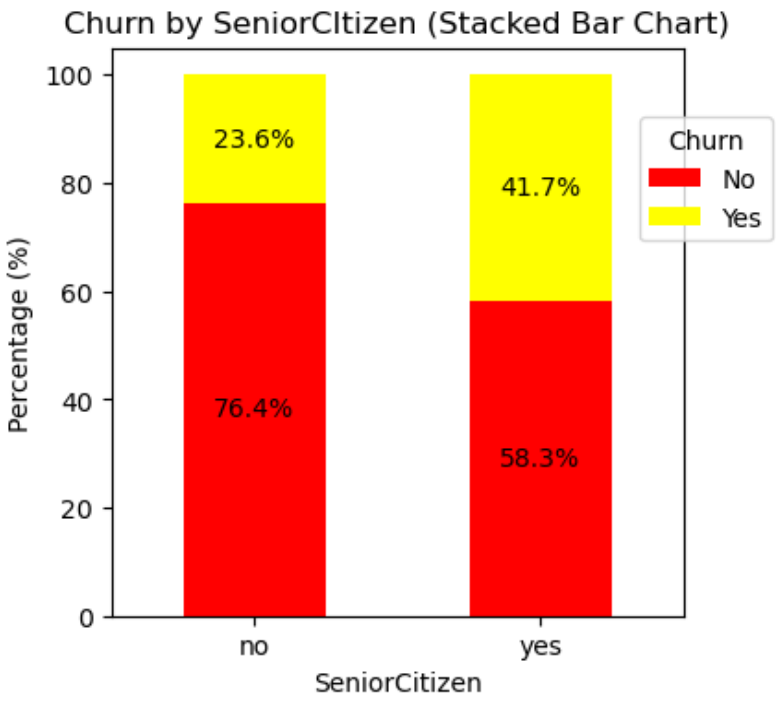



# comparative a gretaed percentage of people in senior-citizen catgeory have churned

In [43]:

```
total_counts = df.groupby('SeniorCitizen')['Churn'].value_counts(normalize=True).unstack
()*100

fig,ax=m.subplots(figsize=(4,4))
total_counts.plot(kind='bar',stacked=True,ax=ax,color=['red','yellow'])

for p in ax.patches:
    width,height=p.get_width(),p.get_height()
    x,y=p.get_xy()
    ax.text(x+width/2,y+height/2,f'{height:.1f}%',ha='center',va='center')


m.title('Churn by SeniorCItizen (Stacked Bar Chart)')
m.xlabel('SeniorCitizen')
```

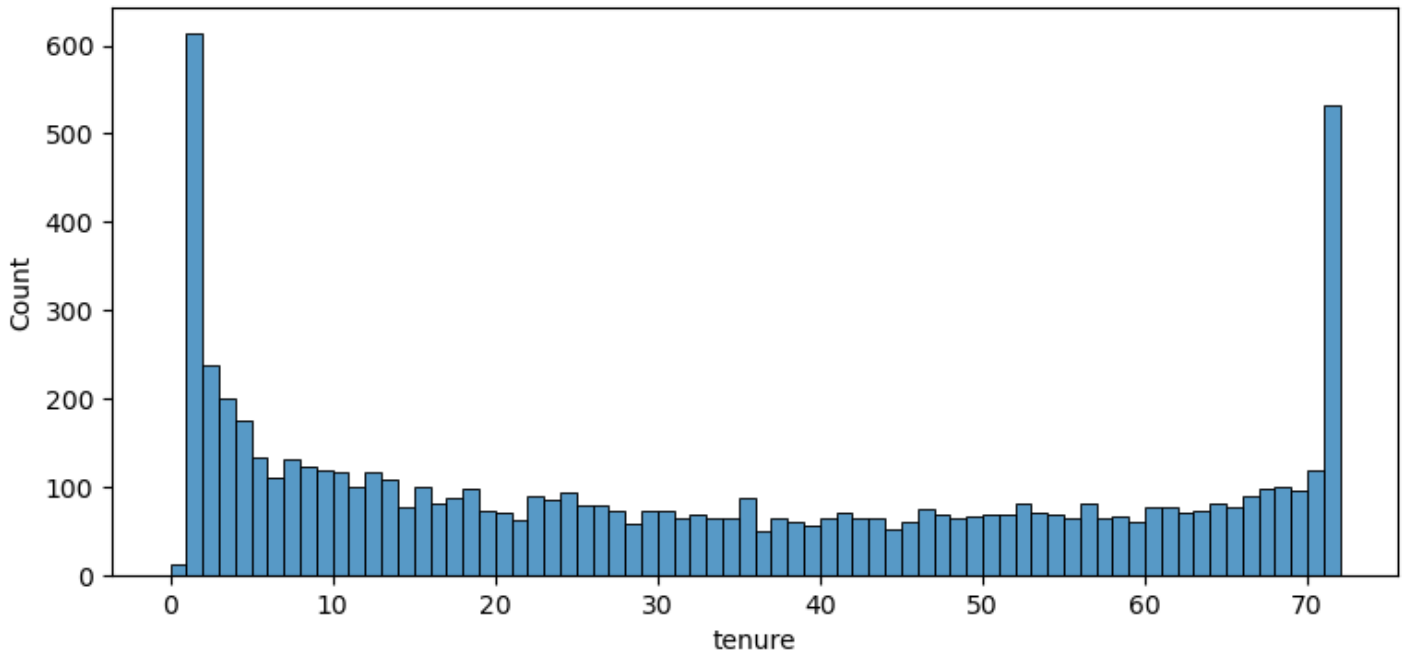
```
m.ylabel('Percentage (%)')
m.xticks(rotation=0)
m.legend(title='Churn',bbox_to_anchor=(0.9,0.9))

m.show()
```




## how many customers have churned out based on tenure:

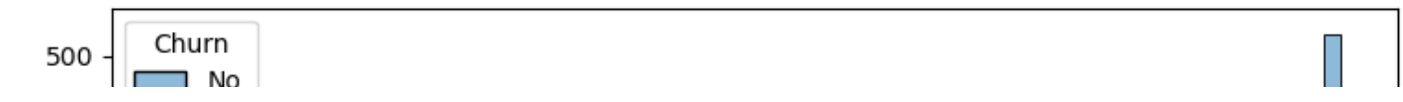
In [44]:

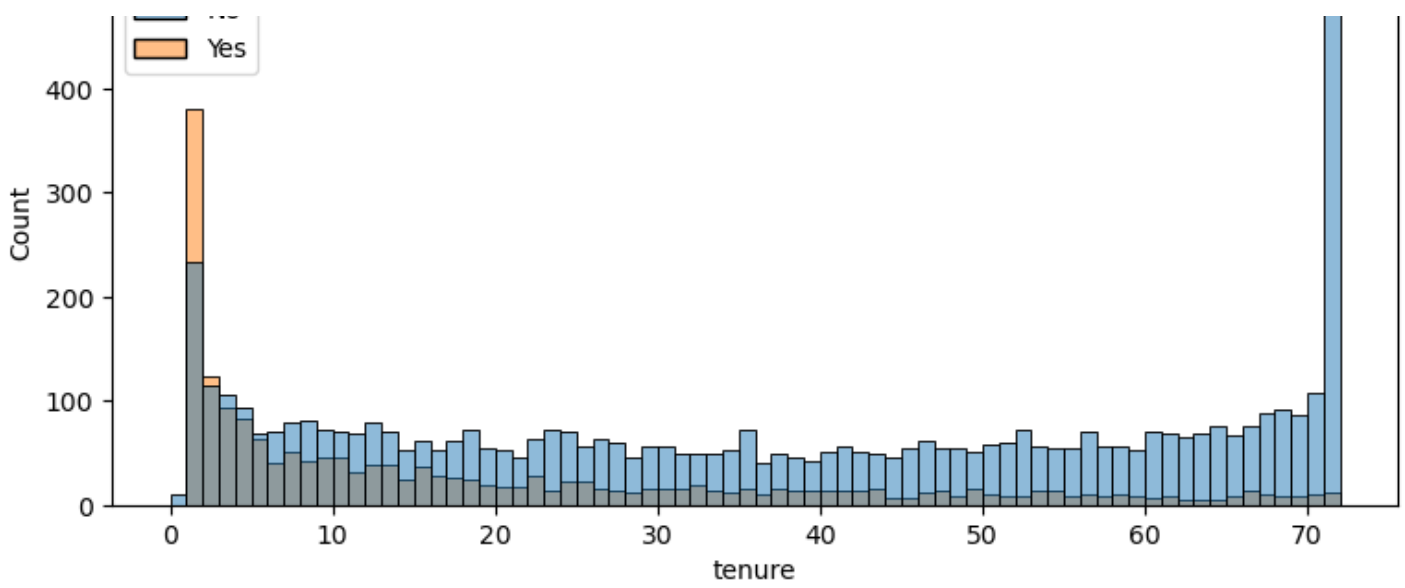
```
m.figure(figsize=(9,4)) #width,height
sns.histplot(x="tenure",data=df,bins=72) #72 bars hence 72 bins
m.show()
```




In [45]:

```
m.figure(figsize=(9,4)) #width,height
sns.histplot(x="tenure",data=df,bins=72, hue="Churn") #72 bars hence 72 bins
m.show()
```



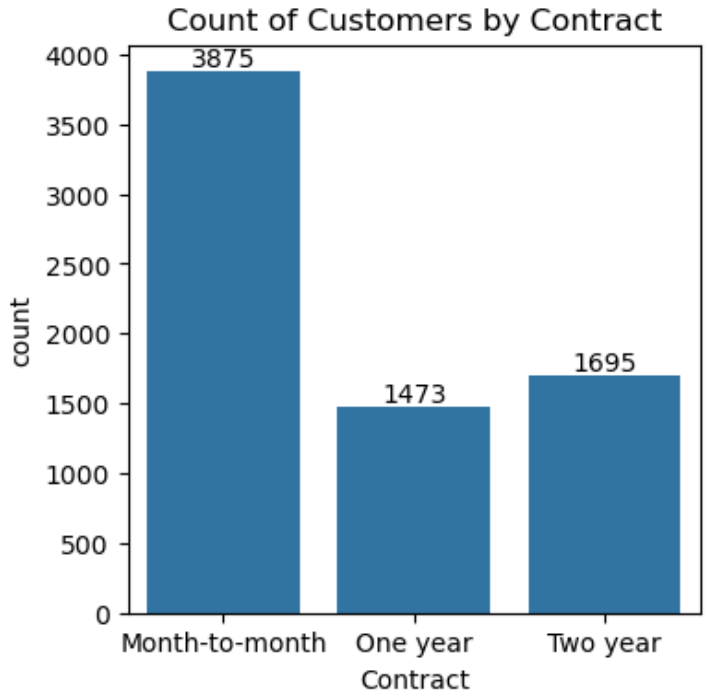



## people who have used our services for a longer time have stayed and people who have used our services

## for 1 or 2 months have churned

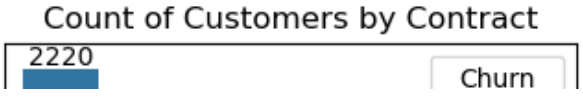
In [46]:

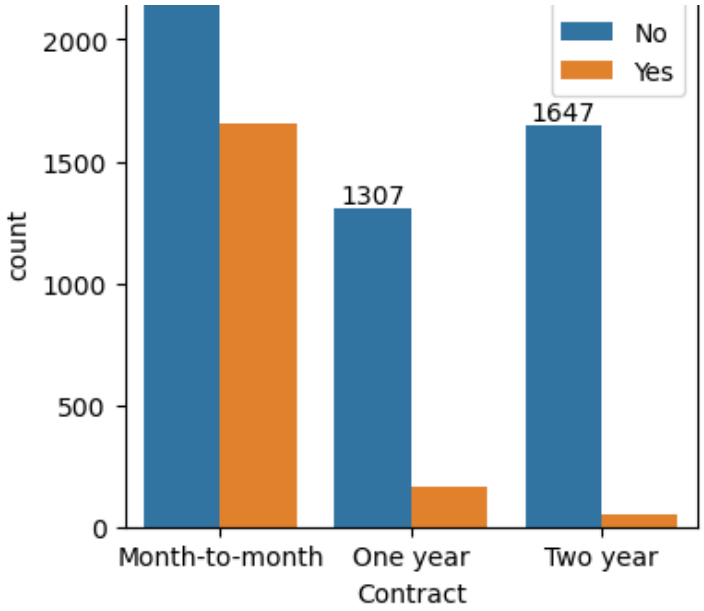
```
m.figure(figsize=(4,4))
ax=sns.countplot(x="Contract",data=df)
ax.bar_label(ax.containers[0])
m.title("Count of Customers by Contract")
m.show()
```




In [47]:

```
m.figure(figsize=(4,4))
ax=sns.countplot(x="Contract",data=df,hue="Churn")
ax.bar_label(ax.containers[0])
m.title("Count of Customers by Contract")
m.show()
```






## people who have month-to-month contract are likely

## to churn out then from those who have 1 or 2 years of contract.

In [48]:

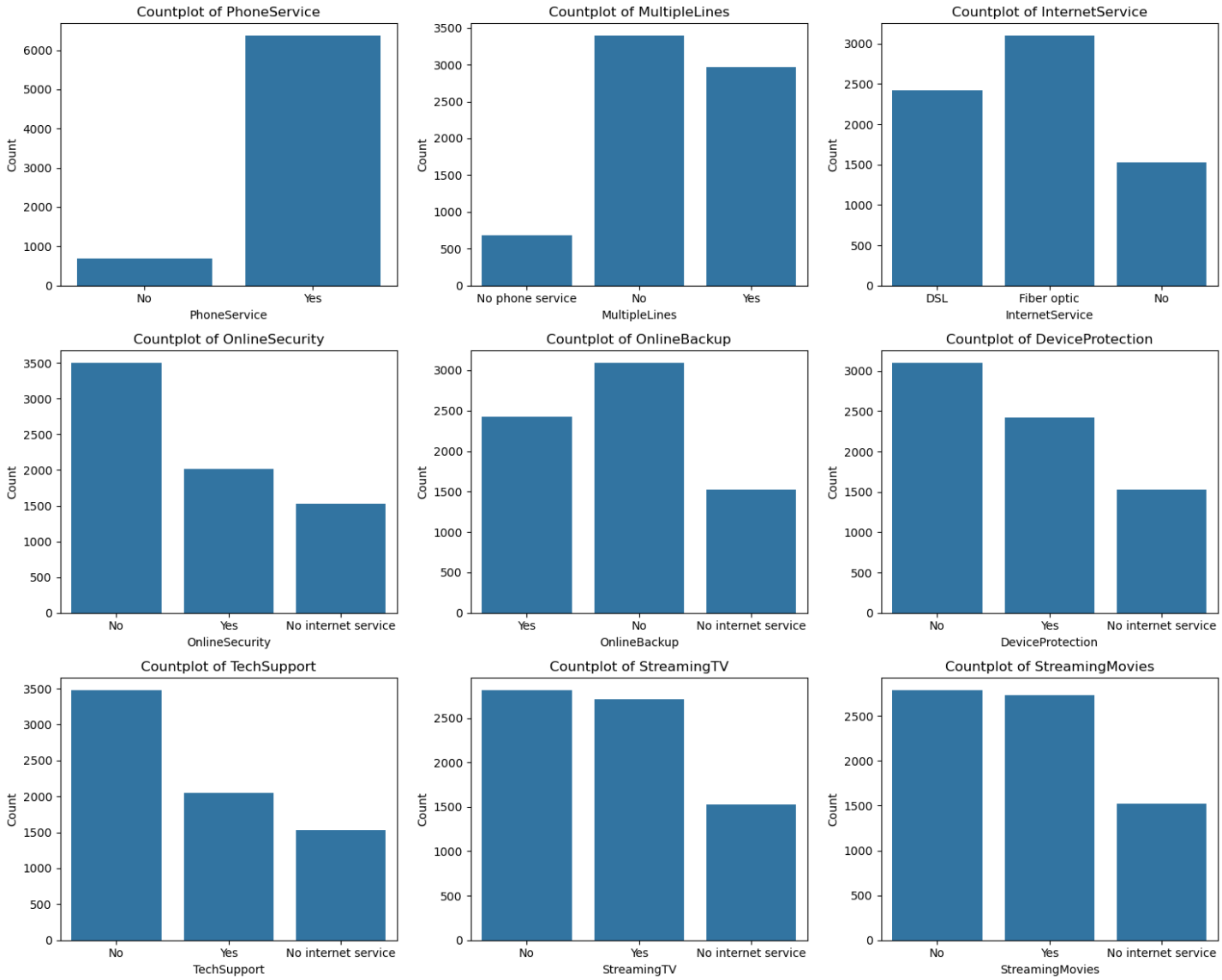
```
df.columns.values
```

Out[48]:

```
array(['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents',
       'tenure', 'PhoneService', 'MultipleLines', 'InternetService',
       'OnlineSecurity', 'OnlineBackup', 'DeviceProtection',
       'TechSupport', 'StreamingTV', 'StreamingMovies', 'Contract',
       'PaperlessBilling', 'PaymentMethod', 'MonthlyCharges',
       'TotalCharges', 'Churn'], dtype=object)
```

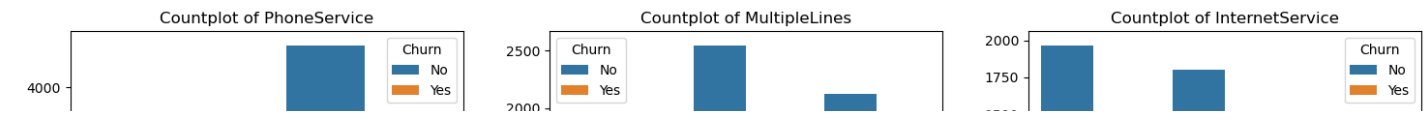
In [49]:

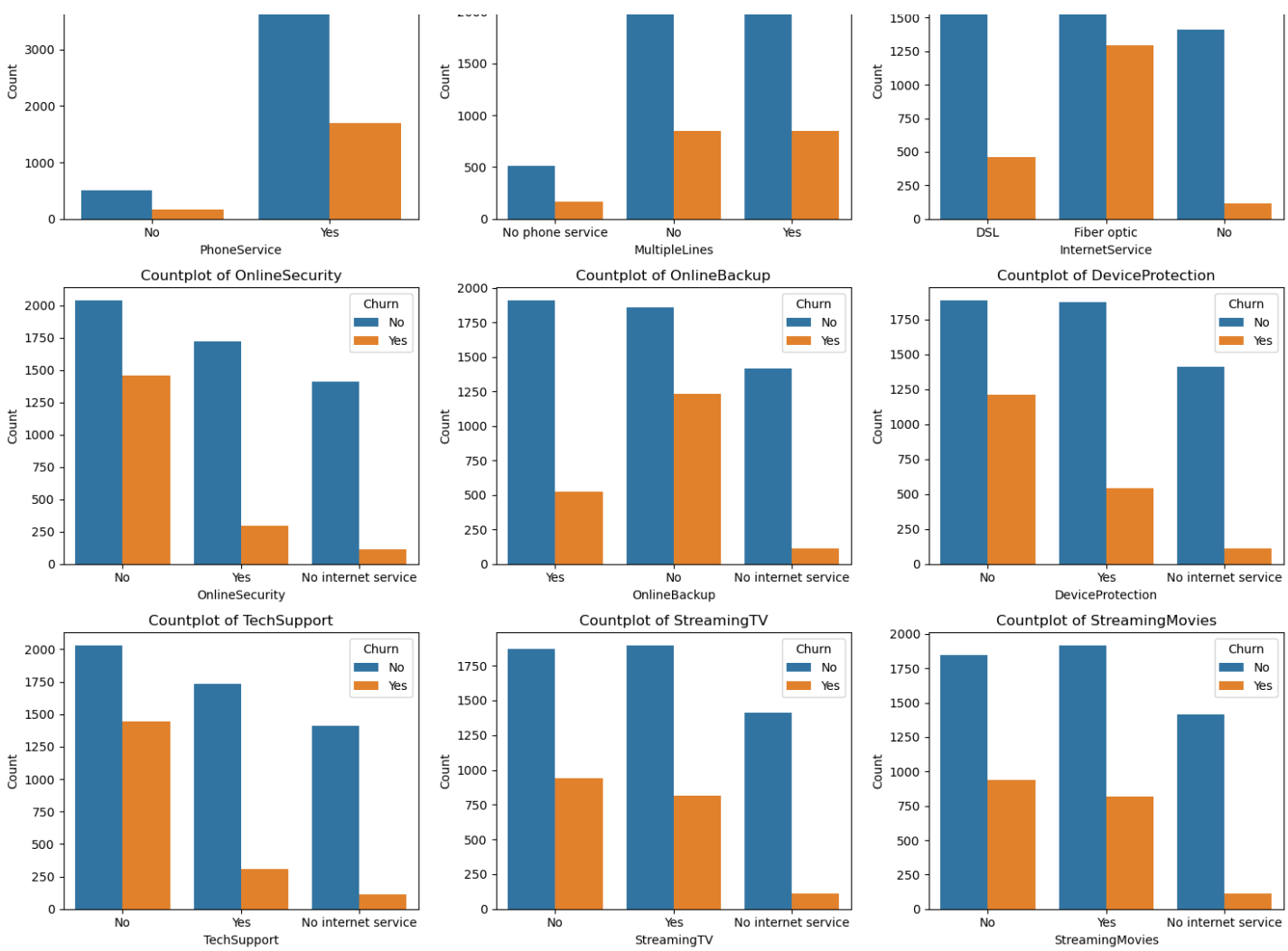
```
# List of columns you want to plot
cols = ['PhoneService', 'MultipleLines', 'InternetService',
        'OnlineSecurity', 'OnlineBackup', 'DeviceProtection',
        'TechSupport', 'StreamingTV', 'StreamingMovies']

# Define number of rows and columns for subplots
n_cols = 3
n_rows =  (len(cols) + n_cols - 1) // n_cols


# Set figure size dynamically
fig, axes = m.subplots(n_rows, n_cols, figsize=(15,n_rows*4))
axes = axes.flatten()  # Flatten to make indexing easier

# Create countplots
for i, col in enumerate(cols):
    sns.countplot(x=col, data=df, ax=axes[i])
    axes[i].set_title(f'Countplot of {col}')
    axes[i].set_xlabel(col)
    axes[i].set_ylabel('Count')

# Remove any extra empty subplots (in case of mismatch)
for j in range(i + 1, len(axes)):
    fig.delaxes(axes[j])

m.tight_layout()
m.show()
```




In [50]:

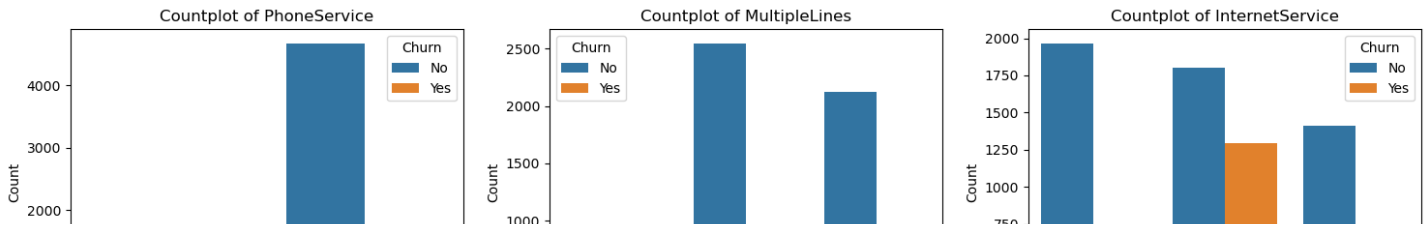
```
# List of columns you want to plot
cols = ['PhoneService', 'MultipleLines', 'InternetService',
        'OnlineSecurity', 'OnlineBackup', 'DeviceProtection',
        'TechSupport', 'StreamingTV', 'StreamingMovies']

# Define number of rows and columns for subplots
n_cols = 3
n_rows =  (len(cols) + n_cols - 1) // n_cols


# Set figure size dynamically
fig, axes = m.subplots(n_rows, n_cols, figsize=(15,n_rows*4))
axes = axes.flatten()  # Flatten to make indexing easier

# Create countplots
for i, col in enumerate(cols):
    sns.countplot(x=col, data=df, ax=axes[i],hue=df["Churn"])
    axes[i].set_title(f'Countplot of {col}')
    axes[i].set_xlabel(col)
    axes[i].set_ylabel('Count')

# Remove any extra empty subplots (in case of mismatch)
for j in range(i + 1, len(axes)):
    fig.delaxes(axes[j])

m.tight_layout()
m.show()
```

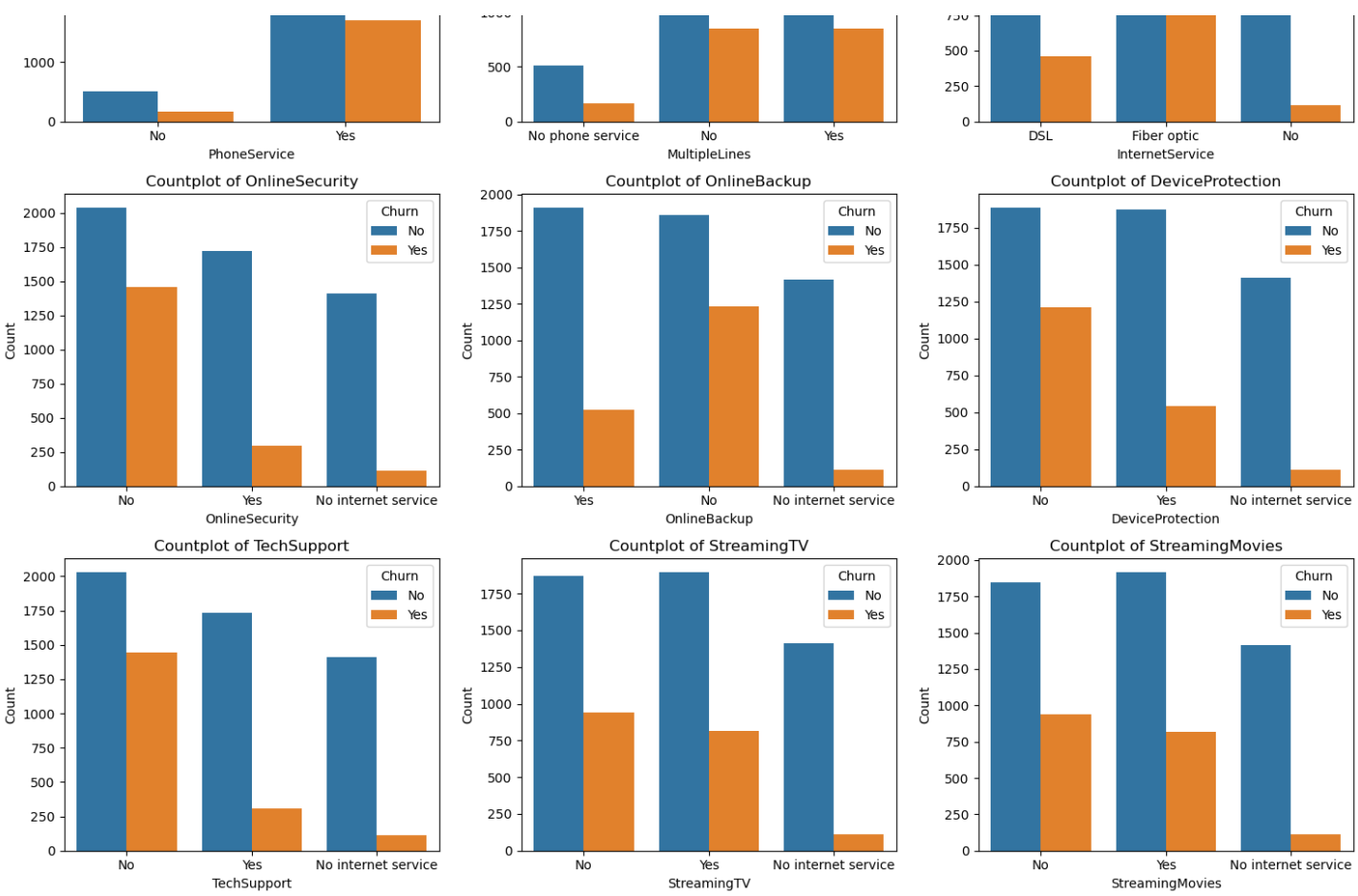





In [51]:

```
# List of columns you want to plot
cols = ['PhoneService', 'MultipleLines', 'InternetService',
        'OnlineSecurity', 'OnlineBackup', 'DeviceProtection',
        'TechSupport', 'StreamingTV', 'StreamingMovies']

# Define number of rows and columns for subplots
n_cols = 3
n_rows =  (len(cols) + n_cols - 1) // n_cols


# Set figure size dynamically
fig, axes = m.subplots(n_rows, n_cols, figsize=(15,n_rows*4))
axes = axes.flatten()  # Flatten to make indexing easier

# Create countplots
for i, col in enumerate(cols):
    sns.countplot(x=col, data=df, ax=axes[i],hue="Churn")
    axes[i].set_title(f'Countplot of {col}')
    axes[i].set_xlabel(col)
    axes[i].set_ylabel('Count')

# Remove any extra empty subplots (in case of mismatch)
for j in range(i + 1, len(axes)):
    fig.delaxes(axes[j])

m.tight_layout()
m.show()
```

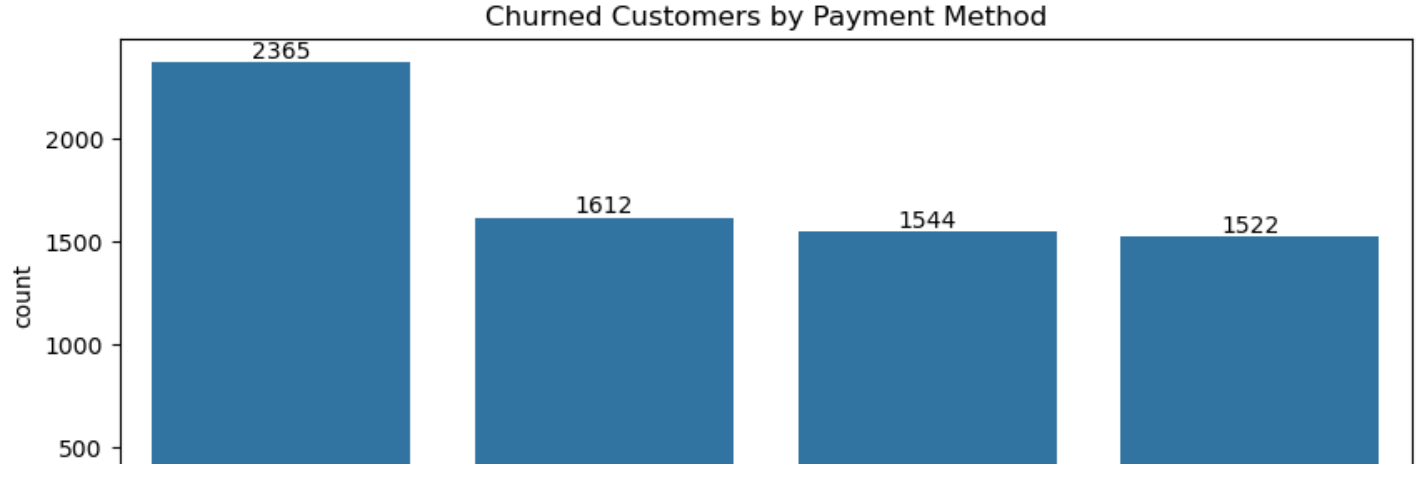


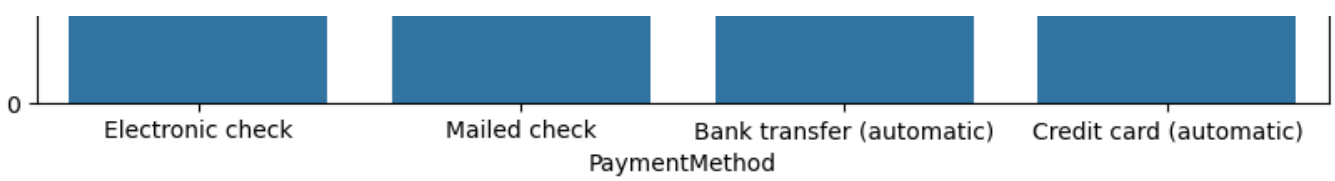



**Customers without internet services or related add-ons like online security, backup, and tech support tend to churn less, indicating that minimal service users are more stable. Those using fiber optic internet have significantly higher churn compared to DSL users. Streaming services such as StreamingTV and StreamingMovies show a more balanced churn rate, suggesting they have less impact on customer retention. Overall, customers subscribed to multiple technical services appear more likely to churn, possibly due to higher costs or dissatisfaction.**

In [52]:

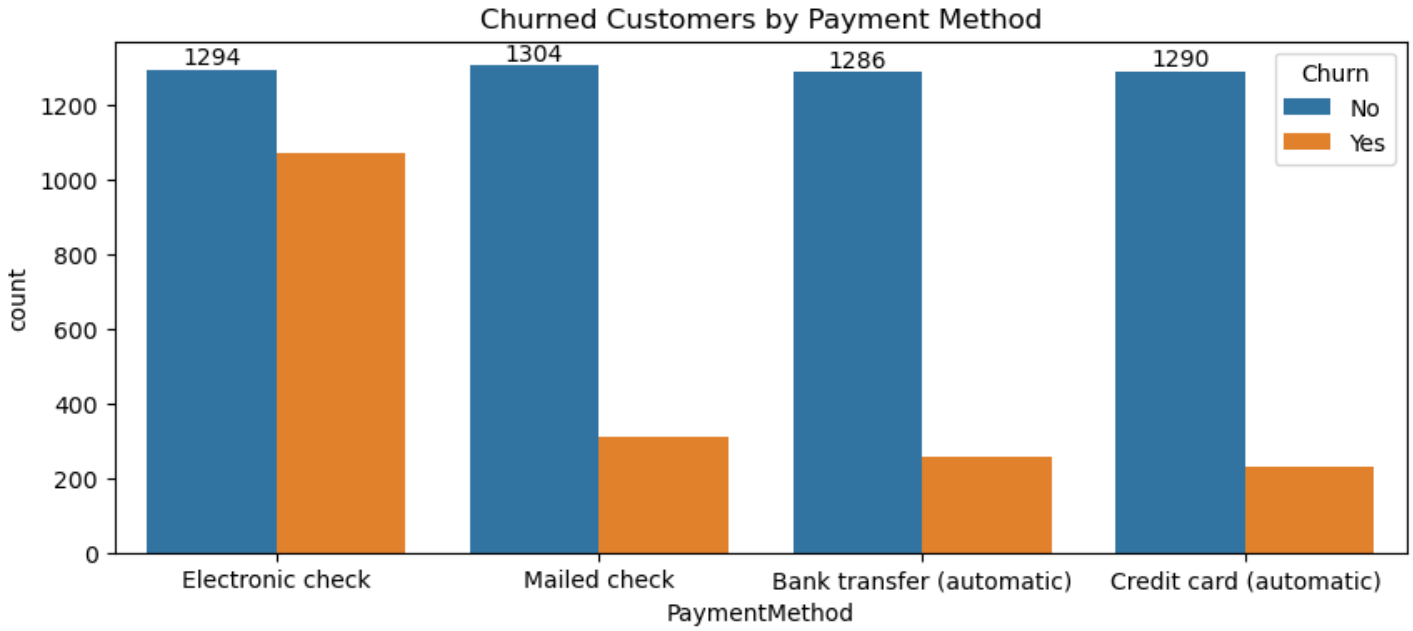
```
m.figure(figsize=(10,4)) #width,height
ax=sns.countplot(x="PaymentMethod",data=df)
ax.bar_label(ax.containers[0])
m.title("Churned Customers by Payment Method")
m.show()
```






In [53]:

```
m.figure(figsize=(10,4)) #width,height
ax=sns.countplot(x="PaymentMethod",data=df,hue="Churn")
ax.bar_label(ax.containers[0])
m.title("Churned Customers by Payment Method")
m.show()
```




## Customer is likely to churn when he is using Electronic-Check as a payment method

In [ ]:

In [ ]:

In [ ]:

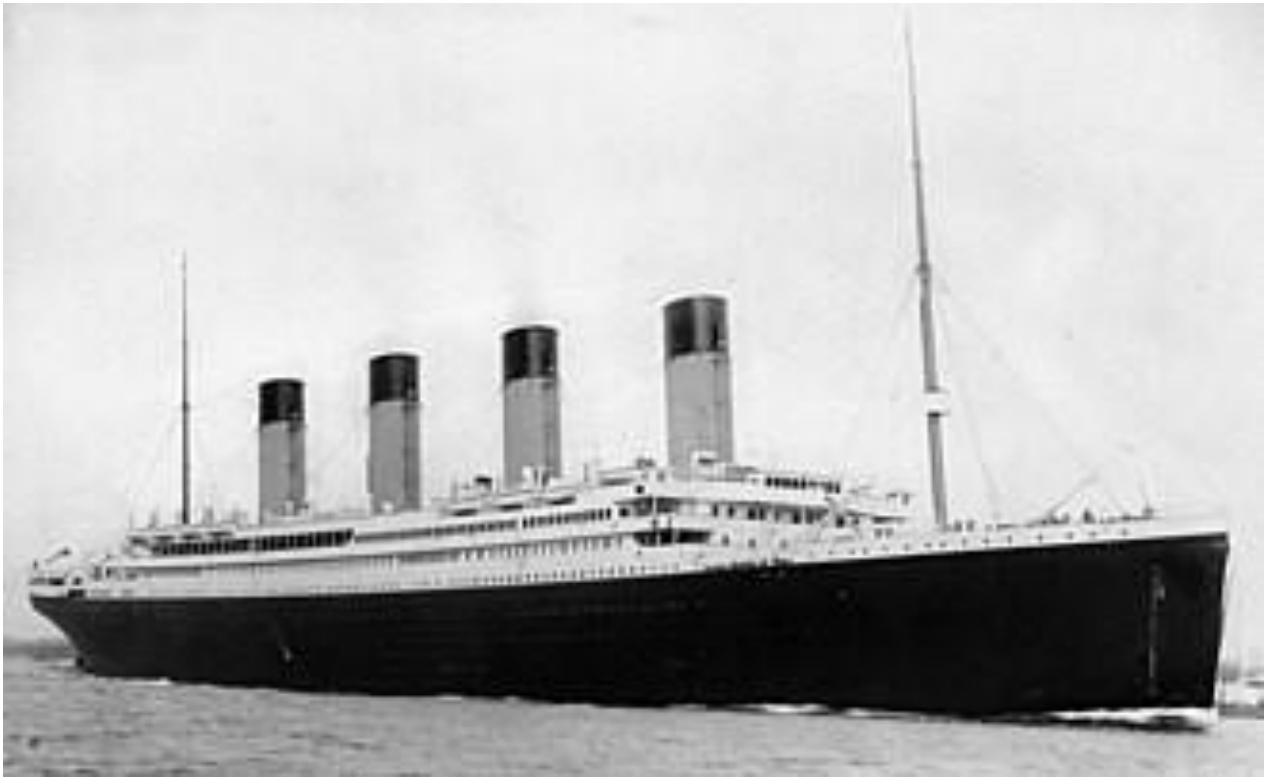
In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:



**Titanic Passengers EDA PROJECT**
**(in Python) (also includes Machine Learning)**

***By Om Satyawan Pathak***
***Github:***
*https://github.com/omsatyawanpathakgit/DataAnalysisProjects/tree/main/Titanic%20ML/Titanic%20ML*

In [327]:

```
from IPython.display import Image, display

display(Image(filename="headerImage.png"))
```

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

# PYTHON ML PROJECT ON TITANIC DATASET:

In [ ]:

## Data Dictionary:

In [328]:

```
'''

Variable Definition          Key


survival Survival          0 = No, 1 = Yes
pclass     Ticket class      1 = 1st, 2 = 2nd, 3 = 3rd
sex         Sex
Age         Age in years
sibsp     no. of siblings / spouses aboard the Titanic
parch     no. of parents / children aboard the Titanic
ticket     Ticket number
fare     Passenger fare (in British pounds (£))
cabin     Cabin number
embarked Port of Embarkation   C = Cherbourg, Q = Queenstown, S = Southampton


'''
```

Out[328]:

```
'\n\nVariable\tDefinition\t         Key\n\n\nsurvival\tSurvival\t         0 = No, 1 = Ye
s\npclass\t    Ticket class\t     1 = 1st, 2 = 2nd, 3 = 3rd\nsex\t         Sex\t\nAge\t
Age in years\t\nsibsp\t    no. of siblings / spouses aboard the Titanic\t\nparch\t    no.
of parents / children aboard the Titanic\t\nticket\t    Ticket number\t\nfare\t    Passen
ger fare (in British pounds (£))\ncabin\t    Cabin number\t\nembarked\tPort of Embarkatio
n\t  C = Cherbourg, Q = Queenstown, S = Southampton\n\n\n'
```

## Import the dataset:

In [329]:

```
import pandas as pd

df = pd.read_csv("train.csv")
```

## Ignore warnings:

In [330]:

```
import warnings
warnings.filterwarnings('ignore')
```

## Do some basic inspections:

### Check how many rows and columns exist:

In [331]:

```
print("No. of rows in the dataset = ",df.shape[0])
print("No. of columns in the dataset = ",df.shape[1])
```

```
No. of rows in the dataset =  891
No. of columns in the dataset =  12
```

In [332]:

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype  
---  ------       --------------  -----  
 0   PassengerId  891 non-null    int64  
 1   Survived     891 non-null    int64  
 2   Pclass       891 non-null    int64  
 3   Name         891 non-null    object 
 4   Sex          891 non-null    object 
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64  
 7   Parch        891 non-null    int64  
 8   Ticket       891 non-null    object 
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object 
 11  Embarked     889 non-null    object 
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

In [333]:

```
df.dtypes
```

Out[333]:

```
PassengerId      int64
Survived         int64
Pclass           int64
Name            object
Sex             object
Age            float64
SibSp            int64
Parch            int64
```

```
Ticket          object
Fare           float64
Cabin           object
Embarked        object
dtype: object
```

In [334]:

```
df.isnull().sum()
```

Out[334]:

```
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age            177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin          687
Embarked         2
dtype: int64
```

### Handle missing-values:

In [335]:

```
df['Age'].fillna(df['Age'].median(), inplace=True)
df['Embarked'].fillna(df['Embarked'].mode()[0], inplace=True)
df.drop('Cabin', axis=1, inplace=True)
```

In [336]:

```
df.isnull().sum()
```

Out[336]:

```
PassengerId    0
Survived       0
Pclass         0
Name           0
Sex            0
Age            0
SibSp          0
Parch          0
Ticket         0
Fare           0
Embarked       0
dtype: int64
```

## View the first few rows of the dataset:

In [337]:

```
df.head(5)
```

Out[337]:

| | PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | Parch | Ticket | Fare | Embarked |
|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 1 | 0 | 3 | Braund, Mr. Owen Harris | male | 22.0 | 1 | 0 | A/5 21171 | 7.2500 | S |
| **1** | 2 | 1 | 1 | Cumings, Mrs. John Bradley (Florence Briggs Th... | female | 38.0 | 1 | 0 | PC 17599 | 71.2833 | C |
| **2** | 3 | 1 | 3 | Heikkinen, Miss. Laina | female | 26.0 | 0 | 0 | STON/O2. 3101282 | 7.9250 | S |

| | PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | Parch | Ticket | Fare | Embarked |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 3 | 4 | 1 | 1 | Futrelle, Mrs. Jacques Heath (Lily May Peel) | female | 35.0 | 1 | 0 | 113803 | 53.1000 | S |
| 4 | 5 | 0 | 3 | Allen, Mr. William Henry | male | 35.0 | 0 | 0 | 373450 | 8.0500 | S |

## Create a new dataframe based on this orignal dataframe upon which we will do Machine-Learning:

In [338]:

```
df_copy = df.copy()
```

In [339]:

```
df_copy['Number of Family Members'] = df_copy['SibSp'] + df_copy['Parch'] + 1
```

## Do some further data-cleaning (going to be useful for ML):

In [340]:

```
df_copy.drop(['PassengerId', 'Name', 'Ticket'], axis=1,inplace=True)
df_copy['Sex'] = df_copy['Sex'].map({'male': 0, 'female': 1})
df_copy["Age"] = df_copy["Age"].astype(int)
df_copy["Fare"] = df_copy["Fare"].astype(int)
df_copy["Pclass"] = df_copy["Pclass"].astype(int)

from sklearn.preprocessing import LabelEncoder
label_encoder = LabelEncoder()
df_copy['Embarkation Point Encoded'] = label_encoder.fit_transform(df_copy['Embarked'])
```

In [341]:

```
df.head(3)
```

Out[341]:

| | PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | Parch | Ticket | Fare | Embarked |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 0 | 3 | Braund, Mr. Owen Harris | male | 22.0 | 1 | 0 | A/5 21171 | 7.2500 | S |
| 1 | 2 | 1 | 1 | Cumings, Mrs. John Bradley (Florence Briggs Th... | female | 38.0 | 1 | 0 | PC 17599 | 71.2833 | C |
| 2 | 3 | 1 | 3 | Heikkinen, Miss. Laina | female | 26.0 | 0 | 0 | STON/O2. 3101282 | 7.9250 | S |

In [342]:

```
df_copy.head(3)
```

Out[342]:

| | Survived | Pclass | Sex | Age | SibSp | Parch | Fare | Embarked | Number of Family Members | Embarkation Point Encoded |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 3 | 0 | 22 | 1 | 0 | 7 | S | 2 | 2 |
| 1 | 1 | 1 | 1 | 38 | 1 | 0 | 71 | C | 2 | 0 |
| 2 | 1 | 3 | 1 | 26 | 0 | 0 | 7 | S | 1 | 2 |

In [ ]:

```

```

# EDA (Exploratory Data Analysis Questions):

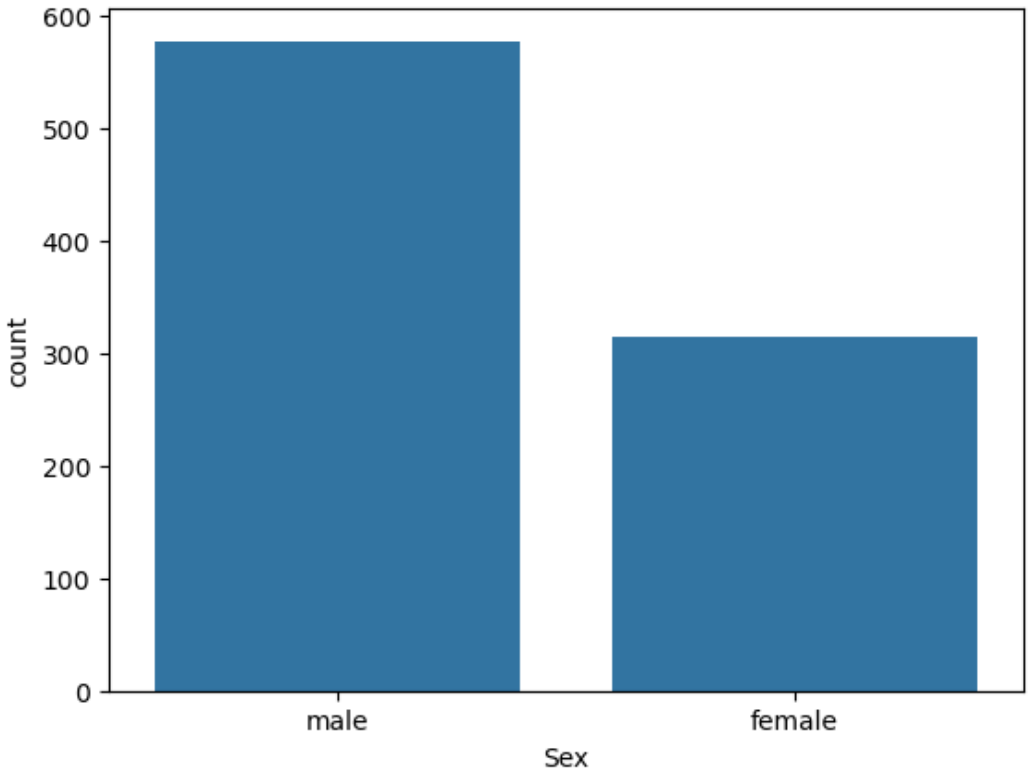
## Q1] HOW MANY males AND females boarded the ship:

In [343]:

```
import seaborn as sns
sns.countplot(x="Sex",data=df)
```

Out[343]:

```
<Axes: xlabel='Sex', ylabel='count'>
```




In [344]:

```
survivors_by_gender = df.groupby("Sex")["Survived"].count().reset_index(name="No. of sur
vivors")
survivors_by_gender
```

Out[344]:

| | Sex | No. of survivors |
|---|---|---|
| 0 | female | 314 |
| 1 | male | 577 |

In [345]:

```
survivors_by_gender = df[df["Survived"]==0].groupby("Sex")["Survived"].count().reset_ind
ex(name="No. of passengers who died")
survivors_by_gender
```

Out[345]:

| | Sex | No. of passengers who died |
|---|---|---|
| 0 | female | 81 |
| 1 | male | 468 |

## CONCLUSIONS AT THIS POINT:

- **Around 80% of females survived the disaster.**
- **55% of males survived the disaster.**

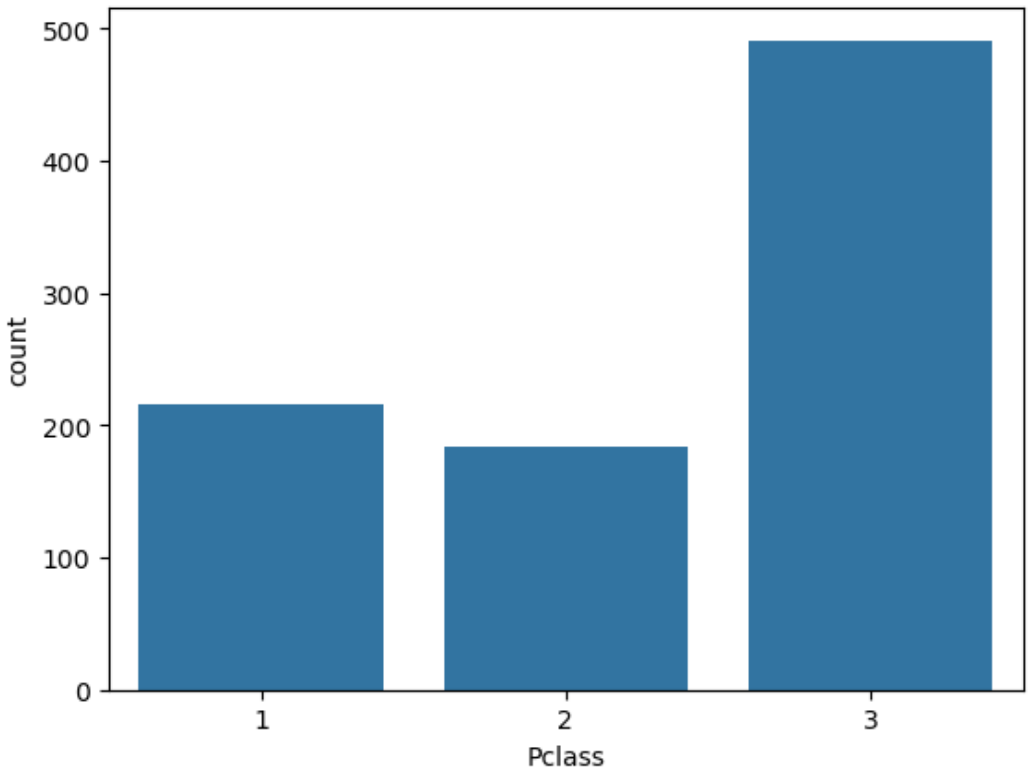
## Q2] HOW MANY PEOPLE WERE THERE IN EACH Passenger-class:

In [346]:

```
import seaborn as sns
sns.countplot(x="Pclass",data=df)
```

Out[346]:

```
<Axes: xlabel='Pclass', ylabel='count'>
```




## Q3] AVERAGE TICKET-PRICES PAID BY EACH Passenger-class:

In [347]:

```
result = df.groupby("Pclass")["Fare"].mean().reset_index()
result
```

Out[347]:

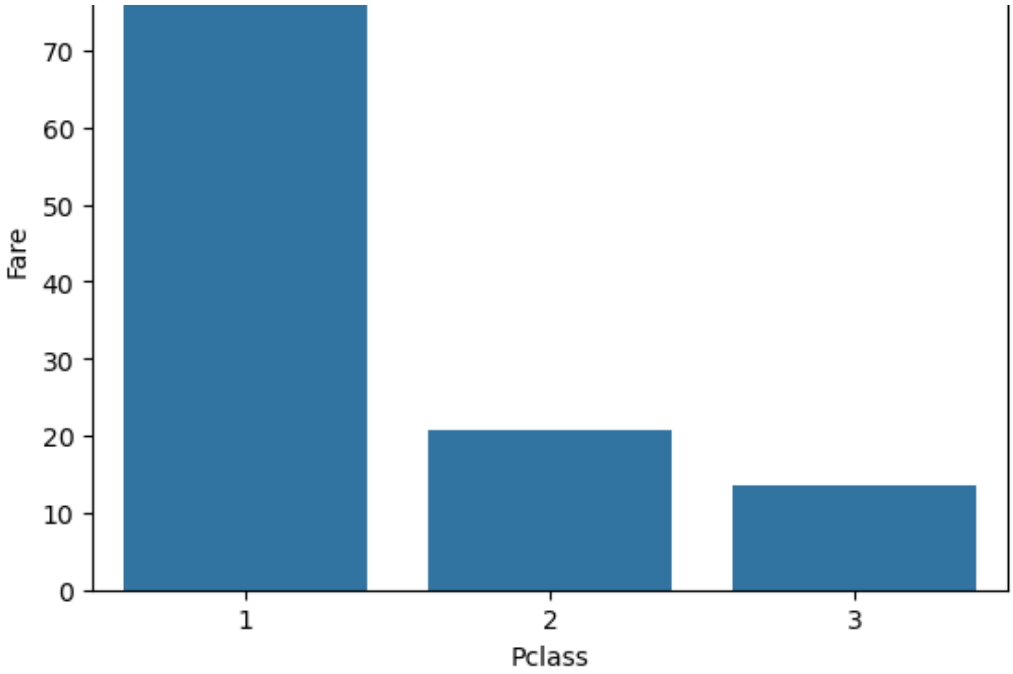
| | Pclass | Fare |
|---|---|---|
| 0 | 1 | 84.154687 |
| 1 | 2 | 20.662183 |
| 2 | 3 | 13.675550 |

In [348]:

```
import seaborn as sns
sns.barplot(x="Pclass",y="Fare",data=result)
```

Out[348]:

```
<Axes: xlabel='Pclass', ylabel='Fare'>
```




## Q4] HOW MANY PEOPLE SURVIVED IN EACH Passenger-class AND HOW MANY DID NOT SURVIVE:

In [349]:

```
result = df["Pclass"].value_counts().reset_index(name="Total no. of Passengers in the cla
ss")
result
```

Out[349]:

| | Pclass | Total no. of Passengers in the class |
|---|---|---|
| 0 | 3 | 491 |
| 1 | 1 | 216 |
| 2 | 2 | 184 |

In [350]:

```
result = result.sort_values(by="Pclass")
result
```

Out[350]:

| | Pclass | Total no. of Passengers in the class |
|---|---|---|
| 1 | 1 | 216 |
| 2 | 2 | 184 |
| 0 | 3 | 491 |

In [351]:

```
result = result.sort_values(by="Pclass").reset_index(drop=True)
result = df[df["Survived"]==1].groupby("Pclass")["Survived"].count().reset_index(name="T
otal no. of survivors")
result
```

Out[351]:

| | Pclass | Total no. of survivors |
|---|---|---|
| 0 | 1 | 136 |

| | Pclass | Total no. of survivors |
|---|---|---|
| 1 | 2 | 87 |
| 2 | 3 | 119 |

In [ ]:

In [352]:

```
result = df[df["Survived"]==0].groupby("Pclass")["Survived"].count().reset_index(name="Did not survive")
result
```

Out[352]:

| | Pclass | Did not survive |
|---|---|---|
| 0 | 1 | 80 |
| 1 | 2 | 97 |
| 2 | 3 | 372 |

### CONCLUSION TILL HERE:

- **Most people on the ship were from 3rd passenger-class.**
- **People belonging to the 1st passenger-class paid the most fare.**
- **More than 50% of people in 1st passenger-class survived the disaster.**
- **Only 47% passengers in 2nd passenger-class survived the disaster.**
- **Only 24% passengers in 3rd passenger-class survived the disaster.**

In [ ]:

In [ ]:

In [ ]:

# ML QUESTIONS:

## Q1] PREDICTING FARE-AMOUNT BASED ON:

- **Passenger Class**
- **Family Members**
- **Embarkation Point**

In [353]:

```
from sklearn import linear_model

reg = linear_model.LinearRegression()

independent_variables = ["Pclass","Number of Family Members","Embarkation Point Encoded"]
reg.fit(df_copy[independent_variables],df["Fare"])
reg.predict([[3,2,2]])
```

Out[353]:

```
array([6.21894201])
```

In [ ]:

```
```

## Q2] PREDICTING PASSENGER-AGE BASED ON:

- **Passenger Class**
- **Number of Family Members**
- **Gender**

In [354]:

```
from sklearn import linear_model

reg = linear_model.LinearRegression()

independent_variables = ["Pclass","Number of Family Members","Sex"]
reg.fit(df_copy[independent_variables],df["Age"])
reg.predict([[3,2,0]])
```

Out[354]:

```
array([26.37839535])
```

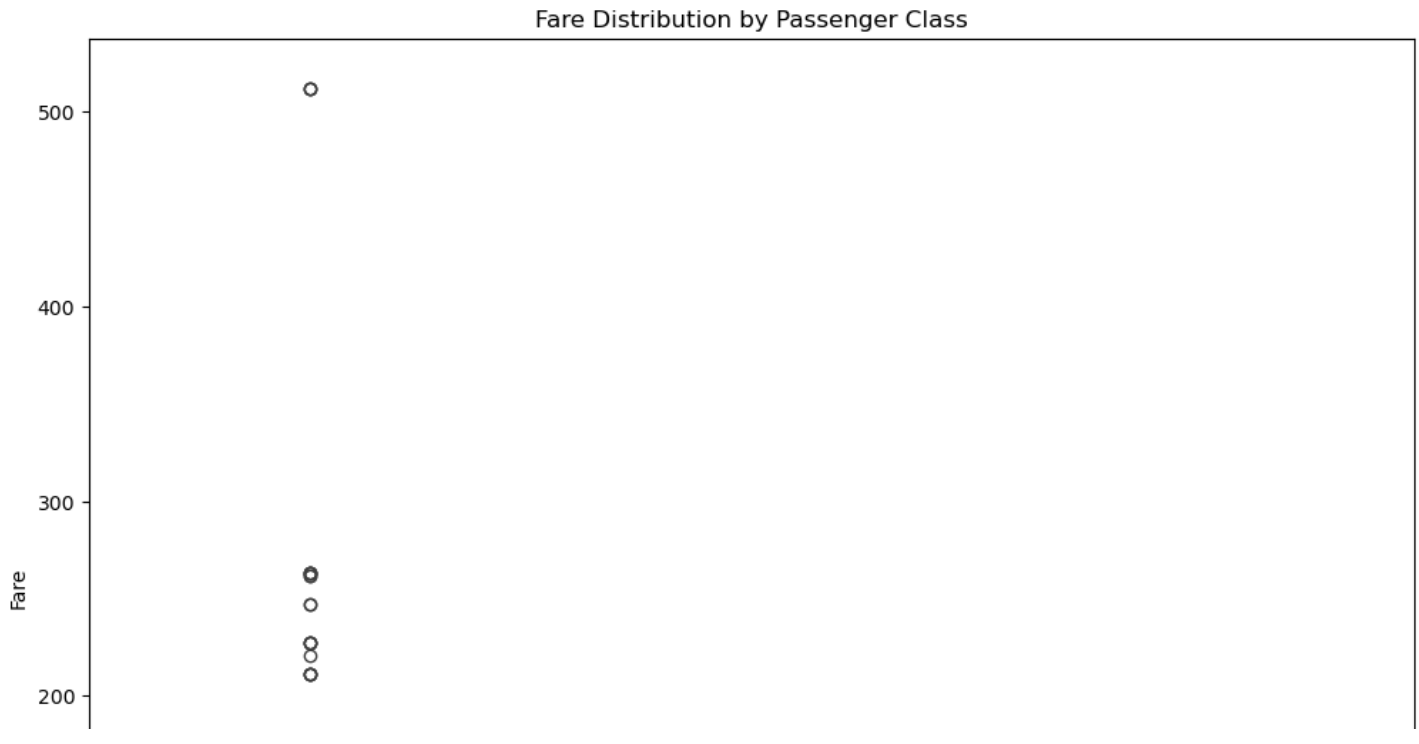
In [ ]:

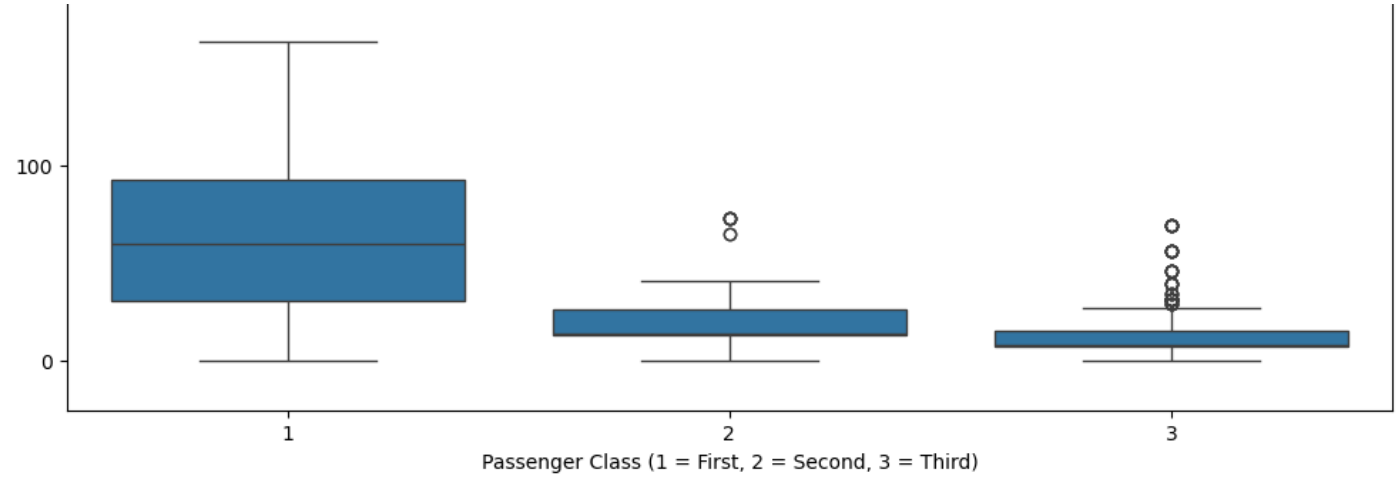
```
```

## Q3] RELATIONSHIP BETWEEN FARE AND PASSENGER-CLASS:

In [355]:

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(12,10))
sns.boxplot(x='Pclass', y='Fare', data=df_copy)
plt.title("Fare Distribution by Passenger Class")
plt.xlabel("Passenger Class (1 = First, 2 = Second, 3 = Third)")
plt.ylabel("Fare")
plt.show()
```






**CONCLUSION AT THIS POINT:**

- **Passengers belonging to class 1 paid highest fair and in the range of 50 to 100 pounds (approx.)**
- **Passengers belonging to class 2 paid moderate fair and in the range of 20 to 40 pounds (approx.)**
- **Passengers belonging to class 3 paid least fair and in the range of 20 to 30 pounds (approx.)**

In [356]:

```
df_copy.head(10)
```

Out[356]:

| | Survived | Pclass | Sex | Age | SibSp | Parch | Fare | Embarked | Number of Family Members | Embarkation Point Encoded |
|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 0 | 3 | 0 | 22 | 1 | 0 | 7 | S | 2 | 2 |
| **1** | 1 | 1 | 1 | 38 | 1 | 0 | 71 | C | 2 | 0 |
| **2** | 1 | 3 | 1 | 26 | 0 | 0 | 7 | S | 1 | 2 |
| **3** | 1 | 1 | 1 | 35 | 1 | 0 | 53 | S | 2 | 2 |
| **4** | 0 | 3 | 0 | 35 | 0 | 0 | 8 | S | 1 | 2 |
| **5** | 0 | 3 | 0 | 28 | 0 | 0 | 8 | Q | 1 | 1 |
| **6** | 0 | 1 | 0 | 54 | 0 | 0 | 51 | S | 1 | 2 |
| **7** | 0 | 3 | 0 | 2 | 3 | 1 | 21 | S | 5 | 2 |
| **8** | 1 | 3 | 1 | 27 | 0 | 2 | 11 | S | 3 | 2 |
| **9** | 1 | 2 | 1 | 14 | 1 | 0 | 30 | C | 2 | 0 |

In [ ]:

```

```

# Q4] Effect of Family Size on Fare

In [357]:

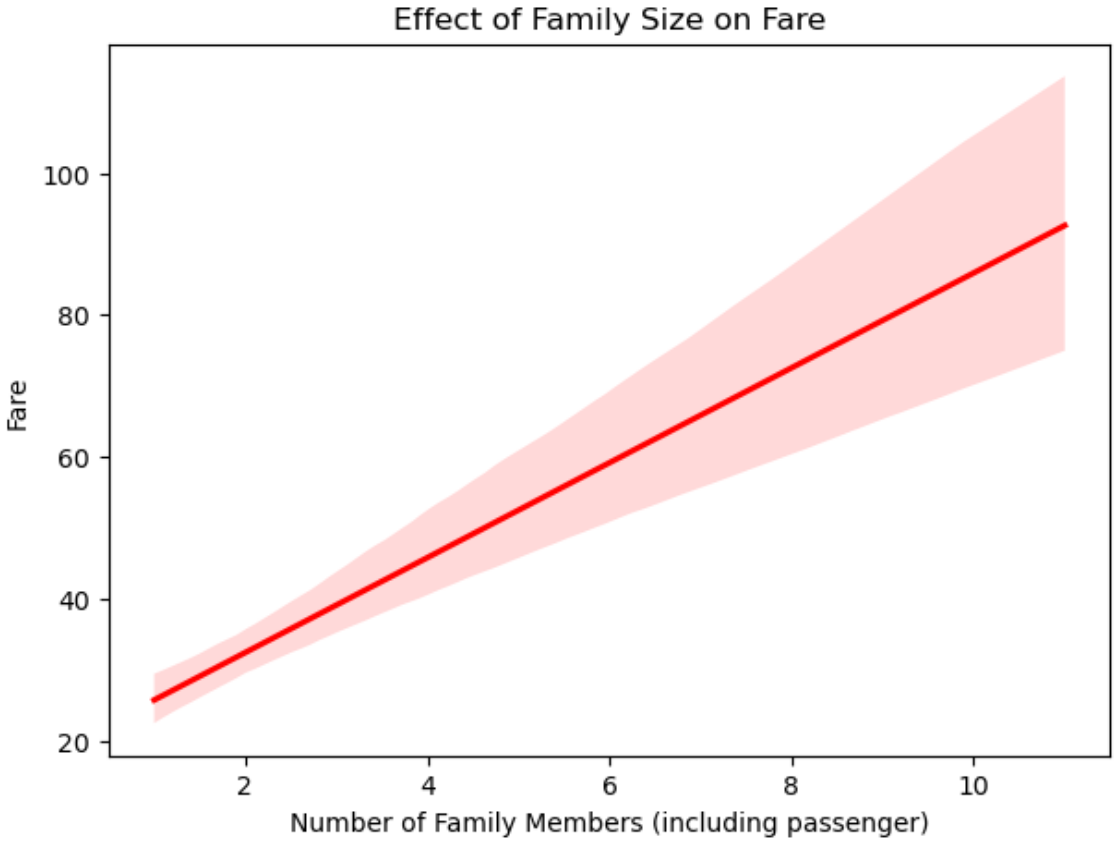
```
from sklearn import linear_model

reg = linear_model.LinearRegression()


reg.fit(df_copy[["Number of Family Members"]],df["Fare"])
reg.predict([[1]])
```

Out[357]:

```
array([26.15448989])
```

In [358]:

```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(7,5))
sns.regplot(x='Number of Family Members', y='Fare', data=df_copy, scatter=False, color='
red')
plt.title("Effect of Family Size on Fare")
plt.xlabel("Number of Family Members (including passenger)")
plt.ylabel("Fare")
plt.show()
```




**CONCLUSION AT THIS POINT: Ticket Prices increases as the no. of family members increases (for all passenger classes 1 , 2 and 3)**

# Q5] PREDICTING SURVIVAL BASED ON:

- **Age**
- **Fare**

In [359]:

```
df_copy.head(10)
```

Out[359]:

| | Survived | Pclass | Sex | Age | SibSp | Parch | Fare | Embarked | Number of Family Members | Embarkation Point Encoded |
|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 0 | 3 | 0 | 22 | 1 | 0 | 7 | S | 2 | 2 |
| **1** | 1 | 1 | 1 | 38 | 1 | 0 | 71 | C | 2 | 0 |
| **2** | 1 | 3 | 1 | 26 | 0 | 0 | 7 | S | 1 | 2 |
| **3** | 1 | 1 | 1 | 35 | 1 | 0 | 53 | S | 2 | 2 |
| **4** | 0 | 3 | 0 | 35 | 0 | 0 | 8 | S | 1 | 2 |
| **5** | 0 | 3 | 0 | 28 | 0 | 0 | 8 | Q | 1 | 1 |
| **6** | 0 | 1 | 0 | 54 | 0 | 0 | 51 | S | 1 | 2 |
| **7** | 0 | 3 | 0 | 2 | 3 | 1 | 21 | S | 5 | 2 |

| | Survived | Pclass | Sex | Age | SibSp | Parch | Fare | Embarked | Number of Family Members | Embarkation Point Encoded |
|---|---|---|---|---|---|---|---|---|---|---|
| 8 | 1 | 3 | 1 | 27 | 0 | 2 | 11 | S | 3 | 2 |
| 9 | 1 | 2 | 1 | 14 | 1 | 0 | 30 | C | 2 | 0 |

In [360]:

```
from sklearn.linear_model import LogisticRegression

reg = linear_model.LogisticRegression()


reg.fit(df_copy[["Age","Fare"]],df["Survived"])
reg.predict([[20,7.25]])
```

Out[360]:

```
array([0])
```

In [361]:

```
from sklearn.linear_model import LogisticRegression

reg = linear_model.LogisticRegression()


reg.fit(df_copy[["Age","Fare"]],df["Survived"])
reg.predict([[54,1030.07]])
```

Out[361]:

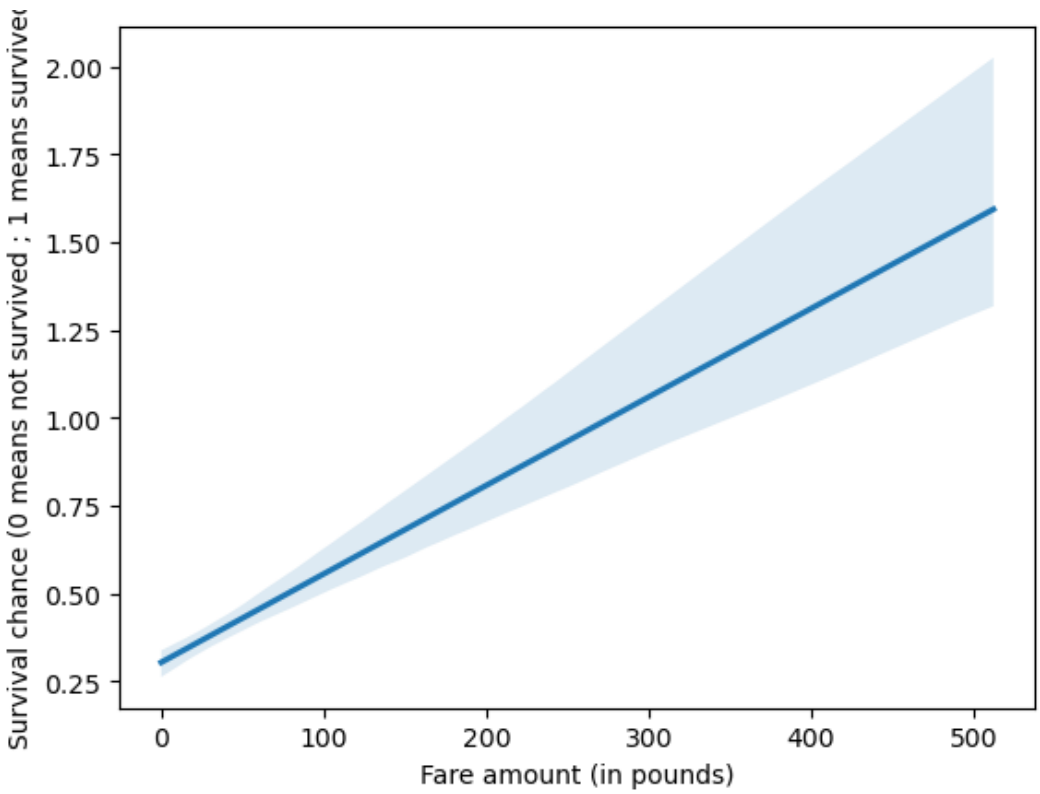
```
array([1])
```

In [362]:

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.regplot(x="Fare",y="Survived",data=df_copy,scatter=False)


plt.xlabel("Fare amount (in pounds)")
plt.ylabel("Survival chance (0 means not survived ; 1 means survived)")
```

Out[362]:

```
Text(0, 0.5, 'Survival chance (0 means not survived ; 1 means survived)')
```

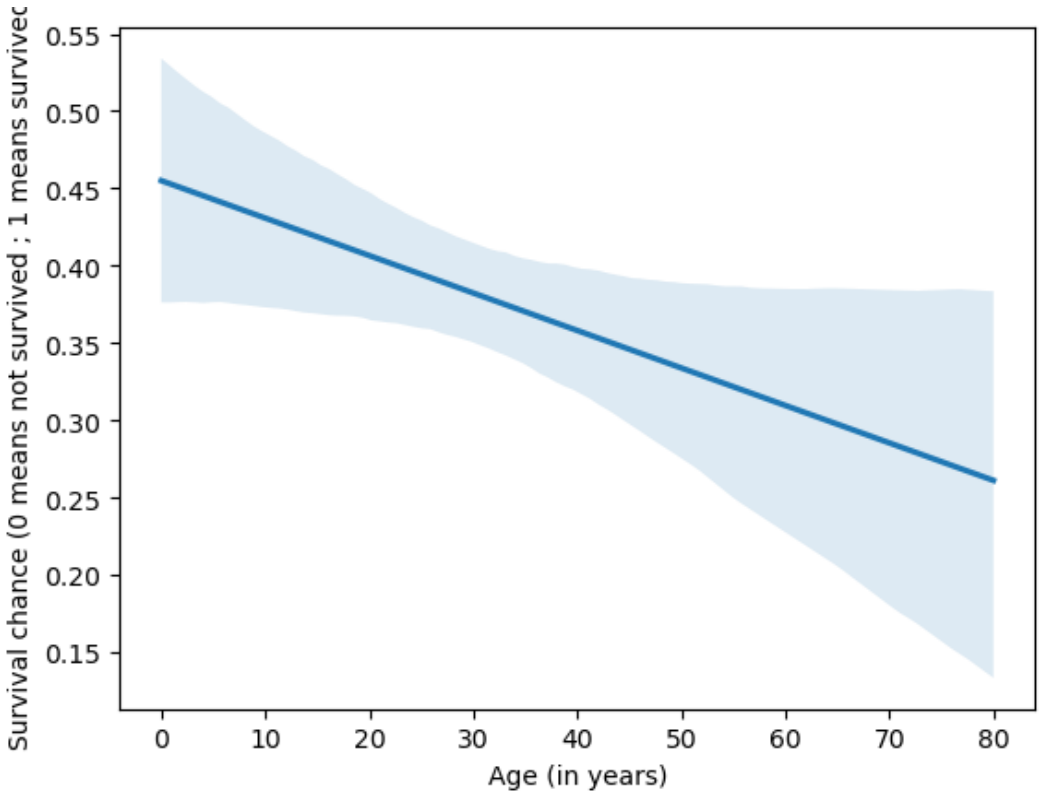


In [363]:

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.regplot(x="Age",y="Survived",data=df_copy,scatter=False)

plt.xlabel("Age (in years)")
plt.ylabel("Survival chance (0 means not survived ; 1 means survived)")
```

Out[363]:

```
Text(0, 0.5, 'Survival chance (0 means not survived ; 1 means survived)')
```




**CONCLUSION AT THIS POINT:**

- **A 20 year old (male/female) who paid nearly 7 pounds was likely not to survive.**
- **A 50 year old (male/female) who paid nearly 1000 pounds was likely to survive.**
- **People who spent more money on the trip; were more likely to survive.**
- **People who were elderly were more likely to not survive.**

In [ ]:

## CONCLUSION DRAWN FROM THE ENTIRE ANALYSIS (TILL HERE):

- **Passengers in higher classes (1st class) paid substantially higher fares than those in lower classes.**
- **Males in higher classes tended to be older, while females and lower-class passengers were generally younger.**
- **Customers in higher passenger-class pay higher fares**
- **Larger families tended to pay slightly higher total fares, possibly because they booked together or purchased multiple tickets.**
- **Younger passengers and those who paid higher fares (likely first-class passengers) had a higher chance of survival.**

In [ ]:

In [ ]:

In [ ]:

## USE DECISION-TREE TO PREDICT Passenger-Survival based on :

- **Pclass**
- **Sex**
- **Age**
- **Sibsp (No. of Siblings if any)**
- **Parch (No. of parents / children aboard on Titanic)**
- **Fare**

In [364]:

```
df_copy.head(5)
```

Out[364]:

| | Survived | Pclass | Sex | Age | SibSp | Parch | Fare | Embarked | Number of Family Members | Embarkation Point Encoded |
|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 0 | 3 | 0 | 22 | 1 | 0 | 7 | S | 2 | 2 |
| **1** | 1 | 1 | 1 | 38 | 1 | 0 | 71 | C | 2 | 0 |
| **2** | 1 | 3 | 1 | 26 | 0 | 0 | 7 | S | 1 | 2 |
| **3** | 1 | 1 | 1 | 35 | 1 | 0 | 53 | S | 2 | 2 |
| **4** | 0 | 3 | 0 | 35 | 0 | 0 | 8 | S | 1 | 2 |

In [365]:

```
from sklearn import tree
model = tree.DecisionTreeClassifier()


independent_variables = ["Pclass","Sex","Age","SibSp","Parch","Fare"]
model.fit(df_copy[independent_variables],df_copy["Survived"])
```

Out[365]:

▼ DecisionTreeClassifier i ?

▶ Parameters

In [366]:

```
#Check the model's accuracy:
model.score(df_copy[independent_variables],df_copy["Survived"])
```

Out[366]:

```
0.9528619528619529
```

In [367]:

```
df.head(5)
```

Out[367]:

| PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | Parch | Ticket | Fare | Embarked |
|---|---|---|---|---|---|---|---|---|---|---|

| | PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | Parch | Ticket | Fare | Embarked |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 0 | 3 | Braund, Mr. Owen Harris | male | 22.0 | 1 | 0 | A/5 21171 | 7.2500 | S |
| 1 | 2 | 1 | 1 | Cumings, Mrs. John Bradley (Florence Briggs Th... | female | 38.0 | 1 | 0 | PC 17599 | 71.2833 | C |
| 2 | 3 | 1 | 3 | Heikkinen, Miss. Laina | female | 26.0 | 0 | 0 | STON/O2. 3101282 | 7.9250 | S |
| 3 | 4 | 1 | 1 | Futrelle, Mrs. Jacques Heath (Lily May Peel) | female | 35.0 | 1 | 0 | 113803 | 53.1000 | S |
| 4 | 5 | 0 | 3 | Allen, Mr. William Henry | male | 35.0 | 0 | 0 | 373450 | 8.0500 | S |

In [368]:

```
df_copy.head(5)
```

Out[368]:

| | Survived | Pclass | Sex | Age | SibSp | Parch | Fare | Embarked | Number of Family Members | Embarkation Point Encoded |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 3 | 0 | 22 | 1 | 0 | 7 | S | 2 | 2 |
| 1 | 1 | 1 | 1 | 38 | 1 | 0 | 71 | C | 2 | 0 |
| 2 | 1 | 3 | 1 | 26 | 0 | 0 | 7 | S | 1 | 2 |
| 3 | 1 | 1 | 1 | 35 | 1 | 0 | 53 | S | 2 | 2 |
| 4 | 0 | 3 | 0 | 35 | 0 | 0 | 8 | S | 1 | 2 |

## Now make a prediction for a person with these details:

- Passenger-class : 3
- Gender : Male
- Age: 22
- No. of siblings: 1
- No. of parents and children: 0
- Trip Fare: 7 pounds (approx.)

In [369]:

```
model.predict([[3,0,22.0,1,0,7.25]])
```

Out[369]:

```
array([0])
```

**CONCLUSION OF THIS POINT: PASSENGER WITH THESE DETAILS WAS LIKELY TO SURVIVE**

- Passenger-class : 3
- Gender : Male
- Age: 22
- No. of siblings: 1
- No. of parents and children: 0
- Trip Fare: 7 pounds (approx.)

In [ ]:

```

```

In [370]:

```
df_copy.head(5)
```

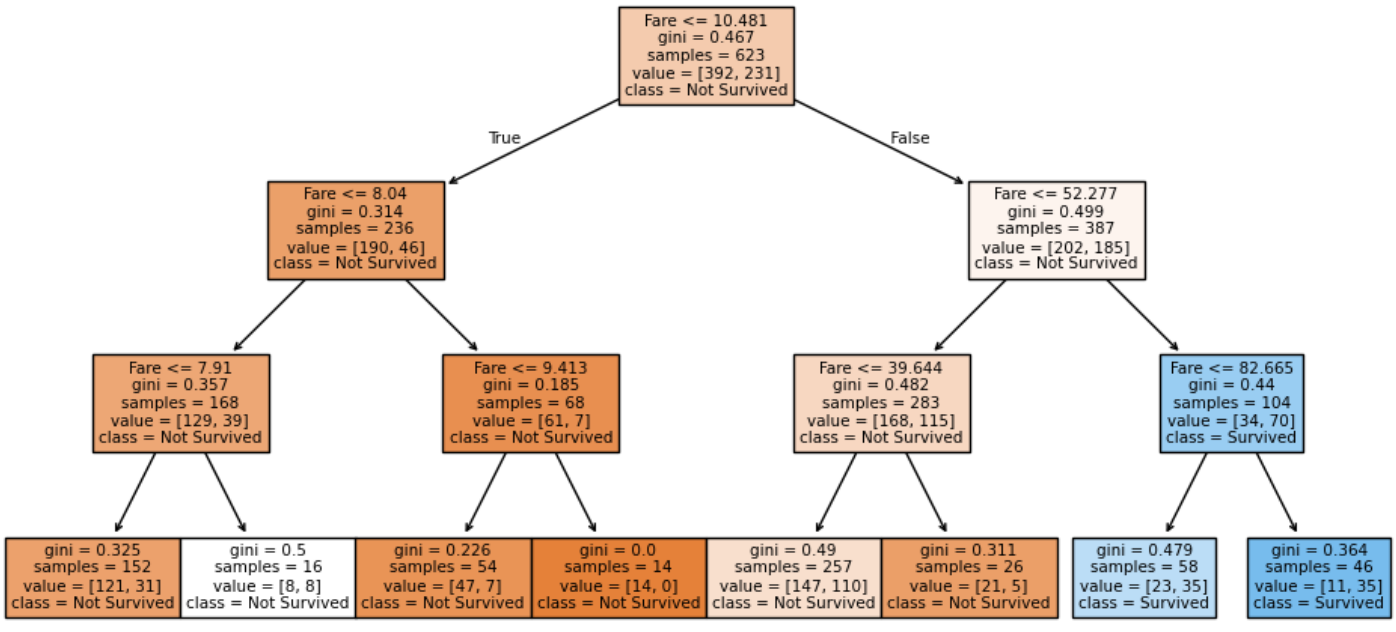
Out[370]:

| | Survived | Pclass | Sex | Age | SibSp | Parch | Fare | Embarked | Number of Family Members | Embarkation Point Encoded |
|---|---|---|---|---|---|---|---|---|---|---|

| | Survived | Pclass | Sex | Age | SibSp | Parch | Fare | Embarked | Number of Family Members | Embarkation Point Encoded |
|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 38 | 1 | 0 | 71 | C | 2 | 0 |
| 2 | 1 | 3 | 1 | 26 | 0 | 0 | 7 | S | 1 | 2 |
| 3 | 1 | 1 | 1 | 35 | 1 | 0 | 53 | S | 2 | 2 |
| 4 | 0 | 3 | 0 | 35 | 0 | 0 | 8 | S | 1 | 2 |

In [371]:

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt


X = df[['Fare']]
y = df['Survived']


X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42
)


model = DecisionTreeClassifier(max_depth=3, random_state=42)
model.fit(X_train, y_train)


plt.figure(figsize=(12,6))
plot_tree(model, feature_names=['Fare'], class_names=['Not Survived','Survived'], filled
=True)
plt.show()


print("Accuracy:", model.score(X_test, y_test))
```




```
Accuracy: 0.6716417910447762
```

**CONCLUSION OF THIS POINT: Passengers who paid higher prices (i.e. more then 83 pounds) had a much greater chance of survival.)**

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [372]:

```
df.head(5)
```

Out[372]:

| | PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | Parch | Ticket | Fare | Embarked |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 0 | 3 | Braund, Mr. Owen Harris | male | 22.0 | 1 | 0 | A/5 21171 | 7.2500 | S |
| 1 | 2 | 1 | 1 | Cumings, Mrs. John Bradley (Florence Briggs Th... | female | 38.0 | 1 | 0 | PC 17599 | 71.2833 | C |
| 2 | 3 | 1 | 3 | Heikkinen, Miss. Laina | female | 26.0 | 0 | 0 | STON/O2. 3101282 | 7.9250 | S |
| 3 | 4 | 1 | 1 | Futrelle, Mrs. Jacques Heath (Lily May Peel) | female | 35.0 | 1 | 0 | 113803 | 53.1000 | S |
| 4 | 5 | 0 | 3 | Allen, Mr. William Henry | male | 35.0 | 0 | 0 | 373450 | 8.0500 | S |

In [373]:

```
df_copy.head(5)
```

Out[373]:

| | Survived | Pclass | Sex | Age | SibSp | Parch | Fare | Embarked | Number of Family Members | Embarkation Point Encoded |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 3 | 0 | 22 | 1 | 0 | 7 | S | 2 | 2 |
| 1 | 1 | 1 | 1 | 38 | 1 | 0 | 71 | C | 2 | 0 |
| 2 | 1 | 3 | 1 | 26 | 0 | 0 | 7 | S | 1 | 2 |
| 3 | 1 | 1 | 1 | 35 | 1 | 0 | 53 | S | 2 | 2 |
| 4 | 0 | 3 | 0 | 35 | 0 | 0 | 8 | S | 1 | 2 |

In [ ]:

**KNN QUESTION ] Using the Titanic dataset, build a K-Nearest Neighbors (KNN) model to predict whether a passenger (male / female) survived or not based on their Age , Gender and Fare. After training, predict the survival status of a female passenger aged 30 years who paid a fare of $100. Finally, find out how well your model performs using the accuracy score.**

In [374]:

```
df.head()
```

Out[374]:

| | PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | Parch | Ticket | Fare | Embarked |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 0 | 3 | Braund, Mr. Owen Harris | male | 22.0 | 1 | 0 | A/5 21171 | 7.2500 | S |
| 1 | 2 | 1 | 1 | Cumings, Mrs. John Bradley (Florence Briggs Th... | female | 38.0 | 1 | 0 | PC 17599 | 71.2833 | C |
| 2 | 3 | 1 | 3 | Heikkinen, Miss. Laina | female | 26.0 | 0 | 0 | STON/O2. 3101282 | 7.9250 | S |

| | PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | Parch | Ticket | Fare | Embarked |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 3 | 4 | 1 | 1 | Futrelle, Mrs. Jacques Heath (Lily May Peel) | female | 35.0 | 1 | 0 | 113803 | 53.1000 | S |
| 4 | 5 | 0 | 3 | Allen, Mr. William Henry | male | 35.0 | 0 | 0 | 373450 | 8.0500 | S |

In [375]:

```
df_copy.head()
```

Out[375]:

| | Survived | Pclass | Sex | Age | SibSp | Parch | Fare | Embarked | Number of Family Members | Embarkation Point Encoded |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 3 | 0 | 22 | 1 | 0 | 7 | S | 2 | 2 |
| 1 | 1 | 1 | 1 | 38 | 1 | 0 | 71 | C | 2 | 0 |
| 2 | 1 | 3 | 1 | 26 | 0 | 0 | 7 | S | 1 | 2 |
| 3 | 1 | 1 | 1 | 35 | 1 | 0 | 53 | S | 2 | 2 |
| 4 | 0 | 3 | 0 | 35 | 0 | 0 | 8 | S | 1 | 2 |

In [376]:

```
df.isnull().sum()
```

Out[376]:

```
PassengerId    0
Survived       0
Pclass         0
Name           0
Sex            0
Age            0
SibSp          0
Parch          0
Ticket         0
Fare           0
Embarked       0
dtype: int64
```

In [377]:

```
df_copy.isnull().sum()
```

Out[377]:

```
Survived                      0
Pclass                        0
Sex                           0
Age                           0
SibSp                         0
Parch                         0
Fare                          0
Embarked                      0
Number of Family Members      0
Embarkation Point Encoded     0
dtype: int64
```

In [378]:

```
df_copy.head()
```

Out[378]:

| | Survived | Pclass | Sex | Age | SibSp | Parch | Fare | Embarked | Number of Family Members | Embarkation Point Encoded |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 3 | 0 | 22 | 1 | 0 | 7 | S | 2 | 2 |
| 1 | 1 | 1 | 1 | 38 | 1 | 0 | 71 | C | 2 | 0 |
| 2 | 1 | 3 | 1 | 26 | 0 | 0 | 7 | S | 1 | 2 |

| | Survived | Pclass | Sex | Age | SibSp | Parch | Fare | Embarked | Number of Family Members | Embarkation Point Encoded |
|---|---|---|---|---|---|---|---|---|---|---|
| 3 | 1 | 1 | 1 | 35 | 1 | 0 | 53 | S | 2 | 2 |
| 4 | 0 | 3 | 0 | 35 | 0 | 0 | 8 | S | 1 | 2 |

In [379]:

```
inputs = df_copy.drop(["Survived","Pclass","SibSp","Parch","Embarked","Number of Family Members","Embarkation Point Encoded"],axis="columns")
```

In [380]:

```
outputs = df_copy.drop(["Age","Fare","Pclass","Sex","SibSp","Parch","Embarked","Number of Family Members","Embarkation Point Encoded"],axis="columns")
```

In [381]:

```
inputs
```

Out[381]:

| | Sex | Age | Fare |
|---|---|---|---|
| 0 | 0 | 22 | 7 |
| 1 | 1 | 38 | 71 |
| 2 | 1 | 26 | 7 |
| 3 | 1 | 35 | 53 |
| 4 | 0 | 35 | 8 |
| ... | ... | ... | ... |
| 886 | 0 | 27 | 13 |
| 887 | 1 | 19 | 30 |
| 888 | 1 | 28 | 23 |
| 889 | 0 | 26 | 30 |
| 890 | 0 | 32 | 7 |

891 rows × 3 columns

In [382]:

```
outputs
```

Out[382]:

| | Survived |
|---|---|
| 0 | 0 |
| 1 | 1 |
| 2 | 1 |
| 3 | 1 |
| 4 | 0 |
| ... | ... |
| 886 | 0 |
| 887 | 1 |
| 888 | 0 |
| 889 | 1 |
| 890 | 0 |

891 rows × 1 columns

In [383]:

```
from sklearn.model_selection import train_test_split
X = inputs
y = outputs
```

In [384]:

```
X
```

Out[384]:

| | Sex | Age | Fare |
|---|---|---|---|
| 0 | 0 | 22 | 7 |
| 1 | 1 | 38 | 71 |
| 2 | 1 | 26 | 7 |
| 3 | 1 | 35 | 53 |
| 4 | 0 | 35 | 8 |
| ... | ... | ... | ... |
| 886 | 0 | 27 | 13 |
| 887 | 1 | 19 | 30 |
| 888 | 1 | 28 | 23 |
| 889 | 0 | 26 | 30 |
| 890 | 0 | 32 | 7 |

**891 rows × 3 columns**

In [385]:

```
y
```

Out[385]:

| | Survived |
|---|---|
| 0 | 0 |
| 1 | 1 |
| 2 | 1 |
| 3 | 1 |
| 4 | 0 |
| ... | ... |
| 886 | 0 |
| 887 | 1 |
| 888 | 0 |
| 889 | 1 |
| 890 | 0 |

**891 rows × 1 columns**

In [386]:

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=1)
```

In [387]:

```
len(X_train)
```

Out[387]:

712

In [388]:

```
len(X_test)
```

Out[388]:

179

### Create KNN (K Neighrest Neighbour Classifier)

In [389]:

```
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors=5)
```

In [390]:

```
knn.fit(X_train, y_train)
```

Out[390]:

▼ KNeighborsClassifier i ?

▶ Parameters

In [391]:

```
knn.score(X_test,y_test)
```

Out[391]:

0.7318435754189944

In [392]:

```
knn.predict([[1,38,71]])
```

Out[392]:

array([1])

In [393]:

```
df.head()
```

Out[393]:

| | PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | Parch | Ticket | Fare | Embarked |
|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 1 | 0 | 3 | Braund, Mr. Owen Harris | male | 22.0 | 1 | 0 | A/5 21171 | 7.2500 | S |
| **1** | 2 | 1 | 1 | Cumings, Mrs. John Bradley (Florence Briggs Th... | female | 38.0 | 1 | 0 | PC 17599 | 71.2833 | C |
| **2** | 3 | 1 | 3 | Heikkinen, Miss. Laina | female | 26.0 | 0 | 0 | STON/O2. 3101282 | 7.9250 | S |
| **3** | 4 | 1 | 1 | Futrelle, Mrs. Jacques Heath (Lily May Peel) | female | 35.0 | 1 | 0 | 113803 | 53.1000 | S |
| **4** | 5 | 0 | 3 | Allen, Mr. William Henry | male | 35.0 | 0 | 0 | 373450 | 8.0500 | S |

In [394]:

```
df_copy.head()
```

Out[394]:

| | Survived | Pclass | Sex | Age | SibSp | Parch | Fare | Embarked | Number of Family Members | Embarkation Point Encoded |
|---|---|---|---|---|---|---|---|---|---|---|

| | Survived | Pclass | Sex | Age | SibSp | Parch | Fare | Embarked | Number of Family Members | Embarkation Point Encoded |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 3 | 0 | 22 | 1 | 0 | 7 | S | 2 | 2 |
| 1 | 1 | 1 | 1 | 38 | 1 | 0 | 71 | C | 2 | 0 |
| 2 | 1 | 3 | 1 | 26 | 0 | 0 | 7 | S | 1 | 2 |
| 3 | 1 | 1 | 1 | 35 | 1 | 0 | 53 | S | 2 | 2 |
| 4 | 0 | 3 | 0 | 35 | 0 | 0 | 8 | S | 1 | 2 |

**CONCLUSION OF THIS KNN MODEL: A Female Person of 38 years of age and with an income of 71 pounds (British pounds) is likely to survive**

**Plot Confusion Matrix**

In [395]:

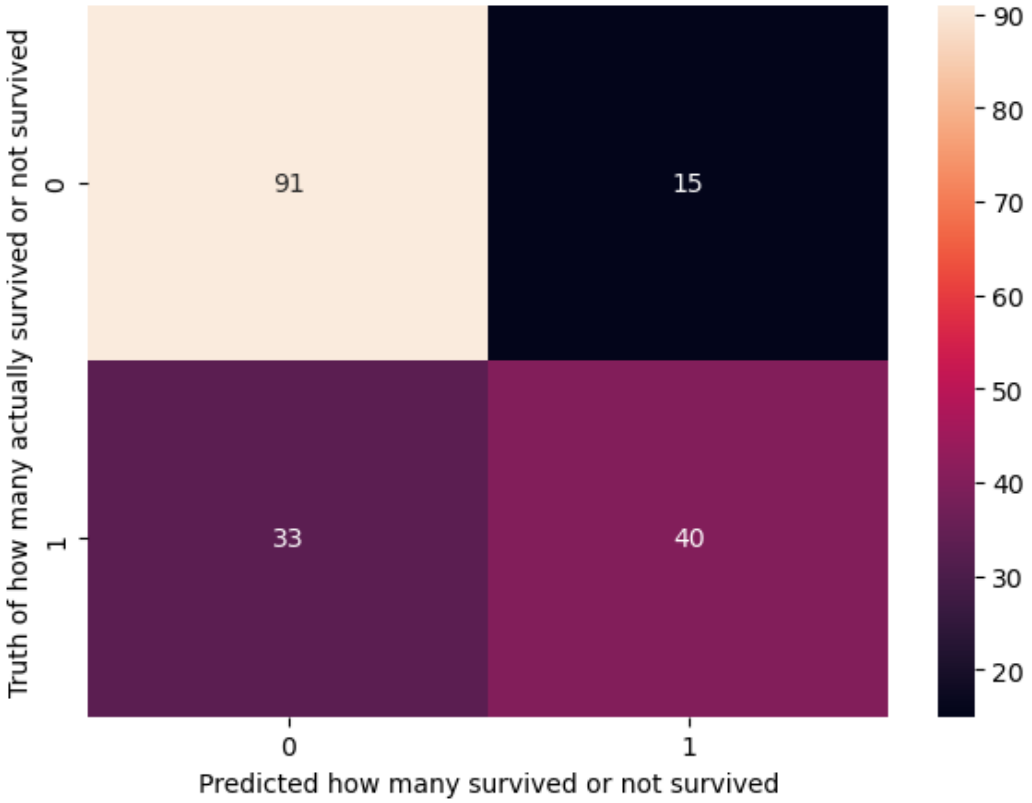
```
from sklearn.metrics import confusion_matrix
y_pred = knn.predict(X_test)
cm = confusion_matrix(y_test, y_pred)
cm
```

Out[395]:

```
array([[91, 15],
       [33, 40]])
```

In [396]:

```
import matplotlib.pyplot as plt
import seaborn as sn
plt.figure(figsize=(7,5))
sn.heatmap(cm, annot=True)
plt.xlabel('Predicted how many survived or not survived')
plt.ylabel('Truth of how many actually survived or not survived')
plt.show()
```




**Prepare a classification report:**

In [397]:

```
from sklearn.metrics import classification_report
```

```
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.73      0.86      0.79       106
           1       0.73      0.55      0.62        73

    accuracy                           0.73       179
   macro avg       0.73      0.70      0.71       179
weighted avg       0.73      0.73      0.72       179
```

**Meaning of the heatmap created : 0s and 1s on the heatmap means :**

- ***0 means passenger did not survived***
- ***1 means passenger survived***

**CONCLUSIONS DRAWN FROM THE ABOVE HEATMAP CREATED :**

```
- We accurately predicted that 91 passengers didnt survive and 40 passengers surviv
ed
- 33 passengers actually survived but we falsely predicted that they didnt survive
- We falsely predicted that 15 passengers survived but actually they didnt.
```

In [ ]:

```
```

In [ ]:

```
```

**RANDOM FOREST QUESTION] Using a Random Forest Classifier, find out which features — Sex, Age, or Fare — are most important for predicting survival.**

In [398]:

```
df.head()
```

Out[398]:

| | PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | Parch | Ticket | Fare | Embarked |
|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 1 | 0 | 3 | Braund, Mr. Owen Harris | male | 22.0 | 1 | 0 | A/5 21171 | 7.2500 | S |
| **1** | 2 | 1 | 1 | Cumings, Mrs. John Bradley (Florence Briggs Th... | female | 38.0 | 1 | 0 | PC 17599 | 71.2833 | C |
| **2** | 3 | 1 | 3 | Heikkinen, Miss. Laina | female | 26.0 | 0 | 0 | STON/O2. 3101282 | 7.9250 | S |
| **3** | 4 | 1 | 1 | Futrelle, Mrs. Jacques Heath (Lily May Peel) | female | 35.0 | 1 | 0 | 113803 | 53.1000 | S |
| **4** | 5 | 0 | 3 | Allen, Mr. William Henry | male | 35.0 | 0 | 0 | 373450 | 8.0500 | S |

In [399]:

```
df_copy.head()
```

Out[399]:

| | Survived | Pclass | Sex | Age | SibSp | Parch | Fare | Embarked | Number of Family Members | Embarkation Point Encoded |
|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 0 | 3 | 0 | 22 | 1 | 0 | 7 | S | 2 | 2 |
| **1** | 1 | 1 | 1 | 38 | 1 | 0 | 71 | C | 2 | 0 |
| **2** | 1 | 3 | 1 | 26 | 0 | 0 | 7 | S | 1 | 2 |

| | Survived | Pclass | Sex | Age | SibSp | Parch | Fare | Embarked | Number of Family Members | Embarkation Point Encoded |
|---|---|---|---|---|---|---|---|---|---|---|
| 3 | 1 | 1 | 1 | 35 | 1 | 0 | 53 | S | 2 | 2 |
| 4 | 0 | 3 | 0 | 35 | 0 | 0 | 8 | S | 1 | 2 |

In [400]:

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import numpy as np

X = df_copy[["Sex","Age","Fare"]]
y = df_copy["Survived"]

X_train, X_test, y_train, y_test = train_test_split( X, y, test_size = 0.25, random_stat
e=42) # Split dataset into 75% train and 25% test

randomForest_Model = RandomForestClassifier(n_estimators=100, random_state=42)
randomForest_Model.fit(X_train, y_train)

#Check model score:
randomForest_Model.score(X_test,y_test)
```

Out[400]:

```
0.7668161434977578
```

In [401]:

```
#Make Predictions:
y_predicted = randomForest_Model.predict(X_test)
```

In [402]:

```
y_predicted
```

Out[402]:

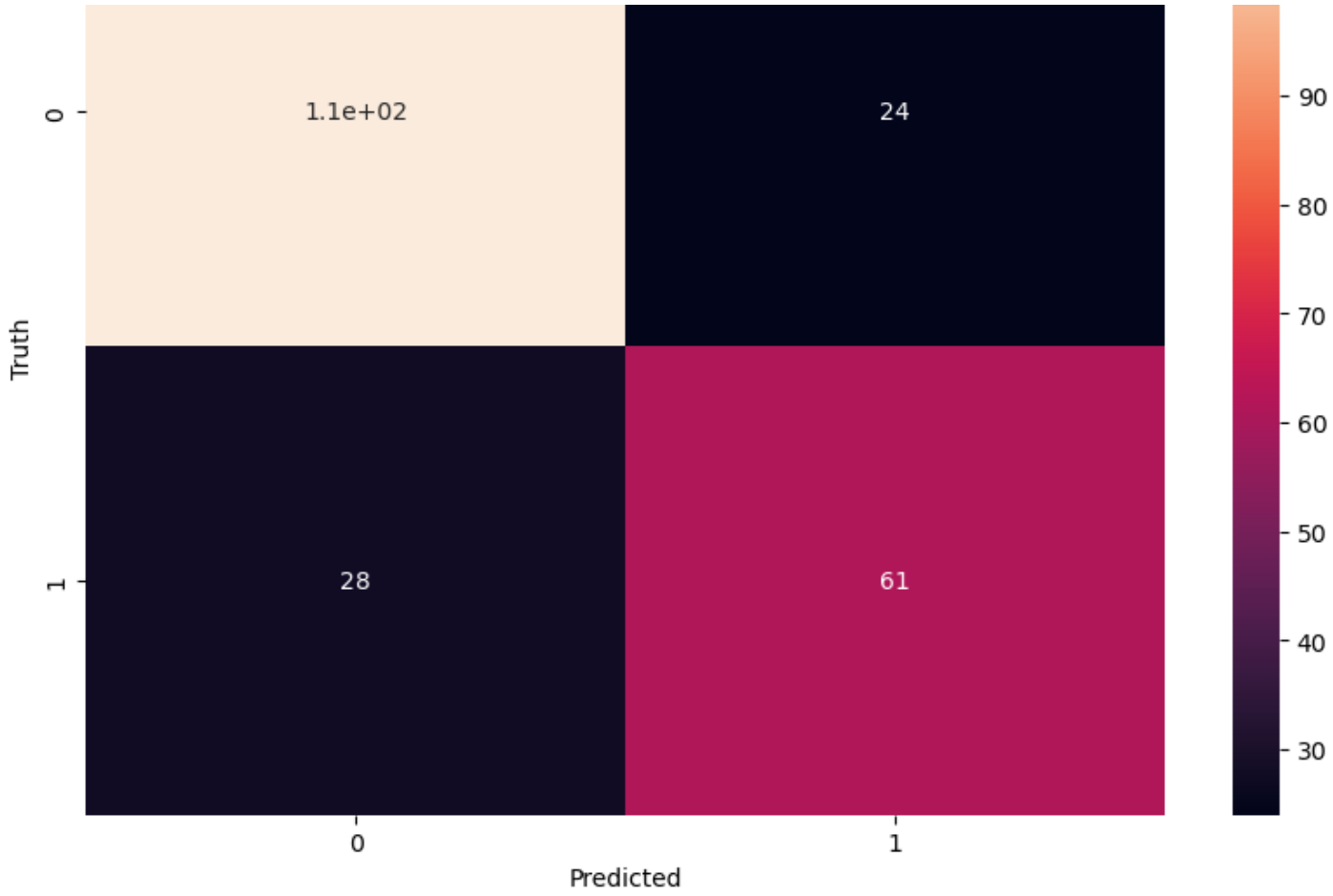
```
array([0, 0, 0, 1, 1, 1, 1, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0,
       1, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1, 1, 0, 0, 0, 1, 1, 0, 0, 1, 0, 0,
       1, 0, 0, 0, 0, 0, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 0, 1, 0, 0, 1,
       0, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 0, 1, 1,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,
       1, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 1, 1, 1, 0, 0, 1, 0, 0, 0, 1, 0,
       0, 1, 1, 0, 1, 1, 0, 0, 0, 1, 1, 0, 0, 1, 0, 1, 1, 0, 0, 0, 0, 1,
       0, 0, 0, 1, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0,
       0, 1, 1, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 1, 1, 0,
       1, 0, 0, 1, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 1, 0, 1, 0,
       0, 1, 0])
```

In [403]:

```
# Confusion Matrix:
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_predicted)


import matplotlib.pyplot as plt
import seaborn as sn
plt.figure(figsize=(10,7))
sn.heatmap(cm, annot=True)
plt.xlabel('Predicted')
plt.ylabel('Truth')
```

Out[403]:

```
Text(95.72222222222221, 0.5, 'Truth')
```




**CONCLUSION FROM THE ABOVE HEATMAP]**

- **The model correctly predicted that :**
  - **110 passengers did not survived**
  - **61 passengers survived**
- **The model incorrectly predicted that :**
  - **28 passengers didnt survive ; but actually they did survive**
  - **24 passengers survived ; but actually they didnt**

In [ ]:

```
```

In [ ]:

```
```

In [ ]:

```
```

## NAIVE BAYES QUESTION] Predict a passenger's survival chances based on his'/her's passenger-class and family-size ; using naive bayes

In [404]:

```
df_copy.head(5)
```

Out[404]:

| | Survived | Pclass | Sex | Age | SibSp | Parch | Fare | Embarked | Number of Family Members | Embarkation Point Encoded |
|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 0 | 3 | 0 | 22 | 1 | 0 | 7 | S | 2 | 2 |
| **1** | 1 | 1 | 1 | 38 | 1 | 0 | 71 | C | 2 | 0 |
| **2** | 1 | 3 | 1 | 26 | 0 | 0 | 7 | S | 1 | 2 |

| | Survived | Pclass | Sex | Age | SibSp | Parch | Fare | Embarked | Number of Family Members | Embarkation Point Encoded |
|---|---|---|---|---|---|---|---|---|---|---|
| 3 | 1 | 1 | 1 | 35 | 1 | 0 | 53 | S | 2 | 2 |
| 4 | 0 | 3 | 0 | 35 | 0 | 0 | 8 | S | 1 | 2 |

In [405]:

```
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score

X = df_copy[["Pclass","Number of Family Members"]]
y = df_copy["Survived"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42
)

naive_bayes_model = MultinomialNB()
naive_bayes_model.fit(X_train, y_train)

# Make predictions for the testing-data:
y_pred = naive_bayes_model.predict(X_test)

# Check the accuracy of the model:
print("Predicted:", y_pred)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
Predicted: [0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 1 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0]
Accuracy: 0.5783582089552238
```

In [406]:

```
new_passengers = pd.DataFrame([
    [1, 3],    # Example 1: 1st class, 3 family members
    [3, 1],    # Example 2: 3rd class, 1 family member
    [2, 0],    # Example 3: 2nd class, traveling alone
    [3, 5],    # Example 4: 3rd class, 5 family members
    [2,10],    # Example 5: 2nd class, 10 family members
    [1,4],     # Example 6: 1st class, 4 family members
    [3,30]     # Example 7: 3rd class, 30 family members
], columns=["Pclass", "Number of Family Members"])

predictions = naive_bayes_model.predict(new_passengers)

# Show results
result_df = new_passengers.copy()
result_df["Predicted Survival (1=Survived, 0=Not Survived)"] = predictions
result_df
```

Out[406]:

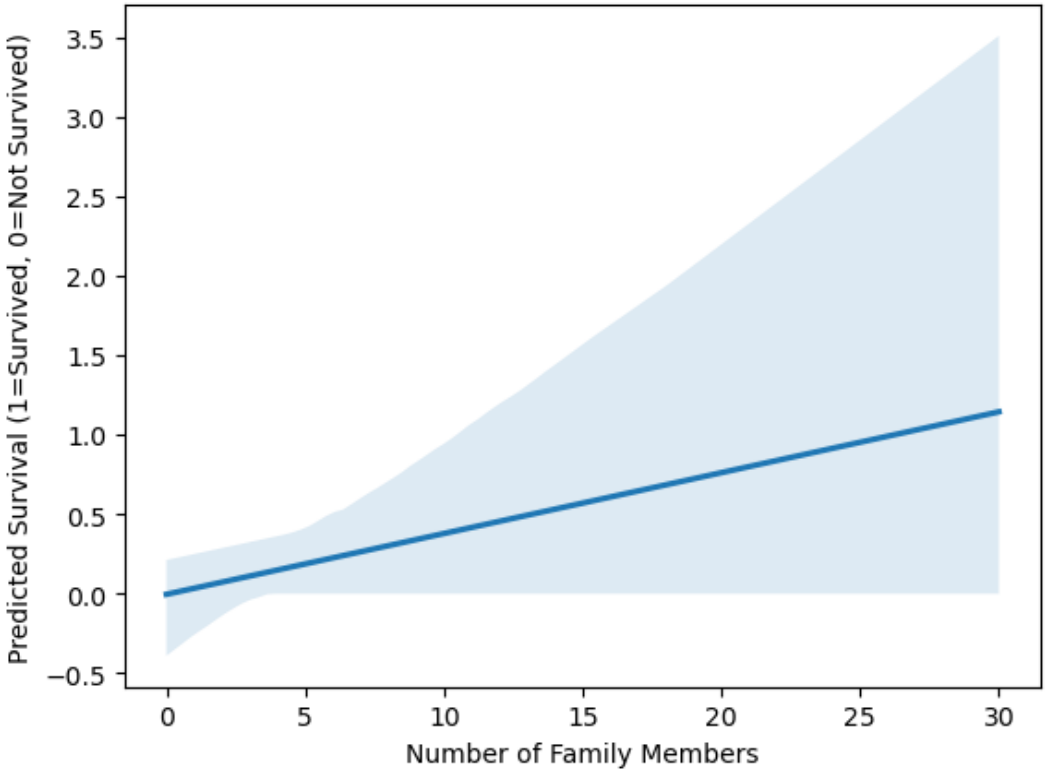
| | Pclass | Number of Family Members | Predicted Survival (1=Survived, 0=Not Survived) |
|---|---|---|---|
| 0 | 1 | 3 | 0 |
| 1 | 3 | 1 | 0 |
| 2 | 2 | 0 | 0 |
| 3 | 3 | 5 | 0 |
| 4 | 2 | 10 | 1 |
| 5 | 1 | 4 | 0 |
| 6 | 3 | 30 | 1 |

In [407]:

```
import seaborn as sns
sns.regplot(scatter=False,data=result_df,x="Number of Family Members",y="Predicted Survival (1=Survived, 0=Not Survived)")
```

Out[407]:

```
<Axes: xlabel='Number of Family Members', ylabel='Predicted Survival (1=Survived, 0=Not Survived)'>
```




**CONCLUSION OF THE ABOVE VISUALIZATION: Larger families had the highest chance of survival.**

In [ ]:

In [ ]:

In [ ]:

In [408]:

```
from IPython.display import Image, display

display(Image(filename="ending_image.jpg", width=700,height=300))
```

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

**END OF THE NOTEBOOK**

**Prepared by:**


- **Name: Om Satyawan Pathak**
- **Contact: omsatyawanpathakwebdevelopment@gmail.com (or) omsatyawanpathakgit@gmail.com**


In [ ]:

**ZOMATO EDA PROJECT**

***(in Python)***

***(including Machine Learning)***

***Github Link:***

*https://github.com/omsatyawanpathakgit/DataAnalysisProjects/tree/main/Zomato%20EDA%20Project%20in%20Python*

***(Project by Om Satyawan Pathak)***

In [169]:

```
from IPython.display import Image
Image("ZomatoCompany.jpg")
```

Out[169]:


In [ ]:

In [ ]:

# EXPLORATORY DATA ANALYSIS (EDA) ON Zomato Dataset:

In [ ]:

In [170]:

```
import pandas as pd
import re
import warnings
warnings.filterwarnings('ignore')

df = pd.read_csv("zomato.csv")
```

## Columns description

- **url:** contains the url of the restaurant in the zomato website
- **address:** contains the address of the restaurant in Bengaluru
- **name:** contains the name of the restaurant
- **online_order:** whether online ordering is available in the restaurant or not
- **book_table:** table book option available or not
- **rate:** contains the overall rating of the restaurant out of 5

- **rate:** contains the overall rating of the restaurant out of 5
- **votes:** contains total number of rating for the restaurant as of the above mentioned date
- **phone:** contains the phone number of the restaurant
- **location:** contains the neighborhood in which the restaurant is located
- **rest_type:** restaurant type
- **dish_liked:** dishes people liked in the restaurant
- **cuisines:** food styles, separated by comma
- **approx_cost(for two people):** contains the approximate cost for meal for two people
- **reviews_list:** list of tuples containing reviews for the restaurant, each tuple
- **menu_item:** contains list of menus available in the restaurant
- **listed_in(type):** type of meal
- **listed_in(city):** contains the neighborhood in which the restaurant is listed

# *DATA CLEANING PART* :

In [171]:

```
df.head()
```

Out[171]:

| | url | address | name | online_order | book_table | rate | votes | |
|---|---|---|---|---|---|---|---|---|
| 0 | https://www.zomato.com/bangalore/jalsa-banasha... | 942, 21st Main Road, 2nd Stage, Banashankari, ... | Jalsa | Yes | Yes | 4.1/5 | 775 | 42297555 9743 |
| 1 | https://www.zomato.com/bangalore/spice-elephan... | 2nd Floor, 80 Feet Road, Near Big Bazaar, 6th ... | Spice Elephant | Yes | No | 4.1/5 | 787 | 080 41 |
| 2 | https://www.zomato.com/SanchurroBangalore?cont... | 1112, Next to KIMS Medical College, 17th Cross... | San Churro Cafe | Yes | No | 3.8/5 | 918 | +91 9663 |
| 3 | https://www.zomato.com/bangalore/addhuri-udupi... | 1st Floor, Annakuteera, 3rd Stage, Banashankar... | Addhuri Udupi Bhojana | No | No | 3.7/5 | 88 | +91 9620 |
| 4 | https://www.zomato.com/bangalore/grand-village... | 10, 3rd Floor, Lakshmi Associates, Gandhi Baza... | Grand Village | No | No | 3.8/5 | 166 | 8026612447 9901 |

In [172]:

```
df.isnull().sum()
```

Out[172]:

```
url                            0
address                        0
name                           0
online_order                   0
```

```
book_table                         0
rate                            7775
votes                              0
phone                           1208
location                          21
rest_type                        227
dish_liked                     28078
cuisines                          45
approx_cost(for two people)      346
reviews_list                       0
menu_item                          0
listed_in(type)                    0
listed_in(city)                    0
dtype: int64
```

In [173]:

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 51717 entries, 0 to 51716
Data columns (total 17 columns):
 #   Column                       Non-Null Count  Dtype 
---  ------                       --------------  ----- 
 0   url                          51717 non-null  object
 1   address                      51717 non-null  object
 2   name                         51717 non-null  object
 3   online_order                 51717 non-null  object
 4   book_table                   51717 non-null  object
 5   rate                         43942 non-null  object
 6   votes                        51717 non-null  int64 
 7   phone                        50509 non-null  object
 8   location                     51696 non-null  object
 9   rest_type                    51490 non-null  object
 10  dish_liked                   23639 non-null  object
 11  cuisines                     51672 non-null  object
 12  approx_cost(for two people)  51371 non-null  object
 13  reviews_list                 51717 non-null  object
 14  menu_item                    51717 non-null  object
 15  listed_in(type)              51717 non-null  object
 16  listed_in(city)              51717 non-null  object
dtypes: int64(1), object(16)
memory usage: 6.7+ MB
```

In [174]:

```
df.shape
```

Out[174]:

```
(51717, 17)
```

In [175]:

```
df["rate"] = df["rate"].fillna("0/5")
df["dish_liked"] = df["dish_liked"].fillna("0/5")
df.dropna(subset=["phone","location","dish_liked","rest_type","approx_cost(for two peopl
e)","cuisines"],inplace=True)
df = df.drop(columns=["listed_in(city)"])
```

In [176]:

```
df.shape
```

Out[176]:

```
(50279, 16)
```

In [177]:

```
df.isnull().sum()
```

Out[177]:

```
url                            0
address                        0
name                           0
online_order                   0
book_table                     0
rate                           0
votes                          0
phone                          0
location                       0
rest_type                      0
dish_liked                     0
cuisines                       0
approx_cost(for two people)    0
reviews_list                   0
menu_item                      0
listed_in(type)                0
dtype: int64
```

In [178]:

```
df['rate'] = df['rate'].str.replace('/5', '', regex=False)

def clean_phone(cell):
    if pd.isna(cell):
        return ''
    # Split by line breaks
    numbers = re.split(r'\r\n|\n', cell)
    cleaned_numbers = []
    for num in numbers:
        # Remove spaces, keep only digits
        num = re.sub(r'\D', '', num)
        cleaned_numbers.append(num)
    # Return as list if multiple, or first number only
    return cleaned_numbers[0] if len(cleaned_numbers) > 1 else cleaned_numbers[0]

# Apply cleaning
df['phone'] = df['phone'].apply(clean_phone)
```

In [179]:

```
df.head()
```

Out[179]:

| | url | address | name | online_order | book_table | rate | votes | phon |
|---|---|---|---|---|---|---|---|---|
| **0** | **https://www.zomato.com/bangalore/jalsa-banasha...** | **942, 21st Main Road, 2nd Stage, Banashankari, ...** | **Jalsa** | **Yes** | **Yes** | **4.1** | **775** | **080422975** |
| **1** | **https://www.zomato.com/bangalore/spice-elephan...** | **2nd Floor, 80 Feet Road, Near Big Bazaar, 6th ...** | **Spice Elephant** | **Yes** | **No** | **4.1** | **787** | **080417141** |
| **2** | **https://www.zomato.com/SanchurroBangalore?cont...** | **1112, Next to KIMS Medical College, 17th Cross...** | **San Churro Cafe** | **Yes** | **No** | **3.8** | **918** | **91966348795** |
| **3** | **https://www.zomato.com/bangalore/addhuri-udupi...** | **1st Floor, Annakuteera, 3rd Stage,** | **Addhuri Udupi Bhojana** | **No** | **No** | **3.7** | **88** | **919620009** |

| | url | address | name | online_order | book_table | rate | votes | phor |
|---|---|---|---|---|---|---|---|---|
| 4 | https://www.zomato.com/bangalore/grand-village... | 10, 3rd Floor, Lakshmi Associates, Gandhi Baza... | Grand Village | No | No | 3.8 | 166 | 9180266124 |

In [ ]:

In [ ]:

In [180]:

```
df.dtypes
```

Out[180]:

```
url                             object
address                         object
name                            object
online_order                    object
book_table                      object
rate                            object
votes                            int64
phone                           object
location                        object
rest_type                       object
dish_liked                      object
cuisines                        object
approx_cost(for two people)     object
reviews_list                    object
menu_item                       object
listed_in(type)                 object
dtype: object
```

In [181]:

```
df["rate"] = pd.to_numeric(df["rate"], errors='coerce')
df["phone"] = df["phone"].astype(int)
# Remove commas and convert to int
df["approx_cost(for two people)"] = df["approx_cost(for two people)"].str.replace(",", "
").astype(int)
df['dish_liked'] = df['dish_liked'].apply(lambda x: x.split(',')[0] if x !='Unknown' els
e 'Unknown')
df['cuisines'] = df['cuisines'].apply(lambda x: x.split(',')[0] if x !='Unknown' else 'U
nknown')
df['rest_type'] = df['rest_type'].apply(lambda x: x.split(',')[0] if x !='Unknown' else
'Unknown')
```

In [182]:

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 50279 entries, 0 to 51716
Data columns (total 16 columns):
 #   Column                       Non-Null Count  Dtype
---  ------                       --------------  -----
 0   url                          50279 non-null  object
 1   address                      50279 non-null  object
 2   name                         50279 non-null  object
 3   online_order                 50279 non-null  object
 4   book_table                   50279 non-null  object
 5   rate                         48022 non-null  float64
 6   votes                        50279 non-null  int64
```

```
 6   votes                        50279 non-null  int64
 7   phone                        50279 non-null  int64
 8   location                     50279 non-null  object
 9   rest_type                    50279 non-null  object
 10  dish_liked                   50279 non-null  object
 11  cuisines                     50279 non-null  object
 12  approx_cost(for two people)  50279 non-null  int64
 13  reviews_list                 50279 non-null  object
 14  menu_item                    50279 non-null  object
 15  listed_in(type)              50279 non-null  object
dtypes: float64(1), int64(3), object(12)
memory usage: 6.5+ MB
```

In [ ]:

```
```

# EDA (Exploratory Data Analysis) QUESTIONS:

In [ ]:

```
```

In [183]:

```
#Places from where we are getting most orders:
df["location"].value_counts().reset_index(name="No. of orders").head(5)
```

Out[183]:

| | location | No. of orders |
|---|---|---|
| 0 | BTM | 4981 |
| 1 | HSR | 2479 |
| 2 | Koramangala 5th Block | 2446 |
| 3 | JP Nagar | 2200 |
| 4 | Whitefield | 2079 |

In [184]:

```
#Places from where we are getting low orders:
df["location"].value_counts().reset_index(name="No. of orders").sort_values(by="No. of or
ders",ascending=True).head(5)
```

Out[184]:

| | location | No. of orders |
|---|---|---|
| 92 | Peenya | 1 |
| 91 | Rajarajeshwari Nagar | 2 |
| 90 | Jakkur | 3 |
| 89 | Yelahanka | 5 |
| 87 | West Bangalore | 6 |

In [185]:

```
import matplotlib.pyplot as plt
import seaborn as sns
```

In [186]:

```
df.head()
```

Out[186]:

Out[186]:

| | url | address | name | online_order | book_table | rate | votes | phor |
|---|---|---|---|---|---|---|---|---|
| 0 | https://www.zomato.com/bangalore/jalsa-banasha... | 942, 21st Main Road, 2nd Stage, Banashankari, ... | Jalsa | Yes | Yes | 4.1 | 775 | 804229755 |
| 1 | https://www.zomato.com/bangalore/spice-elephan... | 2nd Floor, 80 Feet Road, Near Big Bazaar, 6th ... | Spice Elephant | Yes | No | 4.1 | 787 | 804171416 |
| 2 | https://www.zomato.com/SanchurroBangalore?cont... | 1112, Next to KIMS Medical College, 17th Cross... | San Churro Cafe | Yes | No | 3.8 | 918 | 919663487 |
| 3 | https://www.zomato.com/bangalore/addhuri-udupi... | 1st Floor, Annakuteera, 3rd Stage, Banashankar... | Addhuri Udupi Bhojana | No | No | 3.7 | 88 | 919620009 |
| 4 | https://www.zomato.com/bangalore/grand-village... | 10, 3rd Floor, Lakshmi Associates, Gandhi Baza... | Grand Village | No | No | 3.8 | 166 | 918026612 |

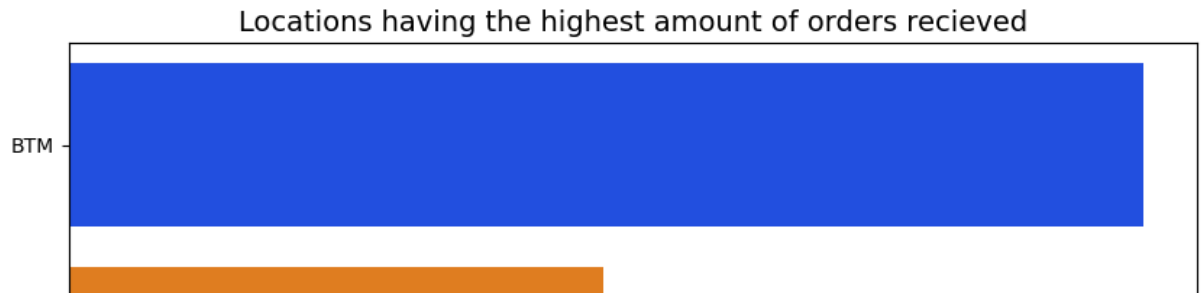
In [ ]:

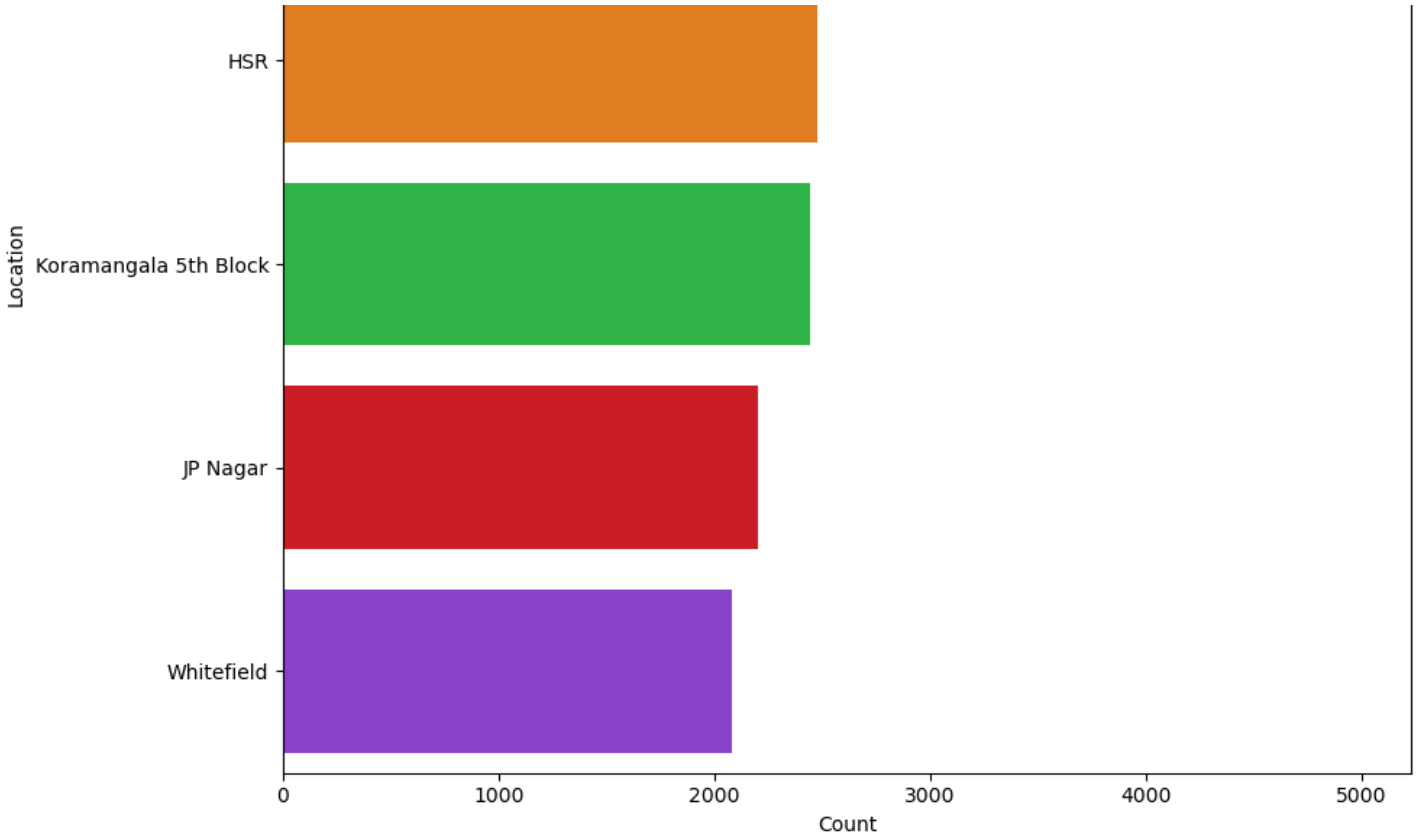
```
```

## EDA QUESTION 1] Show the locations from where most orders were made:

In [187]:

```
plt.figure(figsize=(10,8))
data = df["location"].value_counts().reset_index(name="No. of orders").head(5)
sns.barplot(
    x="No. of orders",
    y="location",
    data=data,
    palette="bright"
)

plt.title("Locations having the highest amount of orders recieved", fontsize=14)
plt.xlabel("Count")
plt.ylabel("Location")

plt.tight_layout()
plt.show()
```

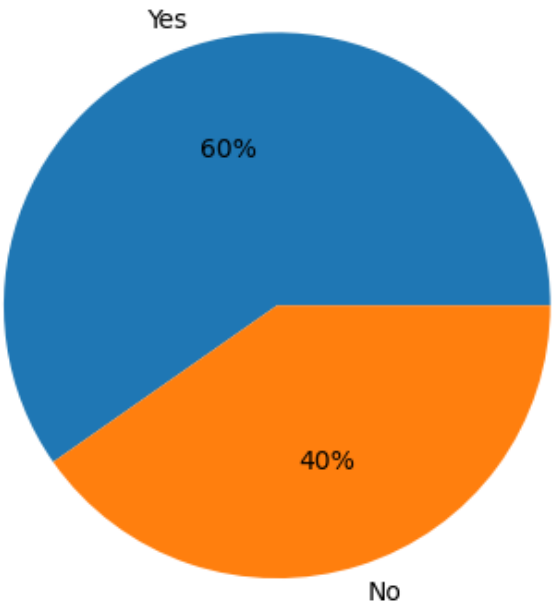





## EDA QUESTION 2] Show how many people placed their orders online and how many not:

In [188]:

```
counts = df["online_order"].value_counts()

plt.pie(counts, labels=counts.index, autopct='%1.0f%%')
plt.show()
```

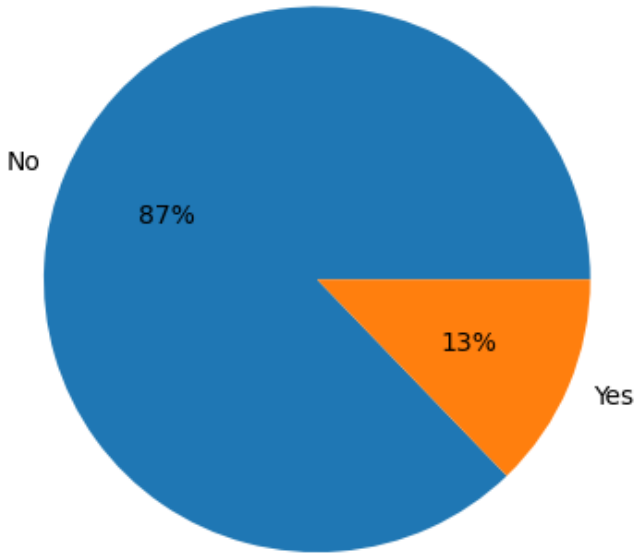



## EDA QUESTION 3] Show how many people booked a table and how many not:

In [189]:

```
counts = df["book_table"].value_counts()
```

```
plt.pie(counts, labels=counts.index, autopct='%1.0f%%')
plt.show()
```




In [ ]:

```

```

## EDA QUESTION 4] Show top 5 cusines:

In [190]:

```
plt.figure(figsize=(10,8))
data=df.value_counts("cuisines").reset_index().head(5)
data
```

Out[190]:

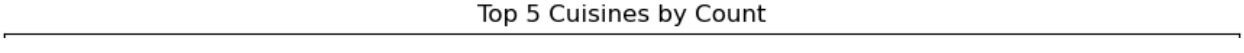
| | cuisines | count |
|---|---|---|
| 0 | North Indian | 11957 |
| 1 | South Indian | 4840 |
| 2 | Cafe | 4204 |
| 3 | Chinese | 3029 |
| 4 | Biryani | 2975 |

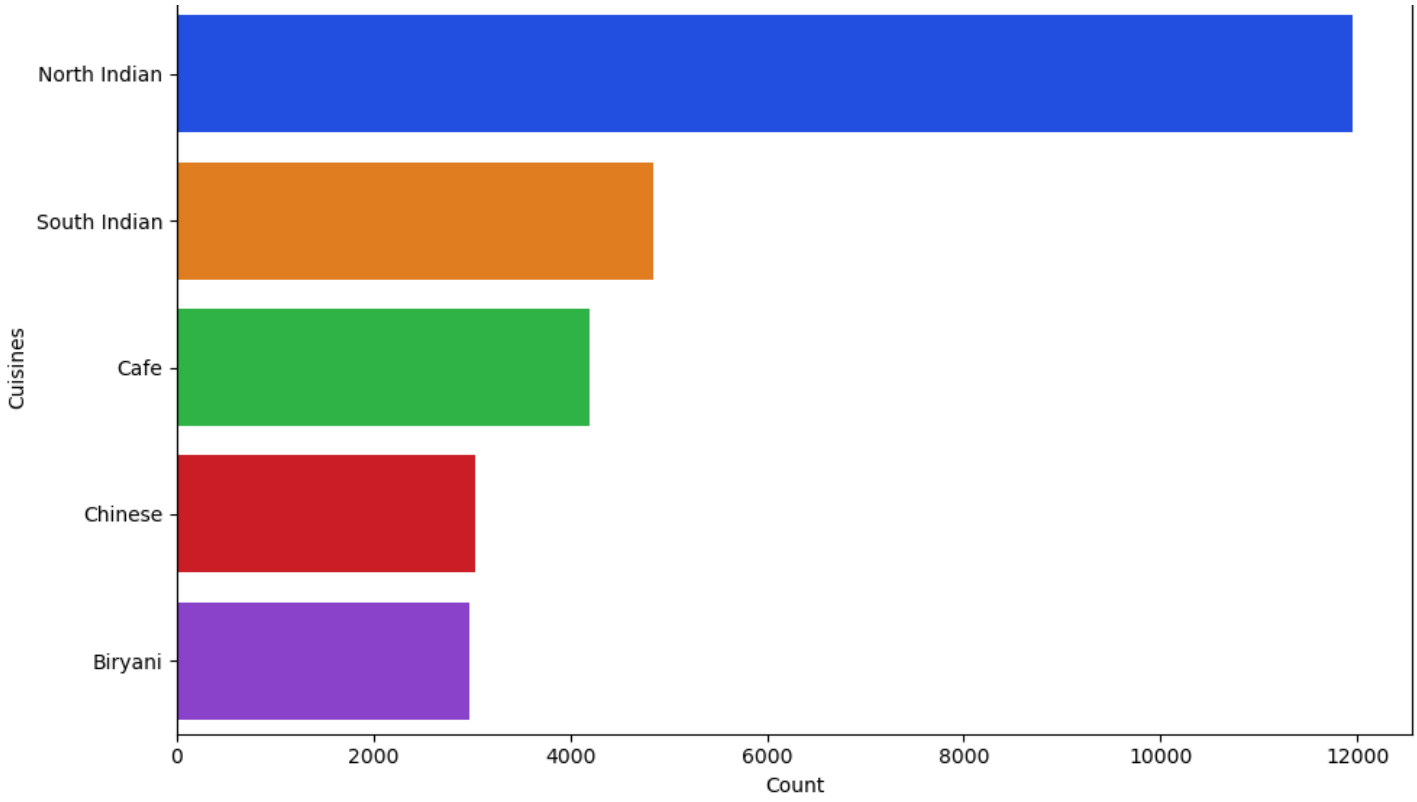
```
<Figure size 1000x800 with 0 Axes>
```

### *visualize the above result:*

In [191]:

```
plt.figure(figsize=(10, 6))
sns.barplot(y="cuisines", x="count", data=data.head(5), palette="bright")

plt.xlabel("Count")
plt.ylabel("Cuisines")
plt.title("Top 5 Cuisines by Count")

plt.tight_layout()
plt.show()
```



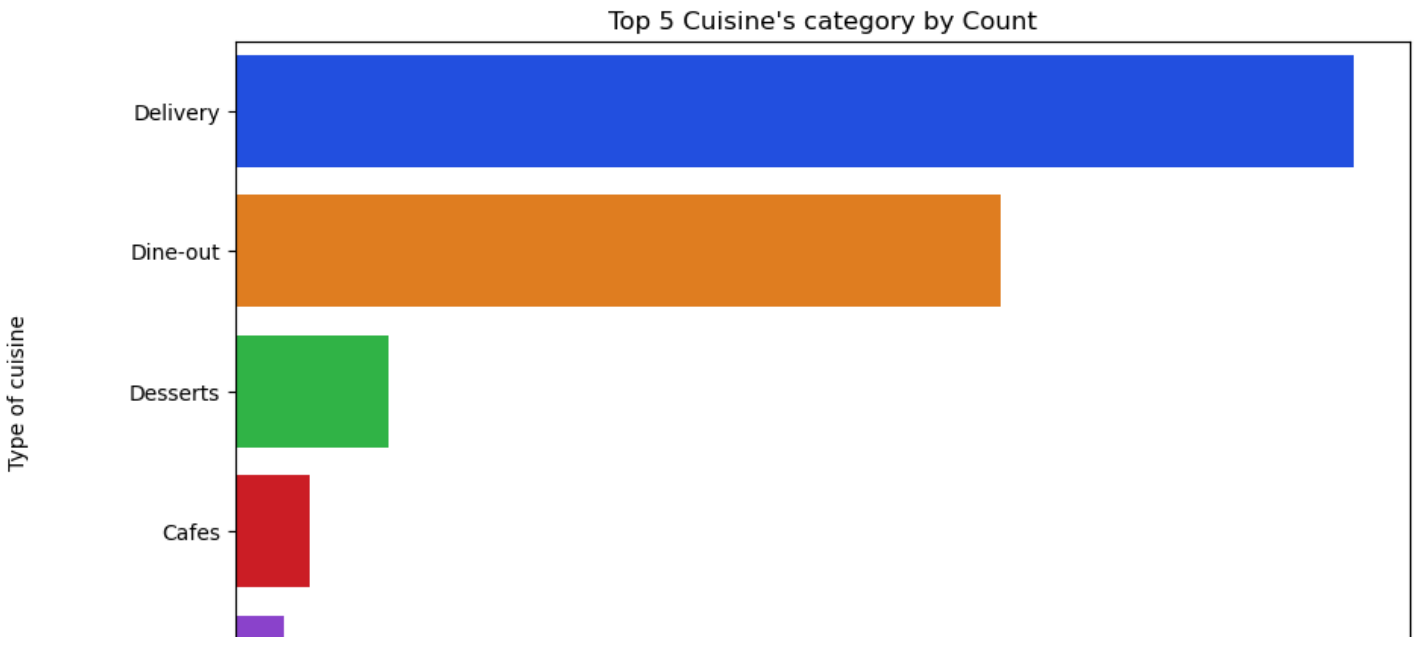


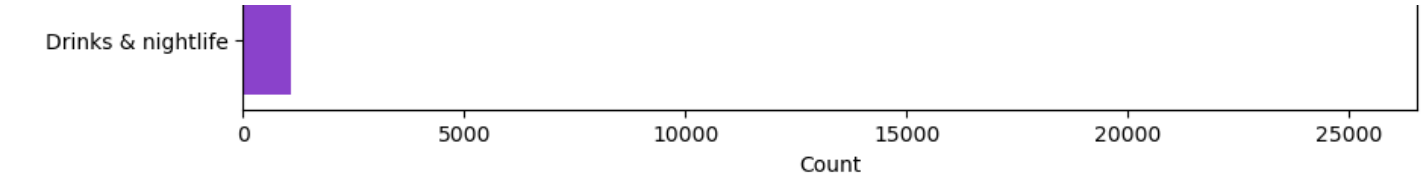

## EDA QUESTION 5] Show top 5 cusines' category:

In [192]:

```
plt.figure(figsize=(10,8))
data = df.value_counts("listed_in(type)").reset_index()

plt.figure(figsize=(10, 6))
sns.barplot(
    y="listed_in(type)",
    x="count",
    data=data.head(5),
    palette="bright"
)

plt.xlabel("Count")
plt.ylabel("Type of cuisine")
plt.title("Top 5 Cuisine's category by Count")

plt.show()
```

```
<Figure size 1000x800 with 0 Axes>
```



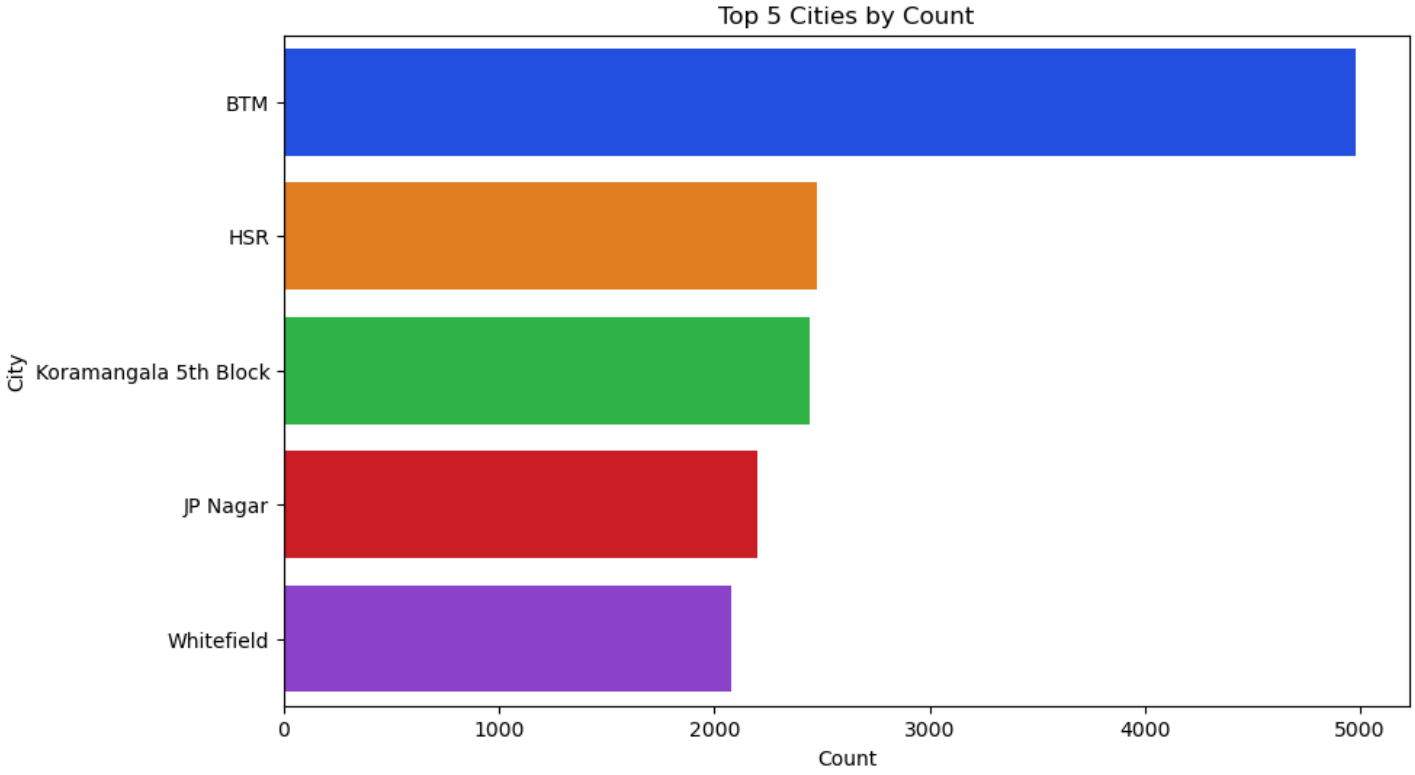



## EDA QUESTION 6] Show the cities having the most placed orders:

In [193]:

```
plt.figure(figsize=(10,8))
data = df.value_counts("location").reset_index()

plt.figure(figsize=(10, 6))
sns.barplot(
    y="location",
    x="count",
    data=data.head(5),
    palette="bright"
)

plt.xlabel("Count")
plt.ylabel("City")
plt.title("Top 5 Cities by Count")

plt.show()
```

```
<Figure size 1000x800 with 0 Axes>
```



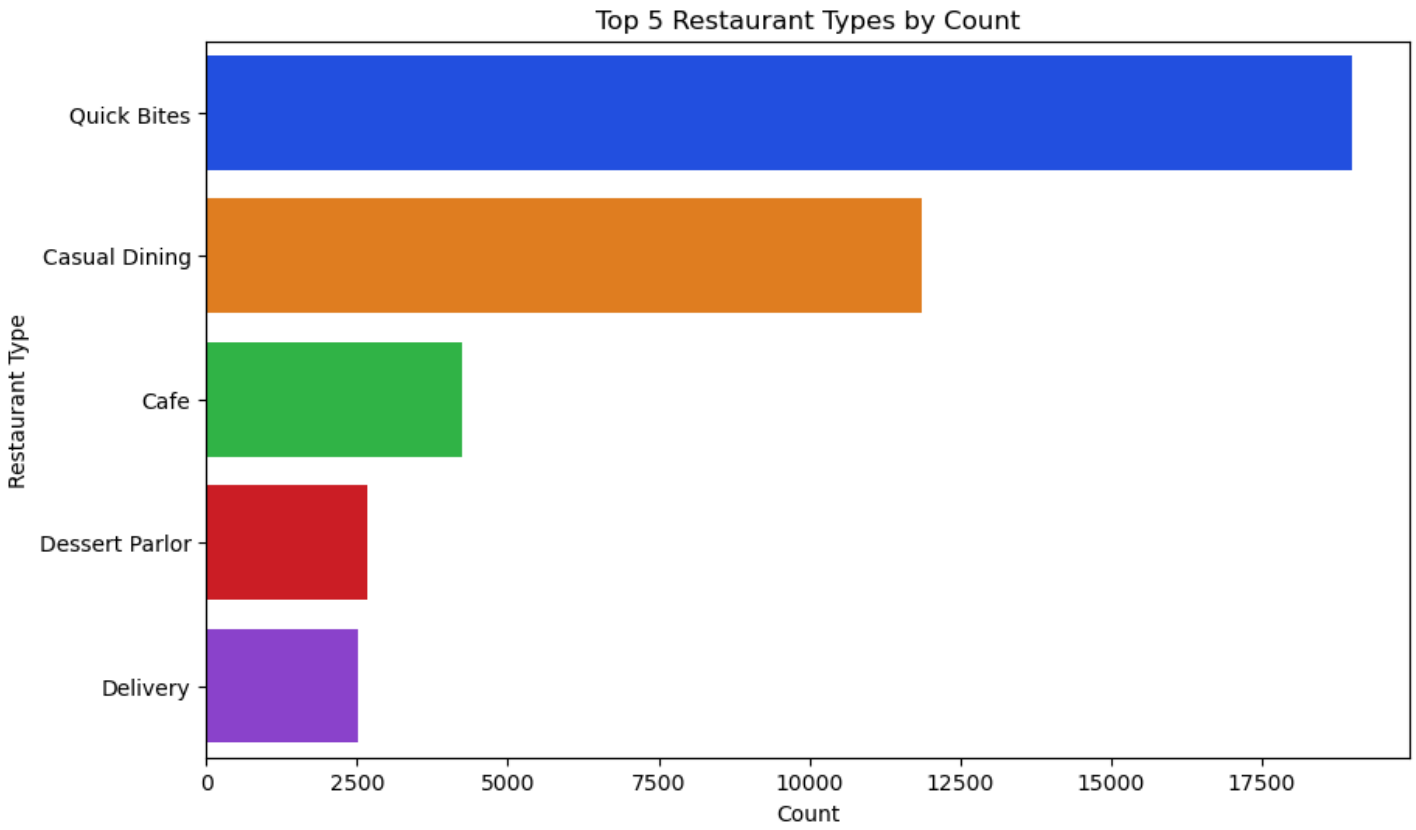

## EDA QUESTION 7] Show the top 5 restaurant-types preferred:

In [194]:

```
plt.figure(figsize=(10,8))
data = df.value_counts("rest_type").reset_index()

plt.figure(figsize=(10, 6))
sns.barplot(
    y="rest_type",
    x="count",
    data=data.head(5),
```

```
    palette="bright"
)

plt.xlabel("Count")
plt.ylabel("Restaurant Type")
plt.title("Top 5 Restaurant Types by Count")

plt.show()
```

```
<Figure size 1000x800 with 0 Axes>
```




In [195]:

```
df.isnull().sum()
```

Out[195]:

```
url                            0
address                        0
name                           0
online_order                   0
book_table                     0
rate                        2257
votes                          0
phone                          0
location                       0
rest_type                      0
dish_liked                     0
cuisines                       0
approx_cost(for two people)    0
reviews_list                   0
menu_item                      0
listed_in(type)                0
dtype: int64
```

In [196]:

```
df["rate"] = df["rate"].fillna(0.0)
```

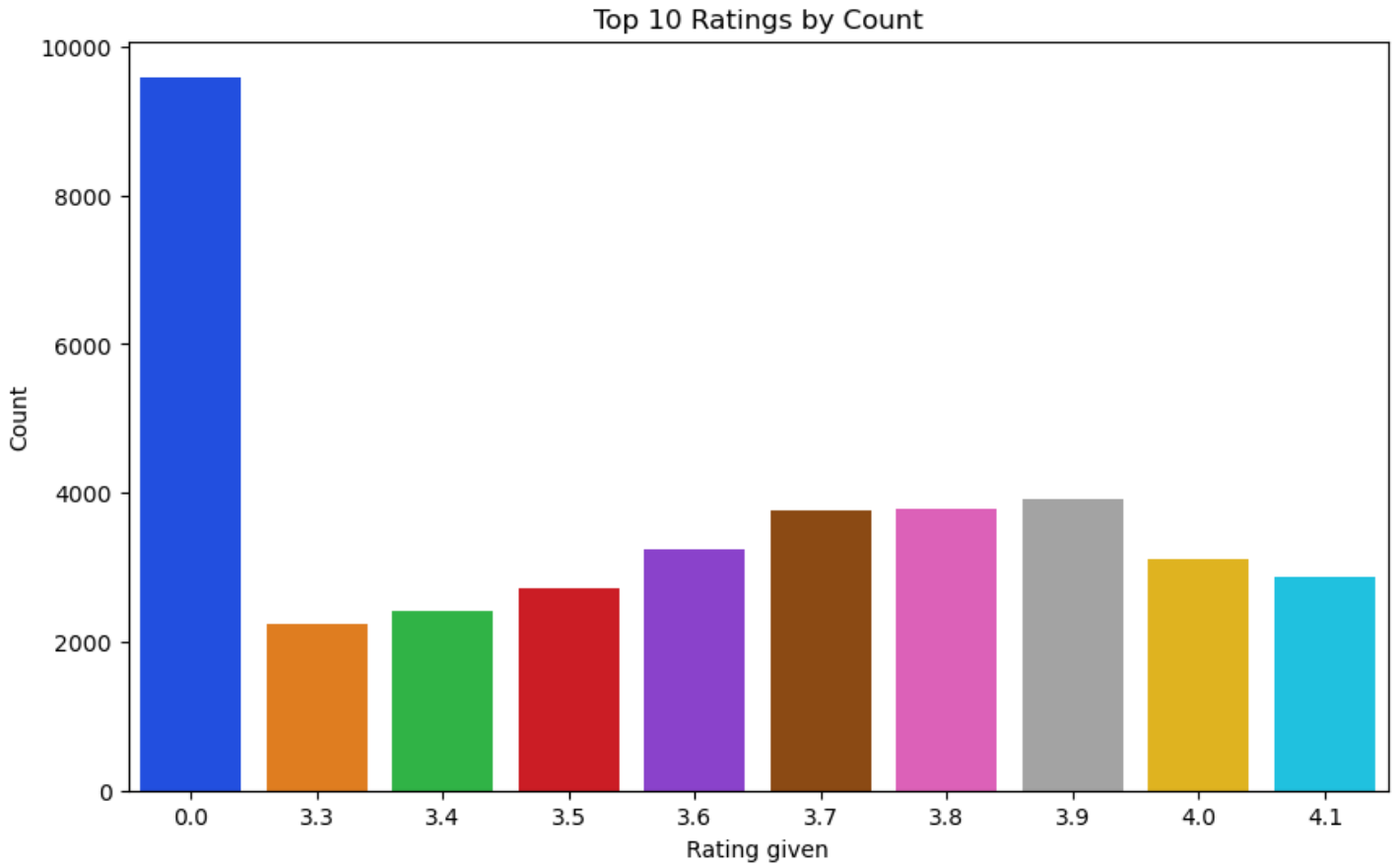
In [197]:

```
data = df["rate"].value_counts().reset_index().head()
```

## EDA QUESTION 8] Show the most often ratings given:

In [198]:

```
data = df.value_counts("rate").reset_index()

plt.figure(figsize=(10, 6))
sns.barplot(
    x="rate",
    y="count",
    data=data.head(10),
    palette="bright"
)

plt.xlabel("Rating given")
plt.ylabel("Count")
plt.title("Top 10 Ratings by Count")

plt.show()
```




In [199]:

```
data = df["location"].value_counts().reset_index()
data = data.head()
data
```

Out[199]:

| | location | count |
|---|---|---|
| 0 | BTM | 4981 |
| 1 | HSR | 2479 |
| 2 | Koramangala 5th Block | 2446 |
| 3 | JP Nagar | 2200 |
| 4 | Whitefield | 2079 |

## EDA QUESTION 9] Show the top 5 locations on the map:

# EDA QUESTION 9] Show the top-5 locations on the map:

In [200]:

```
import folium
# Group by location to get restaurant counts and sort for top 10
location_counts = df.groupby('location').size().sort_values(ascending=False).head(10)

# Dictionary of coordinates for top locations (obtained via web search or geocoding)
# Note: In practice, use geopy or a CSV for accurate coords; these are approximate
coords = {
    'BTM': (12.916576, 77.610116),
    'HSR': (12.908136, 77.647608),
    'Koramangala 5th Block': (12.934533, 77.626579),  # Approximate for 5th Block
    'JP Nagar': (12.906000, 77.585000),
    'Whitefield': (12.971389, 77.750130)
}

# Create Folium map centered on Bengaluru (approx. center: 12.97, 77.59)
m = folium.Map(location=[12.97, 77.59], zoom_start=12)

# Add markers for top locations (only if in coords dict)
for location, count in location_counts.items():
    if location in coords:
        folium.Marker(
            location=coords[location],
            popup=f"{location}: {count} restaurants",
            tooltip=location
        ).add_to(m)

display(m)
```

**Make this Notebook Trusted to load map: File -> Trust Notebook**

In [201]:

```
df.head()
```

Out[201]:

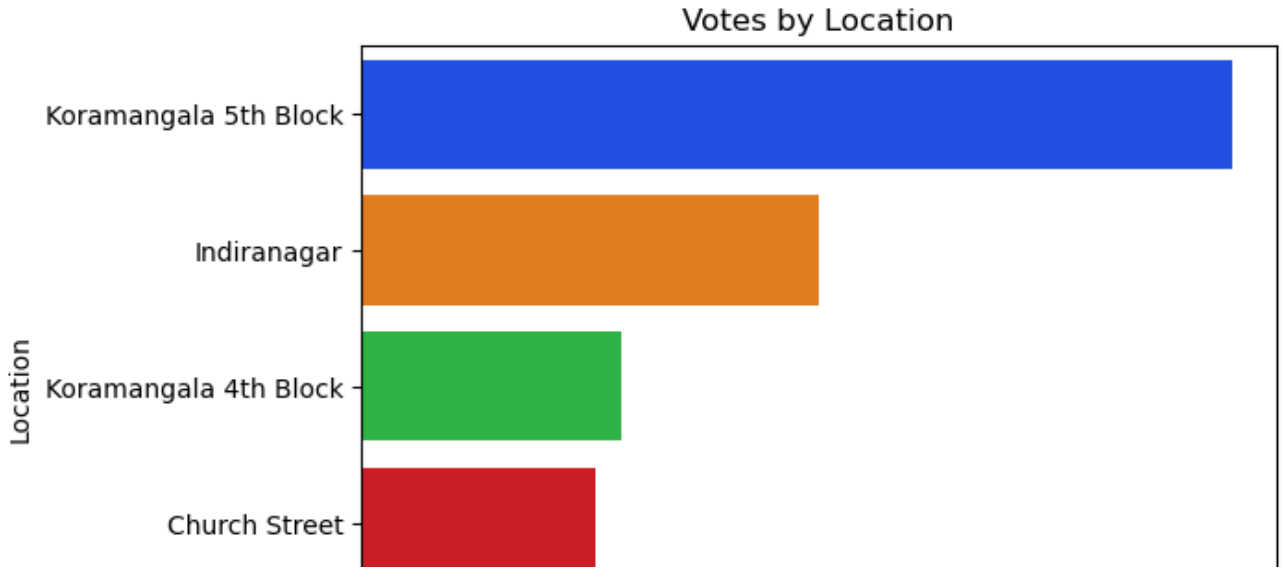
| | url | address | name | online_order | book_table | rate | votes | phor |
|---|---|---|---|---|---|---|---|---|
| | https://www.zomato.com/bangalore/jalsa- | 942, 21st Main Road, 2nd | | | | | | |

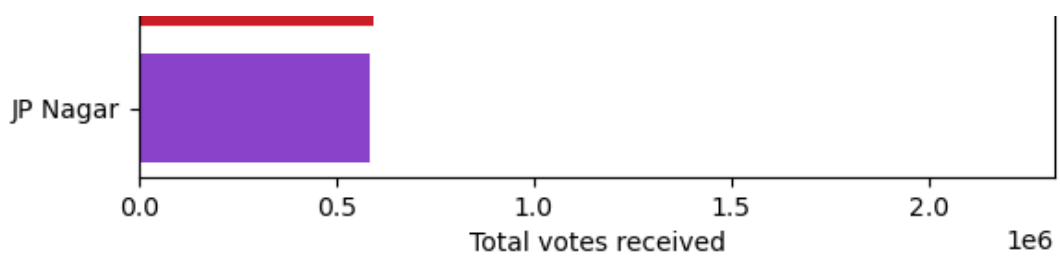
| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| 0 | banasha... | Stage, Banashankari ... | Jalsa | Yes | Yes | 4.1 | 775 | 804229755 |
| 1 | https://www.zomato.com/bangalore/spice-elephan... | 2nd Floor, 80 Feet Road, Near Big Bazaar, 6th ... | Spice Elephant | Yes | No | 4.1 | 787 | 804171416 |
| 2 | https://www.zomato.com/SanchurroBangalore?cont... | 1112, Next to KIMS Medical College, 17th Cross... | San Churro Cafe | Yes | No | 3.8 | 918 | 91966348795 |
| 3 | https://www.zomato.com/bangalore/addhuri-udupi... | 1st Floor, Annakuteera, 3rd Stage, Banashankar... | Addhuri Udupi Bhojana | No | No | 3.7 | 88 | 91962000930 |
| 4 | https://www.zomato.com/bangalore/grand-village... | 10, 3rd Floor, Lakshmi Associates, Gandhi Baza... | Grand Village | No | No | 3.8 | 166 | 91802661244 |

## EDA QUESTION 10] Show the locations from where majority of restaurant's ratings were given:

In [202]:

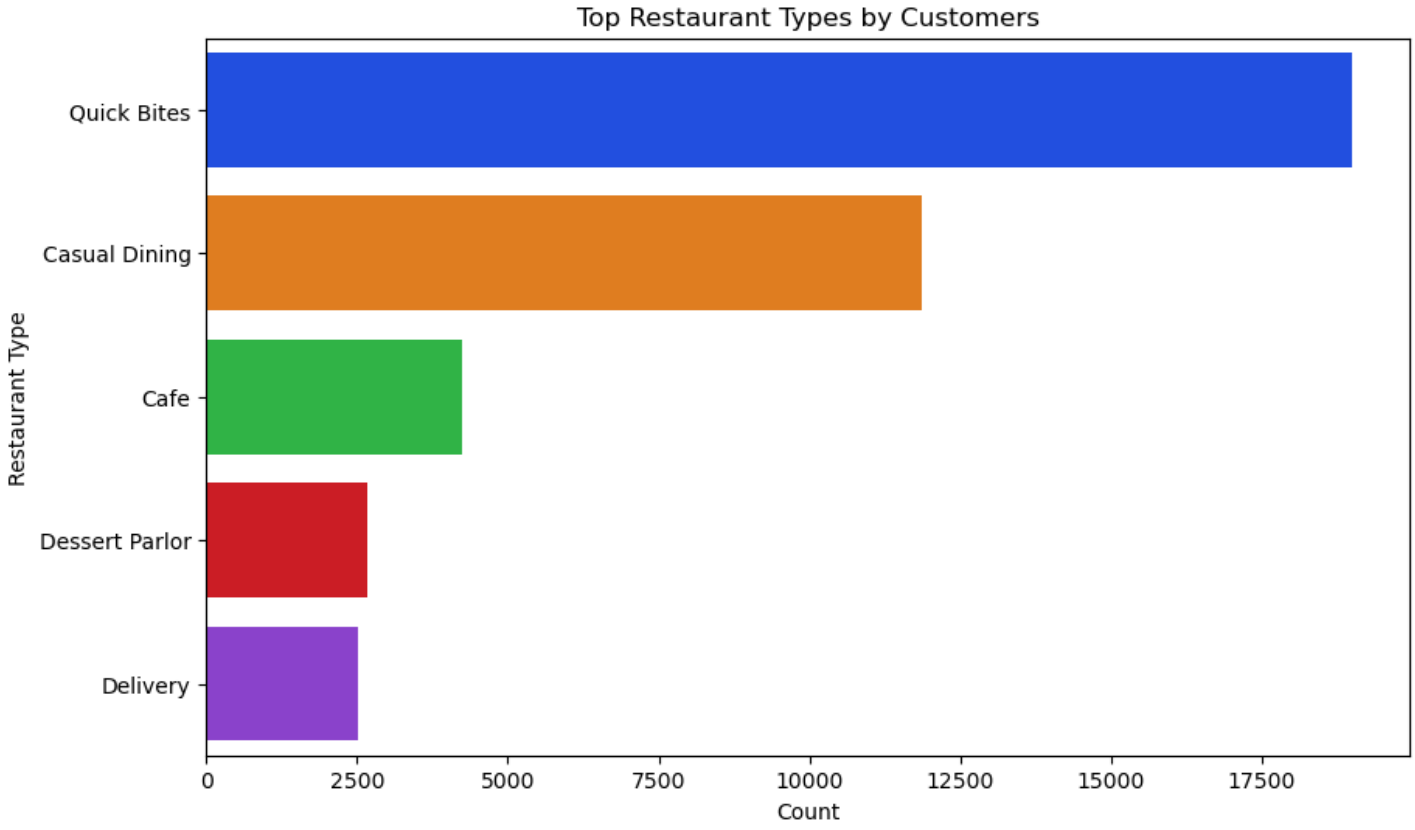
```
data = df.groupby("location")["votes"].sum().reset_index().sort_values(by="votes",ascending=False)

sns.barplot(
    y="location",
    x="votes",
    data=data.head(),
    palette="bright"
)

plt.xlabel("Total votes received")
plt.ylabel("Location")
plt.title("Votes by Location")


plt.show()
```






In [203]:

```
data = df["rest_type"].value_counts().reset_index(name="No. of customers")

plt.figure(figsize=(10, 6))
sns.barplot(
    y="rest_type",
    x="No. of customers",
    data=data.head(),
    palette="bright"
)

plt.ylabel("Restaurant Type")
plt.xlabel("Count")
plt.title("Top Restaurant Types by Customers")


plt.show()
```

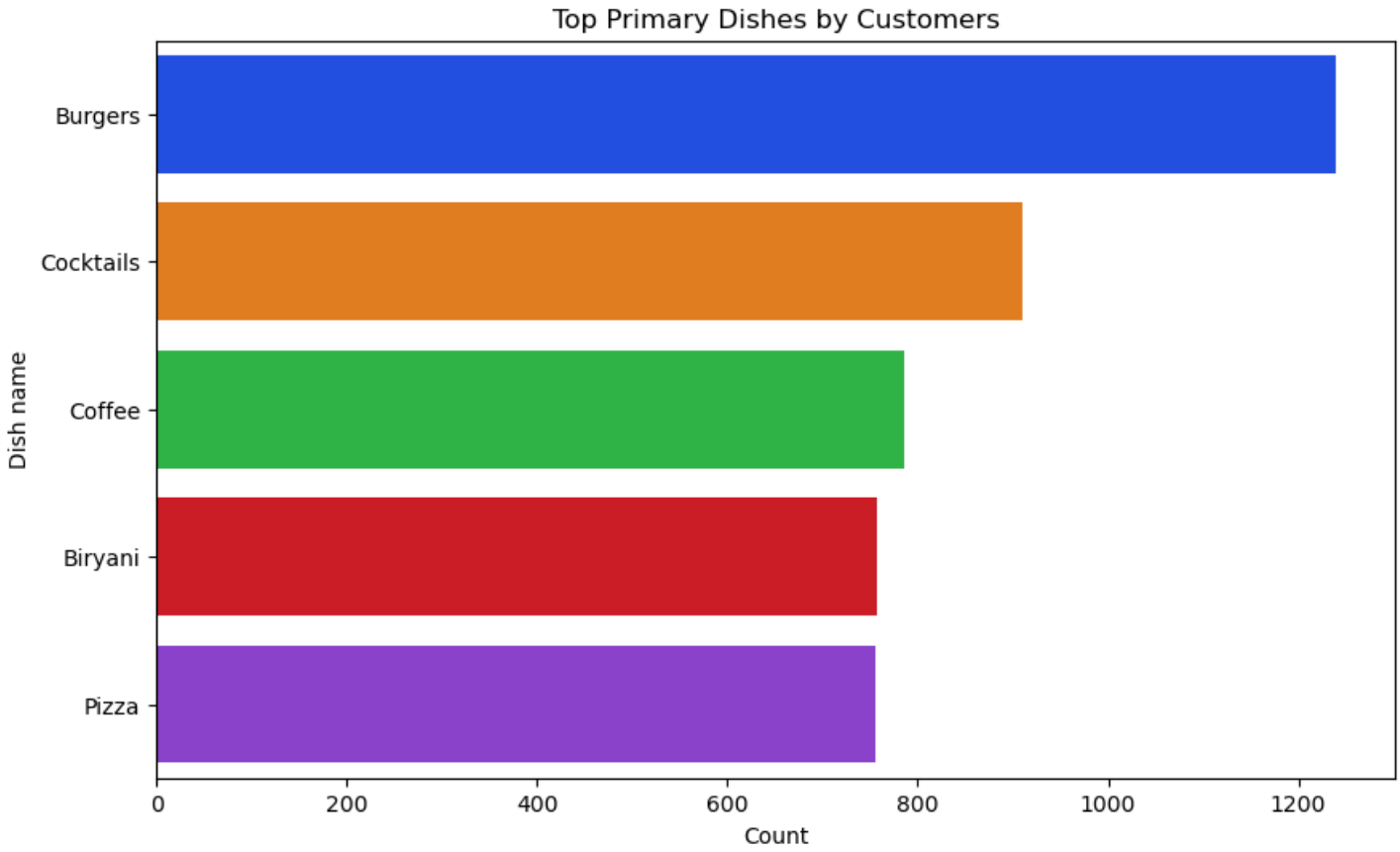



## EDA QUESTION 11] Show the dishes most customers preferred:

In [204]:

```
data = df["dish_liked"].value_counts().reset_index(name="No. of customers")
data = data[data["dish_liked"] != "0/5"]

plt.figure(figsize=(10, 6))
sns.barplot(
    y="dish_liked",
    x="No. of customers",
    data=data.head(),
    palette="bright"
```

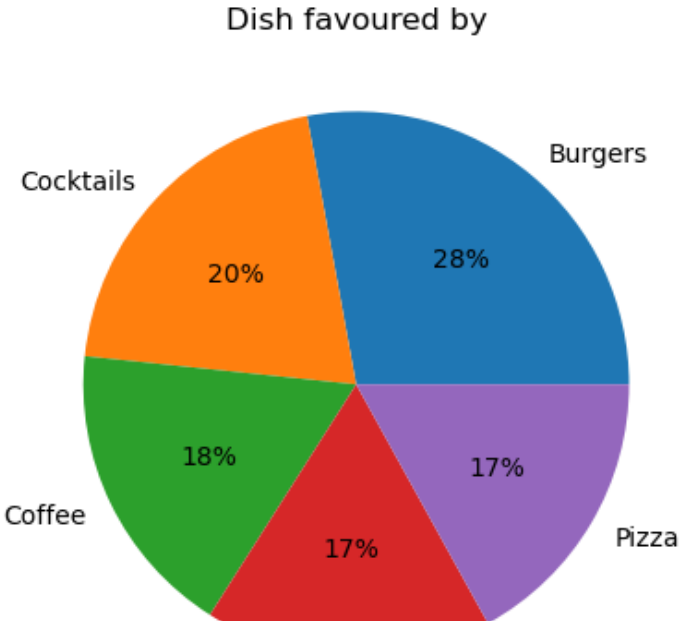
```
)

plt.ylabel("Dish name")
plt.xlabel("Count")
plt.title("Top Primary Dishes by Customers")

plt.show()
```




***visualize the above result in the form of pie-chart also:***

In [205]:

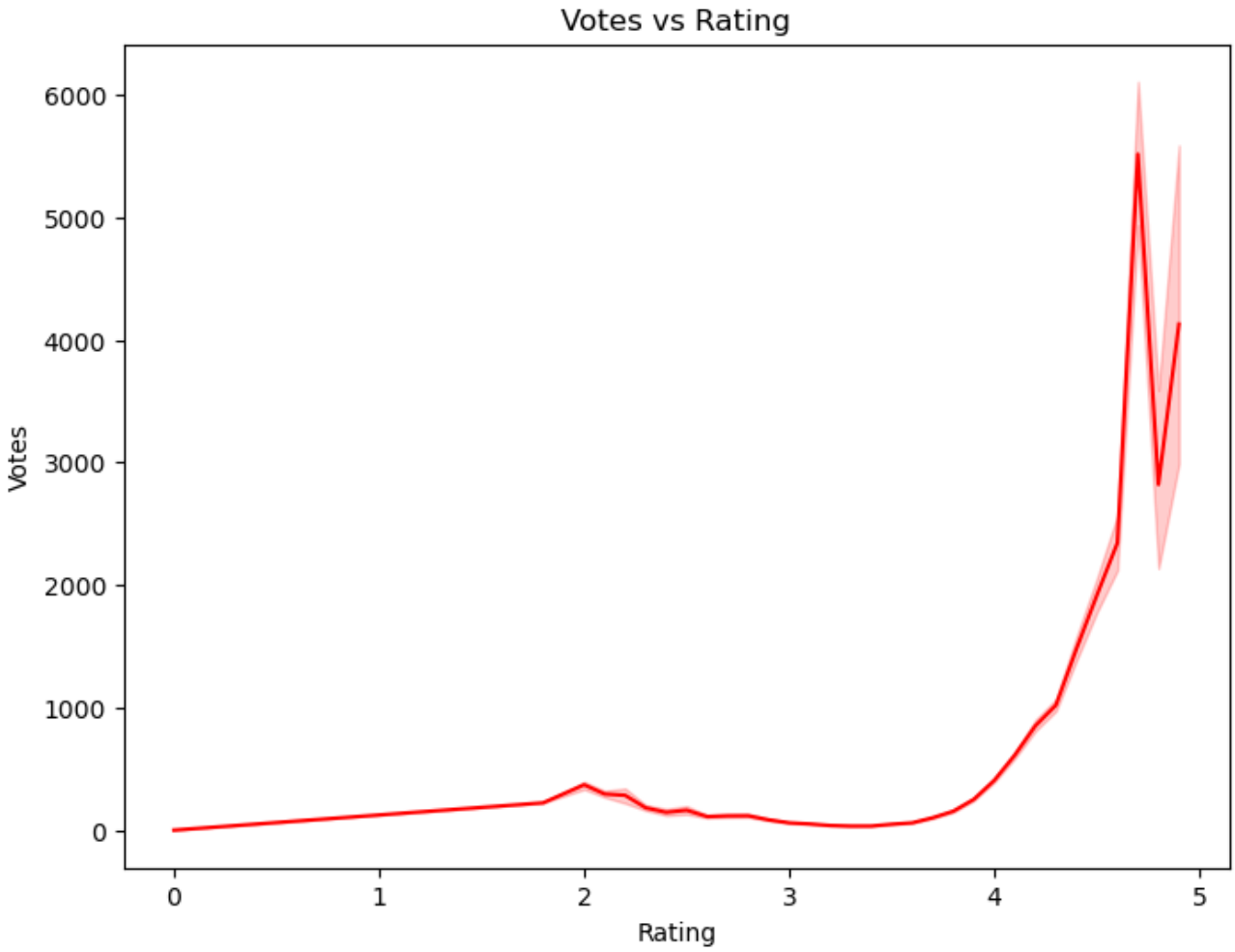
```
data = df["dish_liked"].value_counts().reset_index(name="No. of customers")
data = data[data["dish_liked"]!="0/5"]

data = data.head()

plt.pie(data["No. of customers"], labels=data["dish_liked"], autopct='%1.0f%%')
plt.title("Dish favoured by")
plt.show()
```




### EDA QUESTION 12] Relationship between votes and ratings:

In [206]:

```
plt.figure(figsize=(8,6))
sns.lineplot(x="rate", y="votes",data=df,color="red")
plt.title("Votes vs Rating")
plt.xlabel("Rating")
plt.ylabel("Votes")
plt.show()
```




In [ ]:

```

```

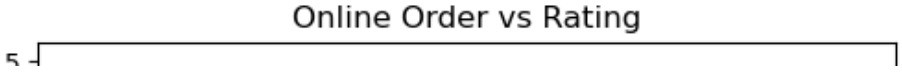
In [ ]:

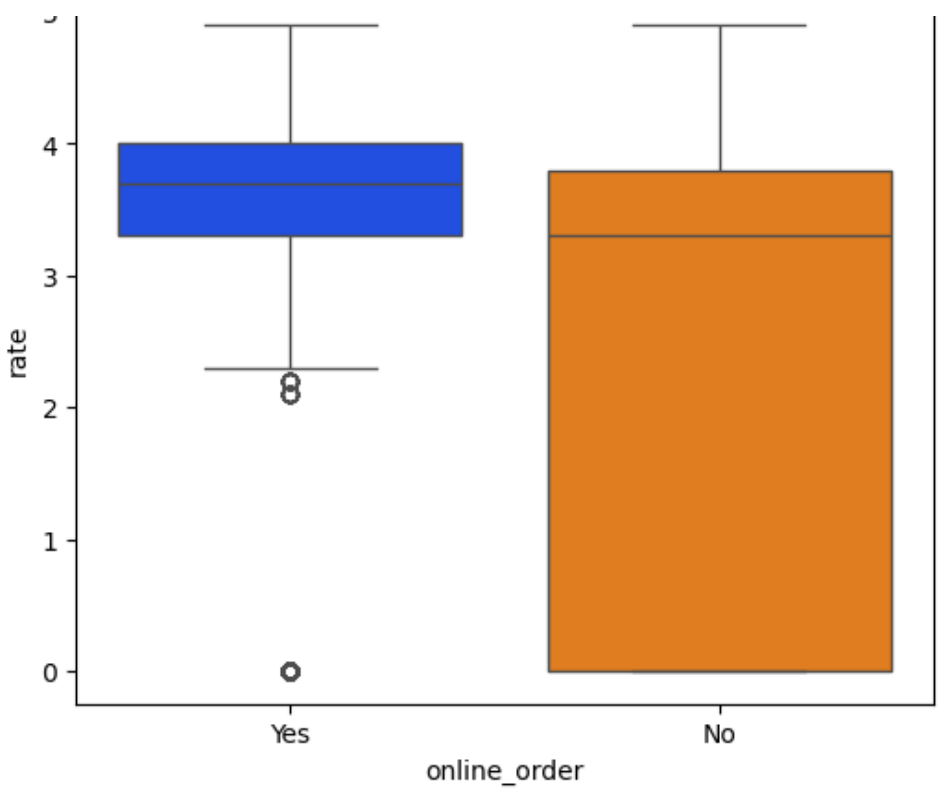
```

```

### EDA QUESTION 13] Distribution in Online-orders and restuarant-ratings:

In [207]:

```
plt.figure(figsize=(6,5))
sns.boxplot(x="online_order", y="rate", data=df,palette="bright")
plt.title("Online Order vs Rating")
plt.show()
```





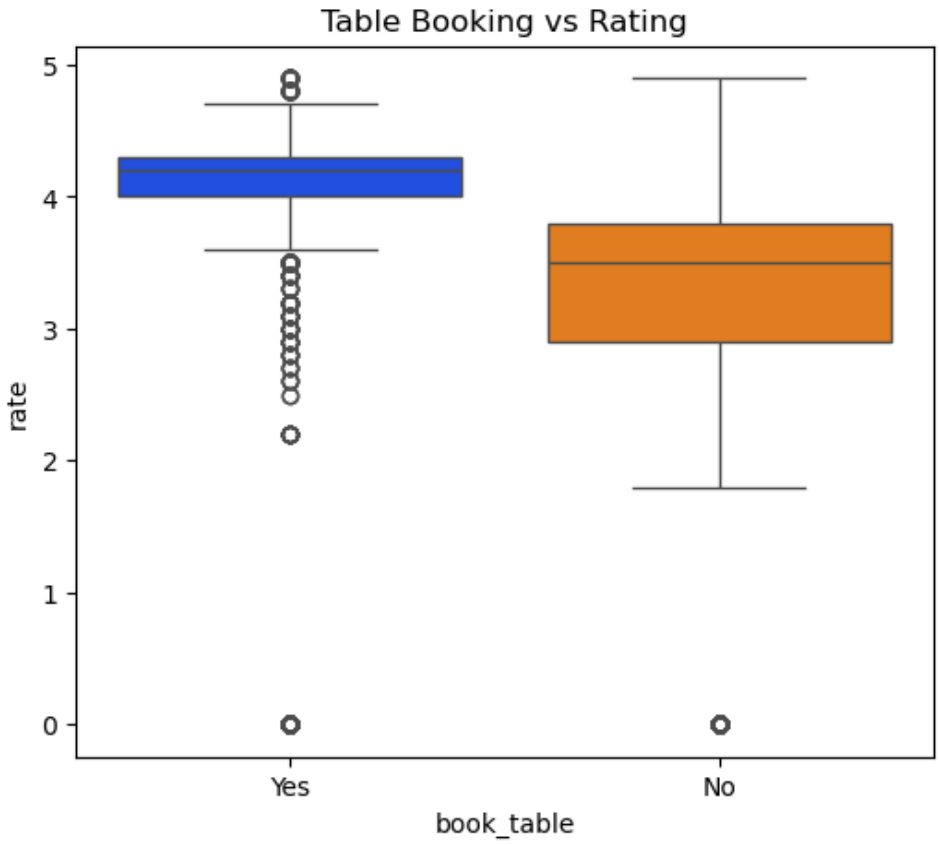

In [ ]:

```
```

**EDA QUESTION 14] Distribution in table-booking options and restuarant-ratings:**

In [208]:

```
plt.figure(figsize=(6,5))
sns.boxplot(x="book_table", y="rate", data=df,palette="bright")
plt.title("Table Booking vs Rating")
plt.show()
```

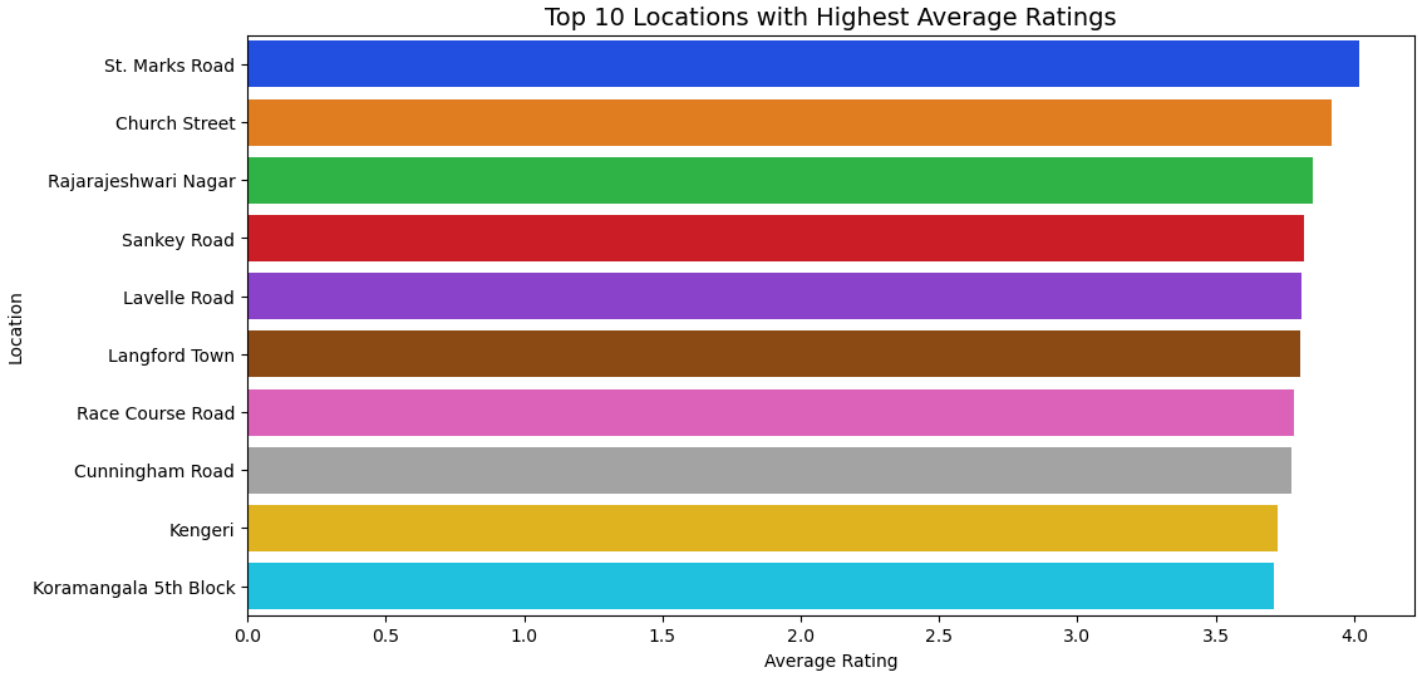



In [ ]:

## EDA QUESTION 15] Show the top-10 locations with Highest Ratings (on average):

In [209]:

```
location_ratings = (
    df.groupby("location")["rate"]
    .mean()
    .sort_values(ascending=False)
    .head(10)
)

plt.figure(figsize=(12,6))
sns.barplot(
    x=location_ratings.values,
    y=location_ratings.index,
    palette="bright"
)

plt.title("Top 10 Locations with Highest Average Ratings", fontsize=14)
plt.xlabel("Average Rating")
plt.ylabel("Location")

plt.show()
```




In [ ]:

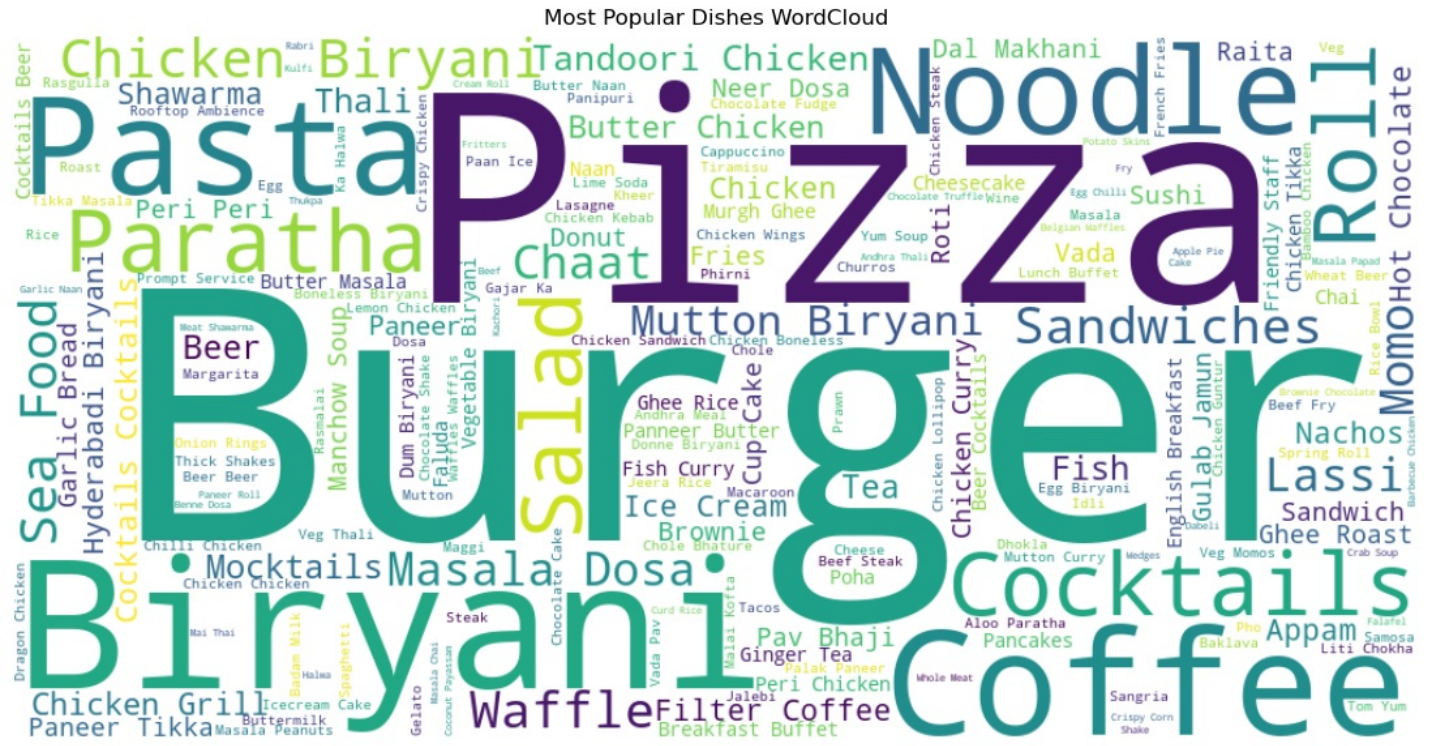
In [ ]:

## *A WORDCLOUD OF DISHES LIKED BY CUSTOMERS:*

In [210]:

```
from wordcloud import WordCloud

text = " ".join(df["dish_liked"].astype(str).tolist())
wordcloud = WordCloud(width=1000, height=500, background_color="white").generate(text)

plt.figure(figsize=(15,7))
plt.imshow(wordcloud, interpolation='bilinear')
```

```
plt.axis("off")
plt.title("Most Popular Dishes WordCloud")
plt.show()
```



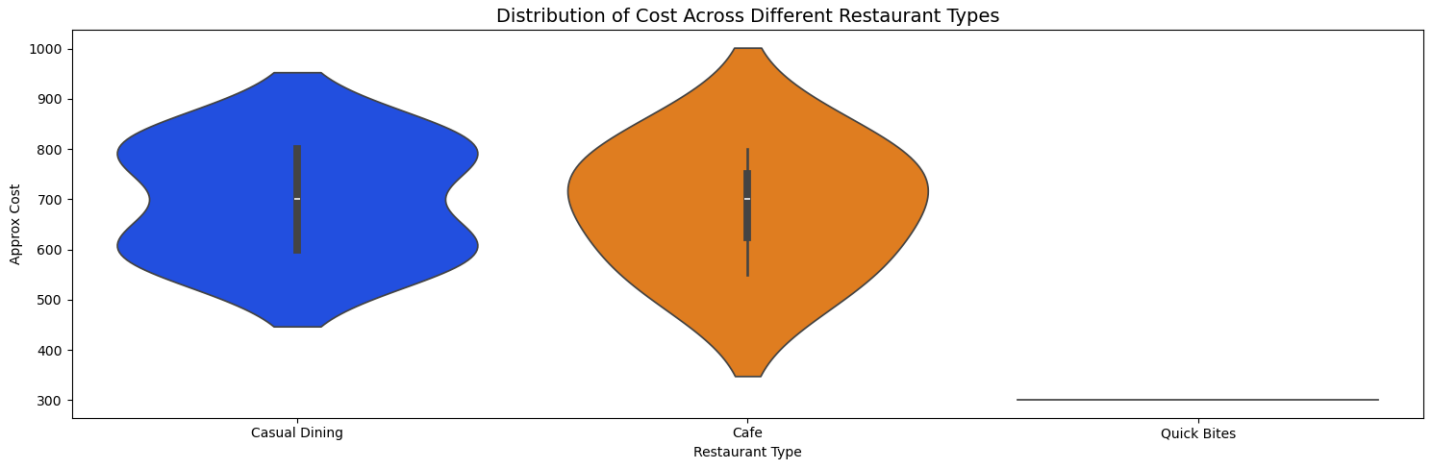

In [ ]:

```
```

## EDA QUESTION 16] Distribution of Cost Across different restaurant types:

In [211]:

```
plt.figure(figsize=(15,5))
sns.violinplot(x="rest_type",y="approx_cost(for two people)",data=df.head(10),palette="bright")
plt.title("Distribution of Cost Across Different Restaurant Types", fontsize=14)
plt.xlabel("Restaurant Type")
plt.ylabel("Approx Cost")
plt.tight_layout()
```



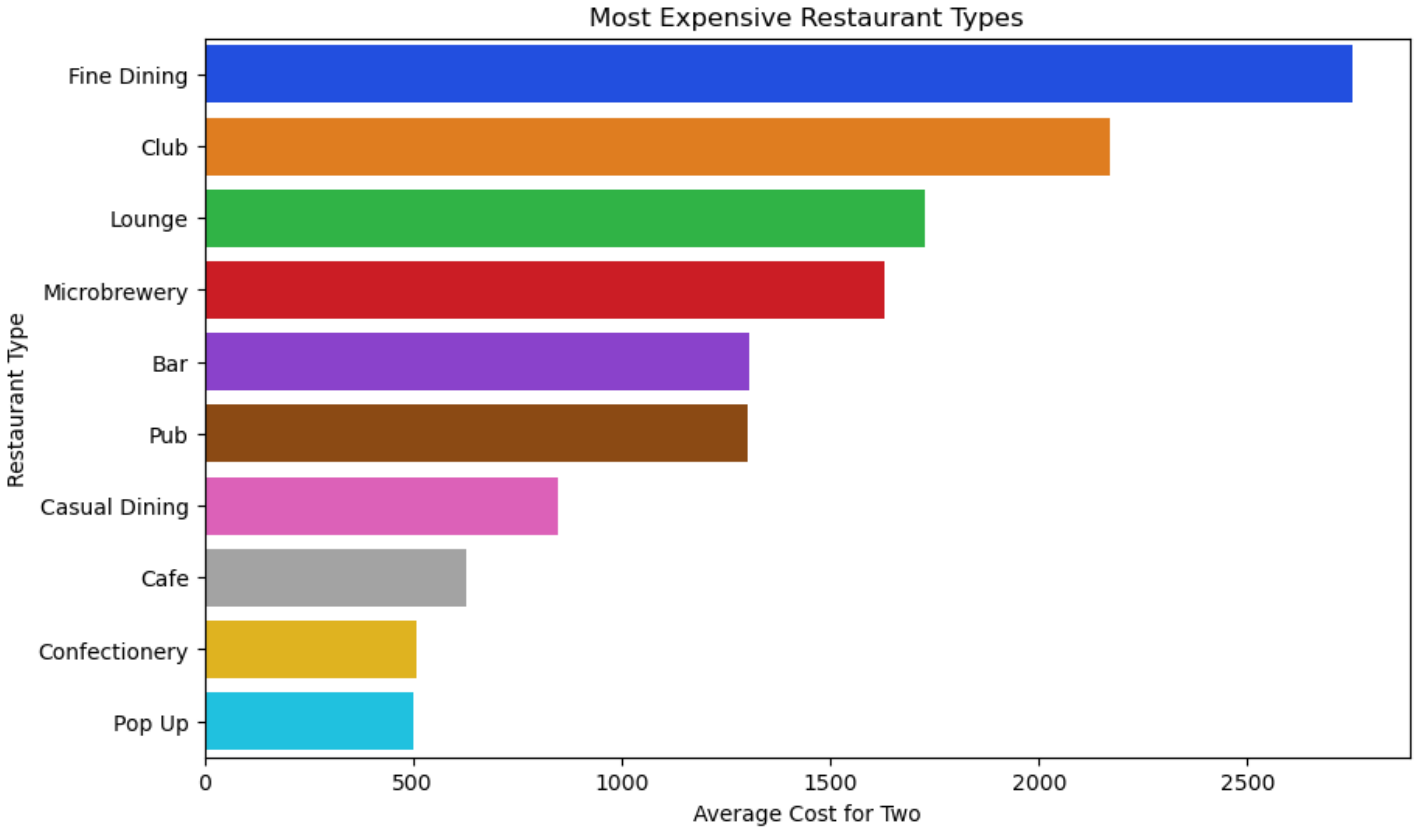

In [ ]:

```
```

## EDA QUESTION 17] Which restaurant types are the most expensive on average?

In [212]:

```
avg_cost_rest_type = df.groupby("rest_type")["approx_cost(for two people)"].mean().sort_
values(ascending=False).head(10)

plt.figure(figsize=(10,6))
sns.barplot(x=avg_cost_rest_type.values, y=avg_cost_rest_type.index,palette="bright")
plt.title("Most Expensive Restaurant Types")
plt.xlabel("Average Cost for Two")
plt.ylabel("Restaurant Type")
plt.show()
```



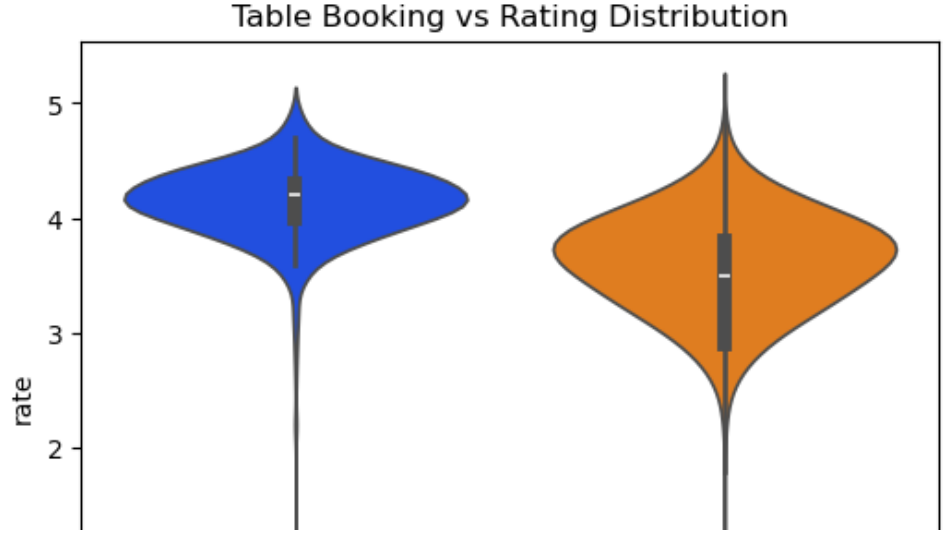

In [ ]:

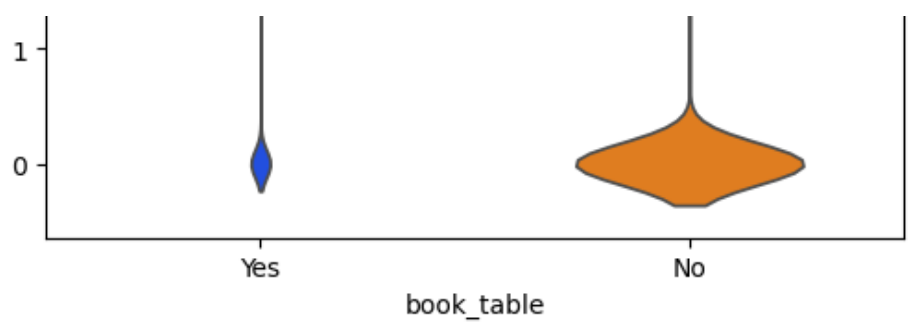
```
```

## EDA QUESTION 18] Rating distribution for restaurants with and without table booking

In [213]:

```
plt.figure(figsize=(6,5))
sns.violinplot(x="book_table", y="rate", data=df,palette="bright")
plt.title("Table Booking vs Rating Distribution")
plt.show()
```





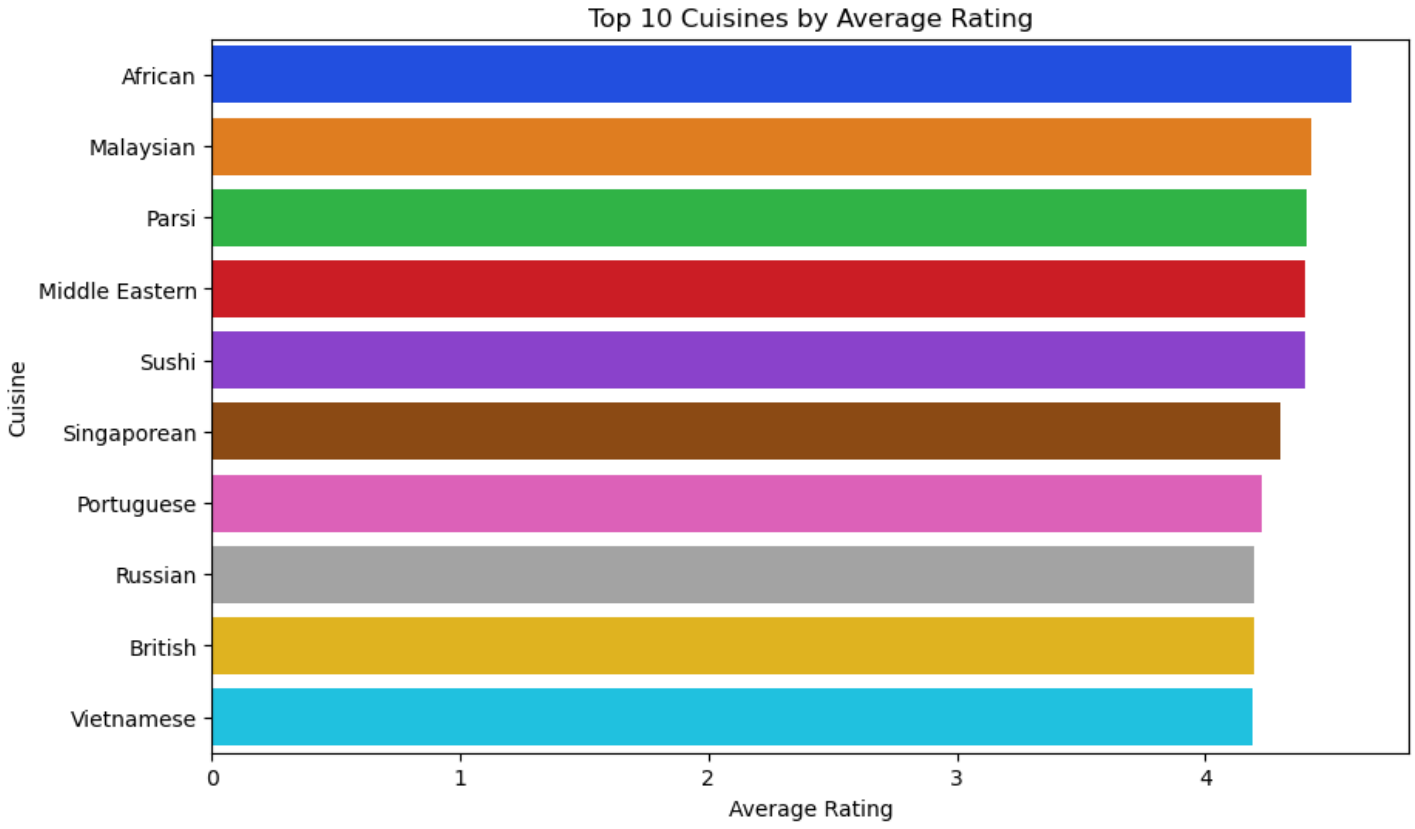

In [ ]:

```
```

## EDA QUESTION 19] Which cuisines have the highest average rating? (Top 10)

In [214]:

```
avg_rating_cuisine = df.groupby("cuisines")["rate"].mean().sort_values(ascending=False).
head(10)

plt.figure(figsize=(10,6))
sns.barplot(x=avg_rating_cuisine.values, y=avg_rating_cuisine.index,palette="bright")
plt.title("Top 10 Cuisines by Average Rating")
plt.xlabel("Average Rating")
plt.ylabel("Cuisine")
plt.show()
```




In [ ]:

```
```

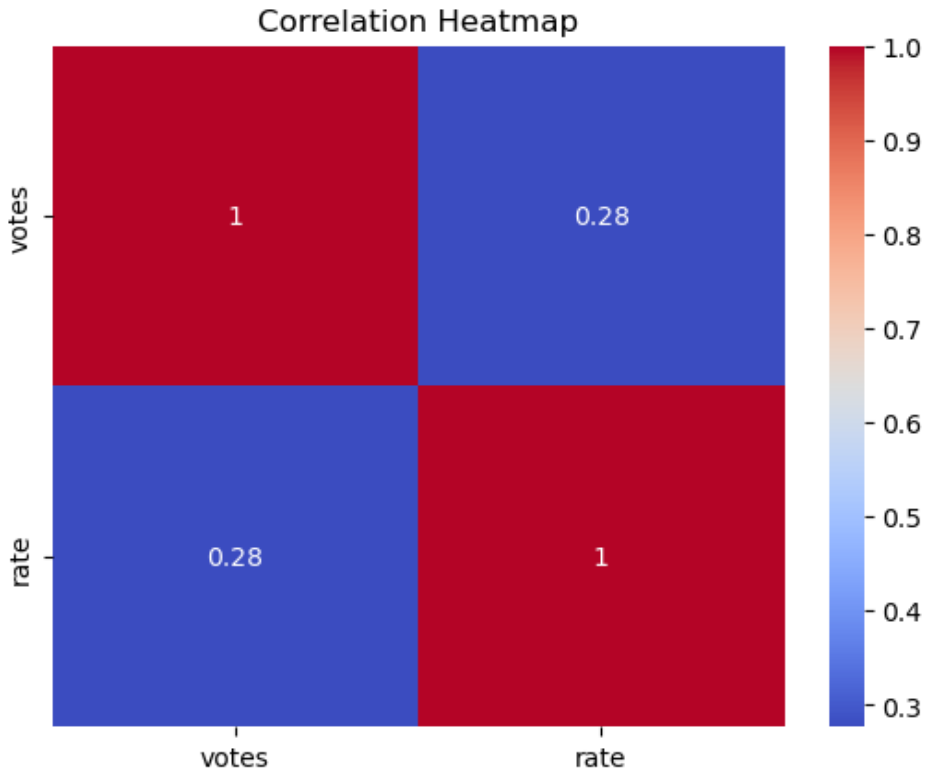
## EDA QUESTION 20] Relationship between votes and rating (Correlation check)

In [215]:

```
print("Correlation : ",df[['votes', 'rate']].corr())
sns.heatmap(df[['votes', 'rate']].corr(), annot=True, cmap="coolwarm")
plt.title("Correlation Heatmap")
```

```
plt.show()
```

```
Correlation :            votes      rate
votes  1.000000  0.276705
rate   0.276705  1.000000
```



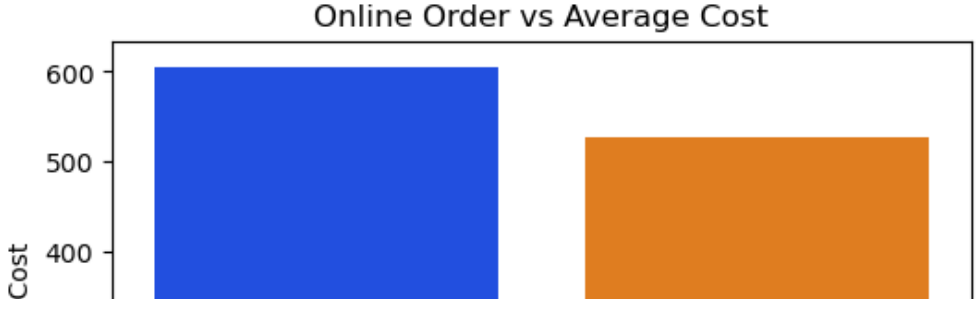

In [ ]:

```
```

## EDA QUESTION 21] Does online ordering impact average cost?

In [216]:

```
avg_cost_online = (
    df.groupby("online_order")["approx_cost(for two people)"]
    .mean()
    .reset_index()
)

plt.figure(figsize=(6,4))
sns.barplot(
    x="online_order",
    y="approx_cost(for two people)",
    data=avg_cost_online,
    palette="bright"
)

plt.title("Online Order vs Average Cost")
plt.xlabel("Online Order")
plt.ylabel("Average Cost")

plt.show()
```



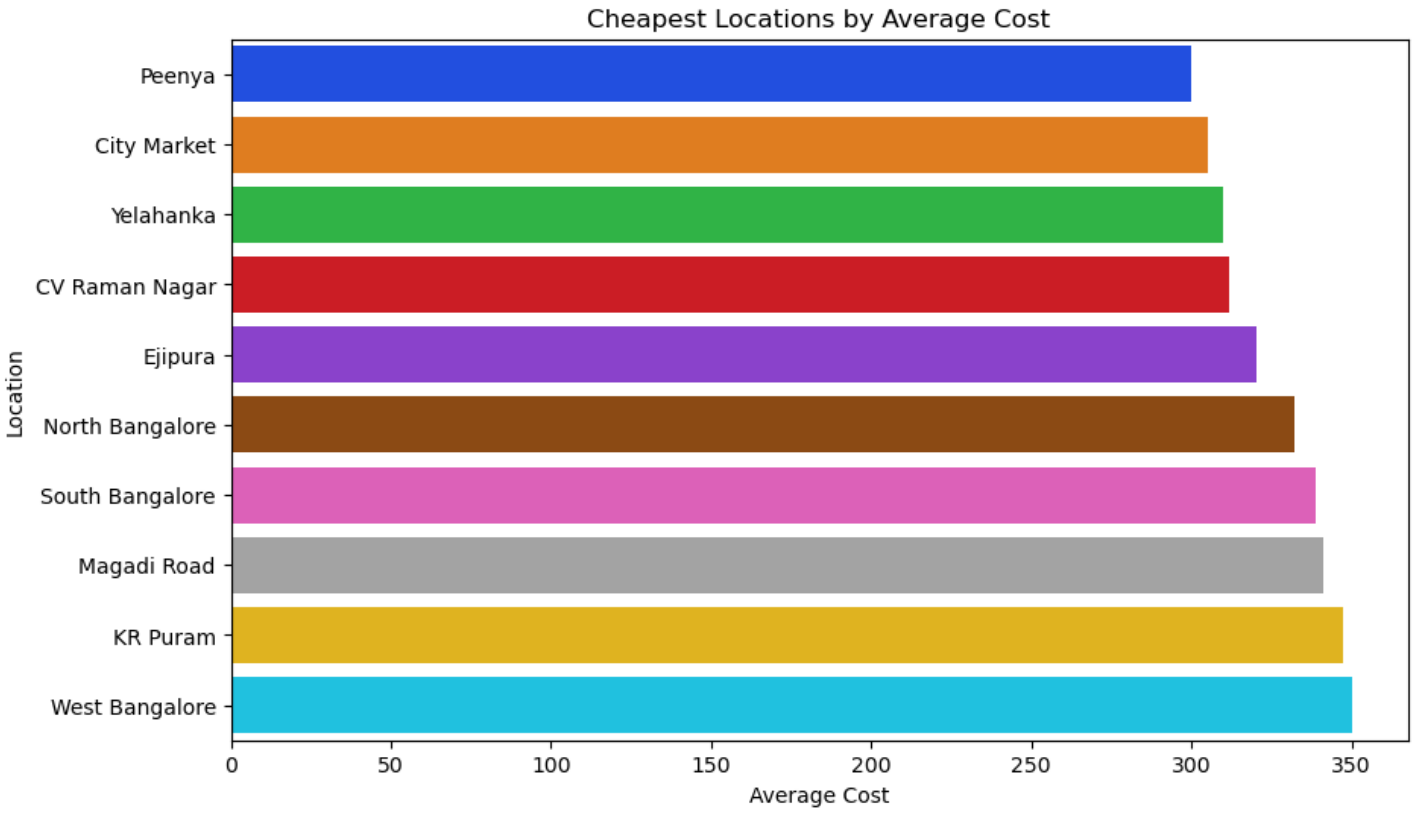

In [ ]:

```
```

### EDA QUESTION 22] Which locations have the cheapest restaurants on average?

In [217]:

```
cheap_locations = df.groupby("location")["approx_cost(for two people)"].mean().sort_valu
es().head(10)

plt.figure(figsize=(10,6))
sns.barplot(x=cheap_locations.values, y=cheap_locations.index,palette="bright")
plt.title("Cheapest Locations by Average Cost")
plt.xlabel("Average Cost")
plt.ylabel("Location")
plt.show()
```




In [ ]:

```
```

## CONCLUSION OF ALL EDA QUESTIONS

- **Most orders were placed from the location BTM.**
- **A majority of customers preferred placing orders online.**
- **Most customers did not book a table before visiting the restaurant**

- Most customers **did not book a table** before visiting the restaurant.
- **Top 5 cuisines preferred by customers:**

    1. North Indian
    2. South Indian
    3. Cafe
    4. Chinese
    5. Biryani
- **Top 5 cuisine categories:**

    1. Delivery
    2. Dine Out
    3. Desserts
    4. Cafes
    5. Drinks & Nightlife
- **Cities from which the most orders were received:**

    1. BTM
    2. HSR
    3. Koramangala 5th Block
    4. JP Nagar
    5. Whitefield
- **Top 5 restaurant types:**

    1. Quick Bites
    2. Casual Dining
    3. Cafe
    4. Dessert Parlor
    5. Delivery
- **Most frequently observed restaurant ratings:**

    1. 0.0
    2. 3.9
    3. 3.8
    4. 3.7
    5. 3.6
- **Locations from where majority of restaurant's ratings were given:**

    1. Koramangala 5th Block
    2. Indiranagar
    3. Koramangala 4th Block
    4. Church Street
    5. JP Nagar
- **Top 5 dishes preferred by customers:**

    1. Burgers
    2. Cocktails
    3. Coffee
    4. Biryani
    5. Pizza
- An **increase in the number of votes** generally resulted in an **increase in restaurant ratings**, showing a positive correlation.
- Most **online orders received ratings between 3 and 4** on average, whereas **offline orders received ratings between 0 and 4.**
- Most orders where tables were booked; recieved ratings between 4 to 4.5.
- Most orders where tables were not booked; recieved ratings between 2.8 to 3.8.
- **Top 10 locations with the highest average ratings:**

    1. St. Marks Road
    2. Church Street
    3. Rajarajeshwari Nagar
    4. Sankey Road
    5. Lavelle Road

6. **Langford Town**
7. **Race Course Road**
8. **Cunningham Road**
9. **Kengeri**
10. **Koramangala 5th Block**

- **Casual Dining restaurants** charged between **₹500 to ₹890**, showing a **large price variation.**
- **Cafe-type restaurants** charged between **₹450 to ₹800**, also indicating a **wide price range.**
- **These are the 5 most expensive restaurant types: a) Fine Dining b) Club c) Lounge d) Microbrewery e) Bar**
- **These are the cuisines have the highest average ratings: a) African b) Malaysian c) Parsi d) Middle Eastern e) Sushi**
- **On average; these locations have the cheapest restaurants: a) Peenya b) City Market c) Yelahanka d) CV Raman Nagar e) Ejipura**

In [ ]:

In [ ]:

In [ ]:

In [ ]:

# MACHINE LEARNING QUESTIONS:

# ML QUESTIONS:

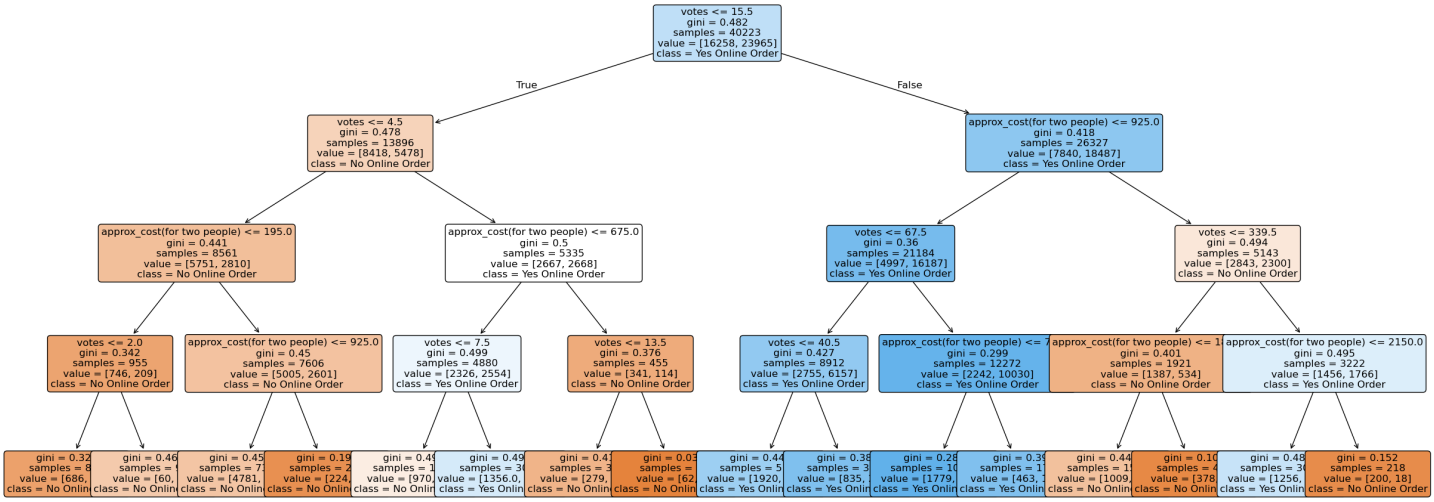
### Q1] Determine whether a person will place an order online based on no. of votes a restaurant has recieved and cost of dining at that restaurant.

In [218]:

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt


# Encode Yes/No to 1/0
df['online_order_encoded'] = df['online_order'].map({'Yes': 1, 'No': 0})

# Features chosen
X = df[['votes', 'approx_cost(for two people)']]
y = df['online_order_encoded']


X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)


model = DecisionTreeClassifier(max_depth=4, random_state=42)
model.fit(X_train, y_train)


y_pred = model.predict(X_test)
```

```
acc = accuracy_score(y_test, y_pred)
print("Decision Tree Accuracy:", acc)


plt.figure(figsize=(30, 12))
plot_tree(
    model,
    feature_names=['votes', 'approx_cost(for two people)'],
    class_names=['No Online Order', 'Yes Online Order'],
    filled=True,
    rounded=True,
    fontsize=12
)
plt.show()
```

```
Decision Tree Accuracy: 0.7052505966587113
```



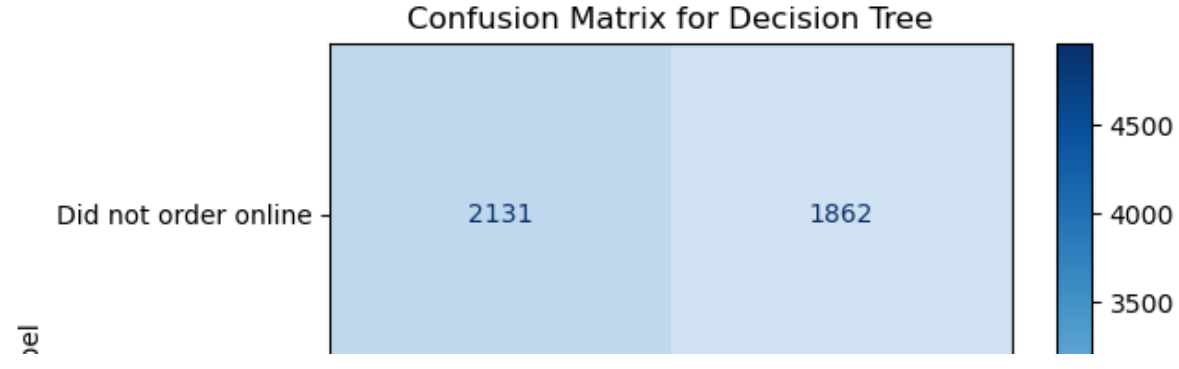

## Conclusion at this point:

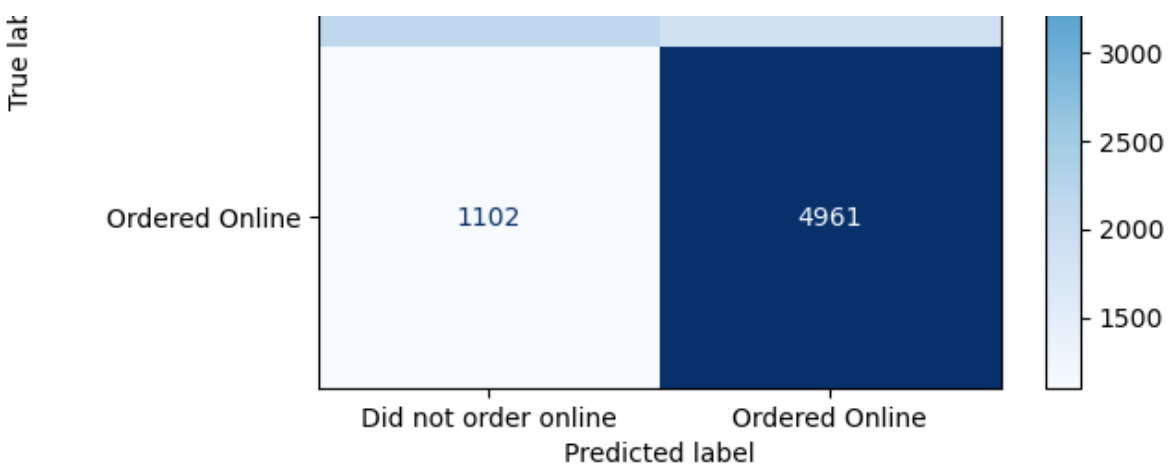
**This decision tree tries to predict that a person will place an online-order or not based on his'/her's trust (i.e. votes) and affordablity.**

- **If votes are low, customers usually avoid online orders unless the cost is very reasonable.**
- **If votes are high, customers are more willing to order online, but extremely high prices reduce this likelihood.**

In [219]:

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

target_names = ['Did not order online', 'Ordered Online']

cm = confusion_matrix(y_test, y_pred)

disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=target_names)
disp.plot(cmap=plt.cm.Blues)
plt.title("Confusion Matrix for Decision Tree")
plt.show()
```






## CONCLUSION AT THIS POINT:

- **Correctly predicted an order to be online = 4961**
- **Correctly predicted an order to be offline = 2131**
- **Incorrect predictions for online-order = 1862**
- **Incorrect predictions for offline-order = 1102**

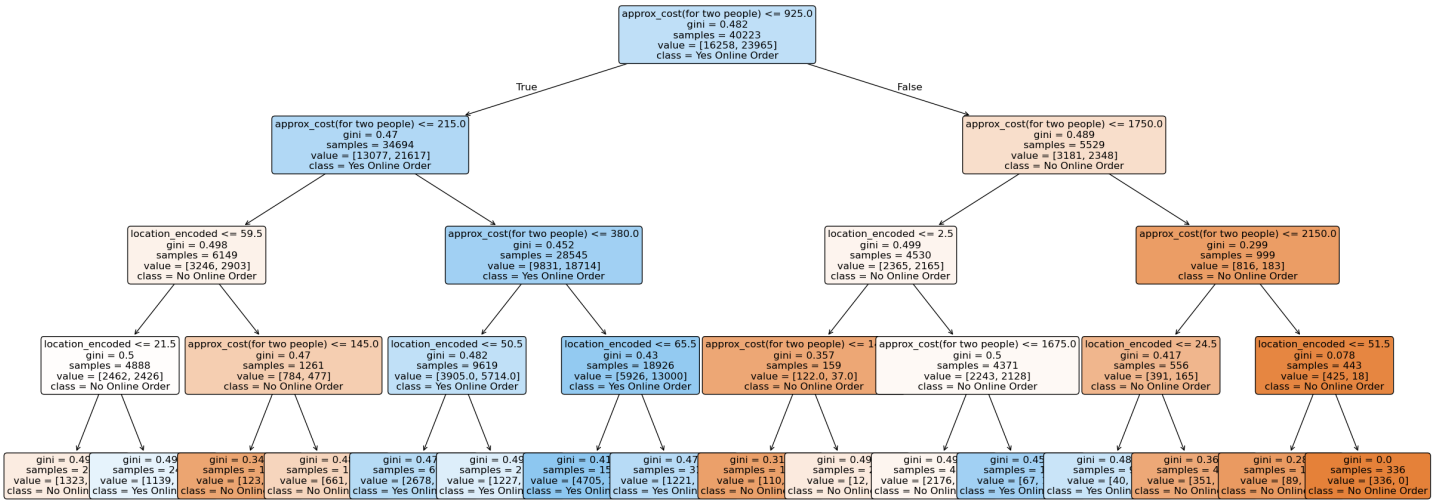
## Q2] Determine whether a person will place an order online based on location of the restaurant and cost of dining at that restaurant.

In [220]:

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
df['location_encoded'] = le.fit_transform(df['location'])


# Features chosen
X = df[['location_encoded', 'approx_cost(for two people)']]
y = df['online_order_encoded']


X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)


model = DecisionTreeClassifier(max_depth=4, random_state=42)
model.fit(X_train, y_train)


y_pred = model.predict(X_test)
acc = accuracy_score(y_test, y_pred)
print("Decision Tree Accuracy:", acc)


plt.figure(figsize=(30, 12))
plot_tree(
    model,
    feature_names=['location_encoded', 'approx_cost(for two people)'],
    class_names=['No Online Order', 'Yes Online Order'],
    filled=True,
    rounded=True,
    fontsize=12
```

```
)
plt.show()
```

```
Decision Tree Accuracy: 0.6228122513922036
```



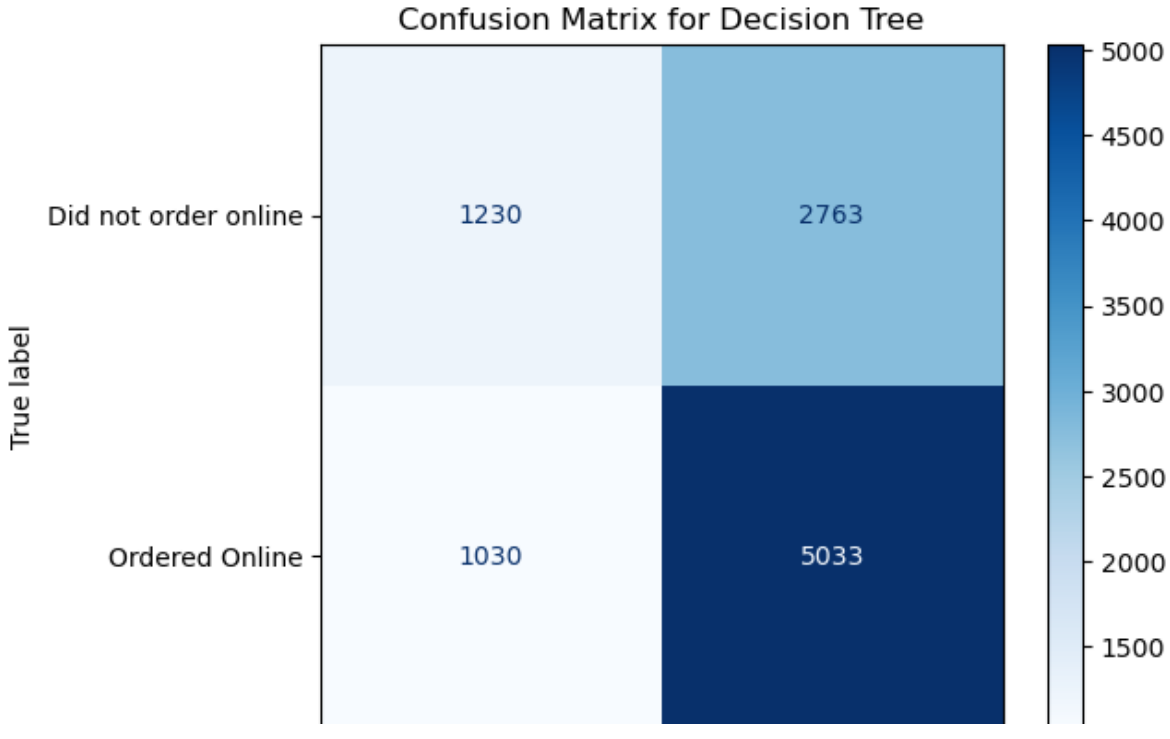

## Conclusion at this point:

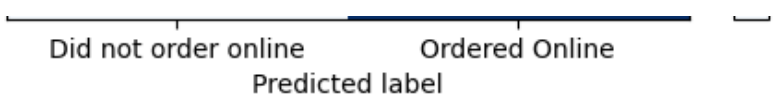
**This decision tree tries to predict that a person will place an online-order or not based on the restaurant's location and affordablity.**

- **For higher cost (more than 2150 INR) people will not prefer placing an online-order (irrespective of the location).**
- **For lower cost; people may prefer online-order.**

In [221]:

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

target_names = ['Did not order online', 'Ordered Online']

cm = confusion_matrix(y_test, y_pred)

disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=target_names)
disp.plot(cmap=plt.cm.Blues)
plt.title("Confusion Matrix for Decision Tree")
plt.show()
```






## CONCLUSION AT THIS POINT:

- **Correctly predicted an order to be online = 5033**
- **Correctly predicted an order to be not online = 1230**
- **Incorrect predictions for online-order = 2763**
- **Incorrect predictions for offline-order = 1030**

In [ ]:

```
```

## Q3] Build a Logistic Regression model to predict whether a restaurant offers online ordering (Yes/No) based on its rating, number of votes, and approximate cost for two people.

In [222]:

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

# Features and target
X = df[["rate", "votes", "approx_cost(for two people)"]]
y = df["online_order_encoded"]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Scaling (important for Logistic Regression)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Model
logr = LogisticRegression(max_iter=1000)
logr.fit(X_train_scaled, y_train)

# Model accuracy
y_pred = logr.predict(X_test_scaled)
print("Accuracy:", accuracy_score(y_test, y_pred))

# Prediction for a new restaurant
sample = scaler.transform([[4.1, 787, 800]])
result = logr.predict(sample)

if result[0] == 1:
    print("Restaurant offers online ordering")
else:
    print("Restaurant does not offer online ordering")
```

```
Accuracy: 0.6828758949880668
Restaurant offers online ordering
```

## CONCLUSION AT THIS POINT:

**Restuarant (with the below details) will get an online-order:**

- **Rating = 4.1**
- **Votes received = 787**

- **Dining cost = 400**

### Q4] Build a Linear Regression model to predict the approximate cost for one person at a restaurant based on its rating and number of votes.

In [223]:

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np

# Features and target
X = df[["rate", "votes"]]
y = df["approx_cost(for two people)"]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Model
reg = LinearRegression()
reg.fit(X_train, y_train)

# Predictions on test data
y_pred = reg.predict(X_test)

# Evaluation
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print("RMSE:", rmse)
print("R² Score:", r2)

# Example prediction (cost for one person)
predicted_cost_for_two = reg.predict([[3.8, 918]])[0]
print("APPROXIMATE COST OF DINING (one person) =", predicted_cost_for_two / 2)
```

```
RMSE: 393.7772517829144
R² Score: 0.19353077681380482
APPROXIMATE COST OF DINING (one person) = 359.79085202854253
```

## CONCLUSION AT THIS POINT:

**Cost of dining for one person is Rs.360 at a restaurant with these details:**

- **Restaurant name = San Churro Cafe**
- **Average Rating = 3.8**
- **Votes = 918**

In [ ]:

```

```

In [ ]:

```

```

## ZOMATO BUSINESS INSIGHTS:

**This analysis brings to light some very informing patterns on restaurant distribution, customer preferences, and pricing across neighborhoods in Bangalore. Cleaning the data guaranteed reliable insights from 50,279 records after handling missing values in ratings, phone numbers, and other fields.**

**Top Locations:**

**Top Locations:**

**BTM heads the list in terms of the number of restaurants and orders, followed by HSR, Koramangala 5th Block, JP Nagar, and Whitefield.**

**Customer Preferences:**

**Food preferences include North Indian, South Indian, Cafe, Chinese and Biryani. Popular dishes include Burgers, Biryani, Pizza, Coffee, and Cocktails. Most customers order online but do not book tables; the rating is 3 to 4 for ordering online and 0 to 4 for ordering offline.**

**Pricing and Types Casual Dining:** *≈₹ 500-890 for two; Cafe: ≈₹ 450-800*

**Top 5 most expensive restaurant types are: Fine Dining, Club, Lounge, Microbrewery and Bar. Top 5 places with cheapest dining-cost (on average) are: Peenya, City Market, Yelahanka, CV Raman Nagar and Ejipura.**

In [ ]:

In [ ]:

In [ ]:

**ABOUT ME:**

**Name:** Om Satyawan Pathak
**Email:** omsatyawanpathakwebdevelopment@gmail.com
**Github:** https://github.com/omsatyawanpathakgit
**Linkedin:** www.linkedin.com/in/om-satyawan-pathak-029b02368